

FUJINET

FOR THE NINTENDO ENTERTAINMENT SYSTEM

PROGRAMMER'S MANUAL



Look for this seal on every program in this manual. It is the FujiNet project's assurance that what you are about to read was built and run, not merely written: every listing assembled or compiled, every transaction made against a live FujiNet, every screen in these pages taken from an emulated NES that was talking to it at the time.



INTRODUCTION

Thank you for selecting the FujiNet cartridge for the Nintendo Entertainment System. This manual requires cc65, the 6502 compiler suite, and a little patience with a video chip that only listens during vblank.

OBJECT OF THIS MANUAL / MANUAL DESCRIPTION

In this manual you will learn to drive the FujiNet from your own NES programs, in 6502 assembly and in C. You will open network connections, read web pages and JSON, list directories on the network, save data in app keys, read the clock, and load and run new cartridge images straight off the network. Then you will take apart four real network games and the CONFIG program, to see how the pieces fit, and finish with a terminal of your own.

Please read this manual to ensure proper handling of your new cartridge, and then save it for future reference.

CAUTION: Misuse of a read-modify-write instruction on the mailbox can cause spurious transactions.

Read Chapter 4 BEFORE writing to addresses \$5500-\$57FF.

Please follow the instructions in Chapter 3 to properly set up your tools, your emulator and your FujiNet.

FujiNet Project
2026 FujiNet contributors. GPL v3.

Nintendo and Nintendo Entertainment System are trademarks of Nintendo. This manual and the FujiNet project are not affiliated with, endorsed by or sponsored by Nintendo. The cover and page design are an affectionate tribute to the 1985 instruction booklets.

CONTENTS

1. INTRODUCTION	7	7. THE FUJI DEVICE	50
What you can do	7	Hosts, devices and paths	50
What you need	7	Listing a directory	50
How this manual is arranged	8	App keys	53
The two roads	8	WiFi	56
Conventions	8	Hosts and device slots	58
2. THE CARTRIDGE	10	Directories	66
The design in one picture	10	App key commands	68
The memory map	11	The adapter	70
Power on	11	Jobs for the coprocessor	72
The claim	12	8. BOOTING AN IMAGE	76
Mappers	12	Four steps	76
What is not finished	13	In C	78
3. SETTING UP YOUR TOOLS	14	In assembly, with progress	79
The compiler	14	The command	82
fujinet-lib	14	Which files are images	82
This manual's programs	14	When it fails	82
checkrom.py	16	Inside the loader	83
A FujiNet to talk to	16	Coming back	84
An NES with the cartridge in it	16	9. THE CLOCK AND OTHER DEVICES	85
4. THE CARTRIDGE MAILBOX	18	What time is it?	85
Two directions	18	The clock device	87
The one rule	18	Disks	89
The status page	19	The printer	91
The registers	20	The modem	91
The TX stream	21	10. THE LIBRARIES	92
Committing a transaction	21	fujinet-lib on the NES	92
Where the sequence number comes from	23	The capped calls	94
The reply window	24	The NES's own calls	94
What can go wrong	24	Keyboards	95
How long to wait	24	Building a transaction by hand	95
The NMI	25	Building with the library's makefiles	96
5. YOUR FIRST TRANSACTION	26	11. PICTURES, PADS AND SOUND	98
In C	27	The PPU's rule	98
In assembly	28	cc65's queue	98
What crossed the mailbox	30	Pattern tables and the claim on CHR	99
Press Reset	31	The cartridge's work RAM	99
The assembly library	31	The controller	100
6. THE NETWORK DEVICE	33	Typing without a keyboard	101
The life of a connection	33	Sound	101
Reading a file	33	12. GAME ONE: 5 CARD STUD	102
A JSON round trip	36	The shape of the client	102
Command reference	39	Memory	103

One request, one GET	104	C. PROGRAM LISTINGS	132
What comes back	105	This manual's programs	132
Drawing without a frame buffer	106	5 Card Stud	141
Your turn	107	Battleship	148
The server	107	Fujitzee	153
Leaving for the Lobby	107	Texas Hold'em	157
13. GAME TWO: BATTLESHIP	109	D. NETCAT	159
What is different here	109	Using it	159
One file for the FujiNet	110	How it works	160
Wire format	111	Building it	161
A whole screen at once	111	Listings	161
Waiting for a frame	112		
Buttons	112		
The server	113		
14. GAME THREE: FUJITZEE	114		
Requests and replies	114		
The active column	115		
Sprites for the cursor	116		
The keyboard as a key source	116		
One player	117		
The server	117		
15. GAME FOUR: TEXAS HOLD'EM	118		
A chapter of differences	118		
Not redrawing what has not changed	119		
The wire	120		
The server's bots	120		
16. CONFIG AND THE LOBBY	121		
Where it lives	121		
The screens	122		
Paging a directory	123		
Booting, with a bar	124		
Typing	125		
The Lobby	125		
A. ERROR CODES	126		
The mailbox	126		
The libraries	126		
Booting	127		
The network device	127		
B. QUICK REFERENCE	129		
A transaction	129		
Devices	129		
Network commands	129		
Fuji commands	130		
Clock commands	130		
Sizes	131		
Programs in this manual	131		

PRECAUTIONS

This is a high precision cartridge. It has a few rules of its own, and a program that breaks one of them fails in ways that are hard to see. Every rule here is explained in the chapter shown.

- 1) **Never use a read-modify-write instruction on \$5500-\$57FF.** INC, DEC, ASL, LSR, ROL and ROR write the old value back first, and on these pages every write is an event. Use STA, STX and STY only. In C, never ++ or |= a mailbox address. (Chapter 4)
- 2) **Take the next sequence number from the cartridge.** It is the cartridge's ACKSEQ plus one, skipping zero – never a counter of your own. Reset restarts your program but not the cartridge. (Chapter 4)
- 3) **Claim the mailbox.** Put "FUJI" at \$FFF0, the last 16 bytes of PRG, or the cartridge shuts the mailbox off the moment your program boots. (Chapter 4)
- 4) **Leave zero page \$E0-\$EF to the assembly library,** and \$0200-\$04FF to cc65's PPU queue. (Chapters 5 and 10)
- 5) **Touch the PPU only in vblank or with rendering off.** The NMI may land in the middle of a transaction; that is fine, as long as the NMI handler never writes to the mailbox. (Chapter 11)
- 6) **Read the reply where it lies.** The 1K at \$5000 holds still until your next commit, so copy from it, draw from it, or stream it back out – but do it before you start the next transaction. (Chapter 4)
- 7) **Give MOUNT_IMAGE a full minute.** It does not answer until the whole image has crossed into the cartridge. (Chapter 8)
- 8) **Send the full size** of SET_SSID, OPEN_APPKEY and the slot tables, and every parameter a command takes: a missing parameter does not earn a NAK, it restarts the FujiNet. (Chapter 7)

1. INTRODUCTION

The FujiNet cartridge gives the Nintendo Entertainment System a network adapter, a file server, a clock and a few hundred kilobytes of somebody else's memory. This manual shows you how to use all of them from your own programs.

WHAT YOU CAN DO

Everything the FujiNet does for the Atari, the Apple II or the Intellivision, it does for the NES. From a program running on the console you can:

- * open a connection to any server on the Internet – HTTP and HTTPS, TCP, UDP, Telnet, TNFS, FTP, SMB and more – and read and write it a byte at a time;
- * fetch a JSON document and pull single fields out of it, with the FujiNet doing the parsing;
- * list the files on the SD card or on a network host, and load any NES cartridge image you find there into the cartridge's own memory and run it;
- * keep a few bytes of your own – a high score, a player name, a server address – in an **app key** that survives a power cycle;
- * read the time of day, generate a random number or a GUID, hash a block of data or turn it into a QR code;
- * type on a Family BASIC or Subor keyboard, if you have one.

WHAT YOU NEED

1. **A FujiNet NES cartridge:** – or something that behaves like one. At the time of writing the cartridge exists as firmware, a schematic and two emulators; no board has been built. Every program in this manual was run in an emulated NES talking to a real FujiNet firmware, and every screen shown was taken from it.
2. **cc65:** the 6502 C compiler, assembler and linker, version 2.19.
3. **fujinet-lib:** for the C programs: the add-nes branch of fujinet-lib-experimental, which carries the NES target.
4. **A FujiNet to talk to:** – fujinet-pc, the FujiNet firmware built for a PC, or the one built into FujiNet Go NES Desktop.

Chapter 3 shows how to put them together.

HOW THIS MANUAL IS ARRANGED

Chapters 2 to 5 are the foundation: what the cartridge is, how to build a program, the **mailbox** through which every byte passes, and a first program that talks to the FujiNet. Read them in order.

Chapters 6 to 9 take the FujiNet's devices one at a time – the network, the Fuji device that manages WiFi, hosts, disks and app keys, the business of booting an image, and the clock. Each command is described on a **command card** with the parameters it takes and the reply it sends, followed by the same call in assembly and in C.

Chapter 10 describes the two libraries, and Chapter 11 the things about the NES itself – the PPU, the vblank NMI and the controllers – that every network program runs into.

Chapters 12 to 16 take apart real programs: the four FujiNet network games that have been ported to the NES, and CONFIG, the program the cartridge runs when you turn the console on. The appendices hold the error codes, a quick reference, the full listings with numbered callouts, and a network terminal to finish with.

THE TWO ROADS

Every example in this manual is given twice, side by side:

6502 ASSEMBLY (CA65)

```
; 6502 assembly, ca65
CALL FNDEVF, FNCADPX, 0
jsr    fn_go
jne    fail
```

C (CC65 + FUJINET-LIB)

```
/* C, cc65 and fujinet-lib */
if (!fuji_get_adapter_config_extended(&acx))
    fail();
```

The assembly road talks to the mailbox directly, through a short library of store-and-wait routines; you see every byte that goes to the cartridge. The C road uses fujinet-lib, which hides the mailbox behind the same `fuji_` and `network_` calls a FujiNet program uses on every other computer, so the code moves between platforms. Neither road is the "real" one. The assembly road is how the C library works inside.

CONVENTIONS

Numbers written with a \$ are hexadecimal: \$5000 is 20480. Register numbers, command numbers and device numbers are always given in hexadecimal, as the firmware's own sources give them.

CAUTION: marks something that will cost you an afternoon if you get it wrong.

NOTE: marks something worth knowing that will not hurt you.

The programs in Chapters 5 to 9 are in the `listings/` directory beside this manual, in `asm/` and `c/`, with a Makefile that builds all of them.

2. THE CARTRIDGE

Most cartridges that load games from somewhere else put a programmable logic chip between the NES and its memory. This one puts real memory there, and lets a microcontroller watch.

THE DESIGN IN ONE PICTURE

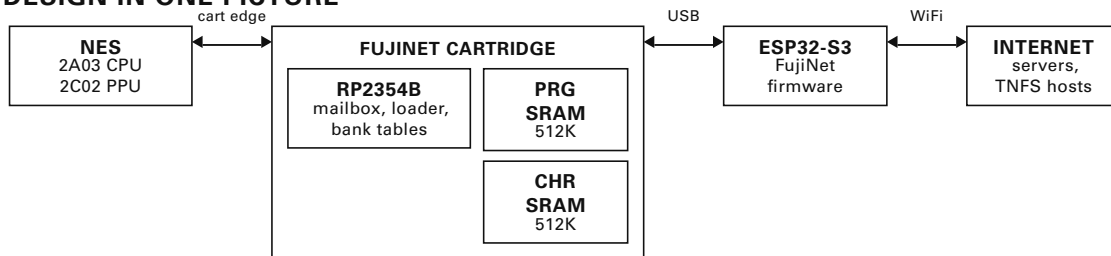


Fig. 1 – The FujiNet NES cartridge and the FujiNet it talks to.

The cartridge holds three chips that matter to a programmer:

1. **Two 512K static RAMs:** stand in for the PRG ROM and the CHR ROM of an ordinary cartridge. The console addresses them itself, at full speed, so a game running from them cannot tell they are not ROM.
2. **An RP2354B microcontroller:** watches every write the CPU makes, and when a write lands on a mapper register it changes the high address lines of the two RAMs, so the cartridge banks the way the game's original board did. It also serves 4K of its own memory at \$5000 – the **mailbox** and a small loader – and 8K of work RAM at \$6000.
3. **A USB link:** to an ESP32-S3 running the FujiNet firmware. The RP2354B turns what your program writes into the mailbox into FujiBus packets, sends them over USB, and paints the answer back into the mailbox.

The design follows the memory architecture of the original EverDrive N8 – two SRAMs behind the cartridge edge – with the RP2354B's programmable I/O state machines doing the banking. Each bank table is a handful of instructions that the mapper code rewrites in one store; it takes about 45 ns for a bank change to reach the RAMs, against 372 ns for a PPU memory access.

THE MEMORY MAP

This is what your program sees between \$4020 and \$FFFF:

\$5000	REPLY WINDOW	1K: the last reply, cart-painted
\$5400	STATUS PAGE	ACKSEQ, errors, boot progress
\$5500	REGISTERS	STA \$5500+n sets register n
\$5600	(RAW REGDATA)	not used by programs
\$5700	TX STREAM	any store appends one byte
\$5800	LOADER ROM	2K, cart-served
\$6000	WORK RAM	8K, cart-served
\$8000	PRG SRAM	your program, through the mapper
\$10000		

Fig. 2 – The cartridge's half of the CPU address space.

Below \$5000 the cartridge answers nothing: \$4020–\$4FFF is open bus. The pages from \$5000 to \$57FF are the mailbox, described in Chapter 4. The cartridge also serves the reset vectors at \$FF00–\$FFFF while its SRAMs are switched off, which is how it starts.

POWER ON

When the console powers up the SRAMs are empty and switched off. The RP2354B answers the CPU's reset-vector fetch itself, pointing it at the **loader ROM** at \$5800. The loader copies CONFIG – a 32K NES program baked into the cartridge's flash – into the SRAMs, 1K at a time, and jumps to it. This takes about a sixth of a second.

CONFIG is the cartridge's menu (Chapter 16). When you pick a game, CONFIG asks the FujiNet to fetch the image; the FujiNet pushes it over USB into a staging store on the cartridge; and the same loader copies it into the SRAMs and starts it – about 9.5 CPU cycles a byte, so a 512K-plus-256K game takes four seconds. Chapter 8 shows how to do this from your own program.

THE CLAIM

A commercial game knows nothing about the mailbox, and may well have its own use for \$5000-\$5FFF. So after the loader starts an image, the cartridge looks for a four-byte signature, FUJI, in the last 16 bytes of its PRG – console address \$FFF0, just below the vectors.

CAUTION: An image without the claim gets no mailbox. The cartridge stops answering \$5000-\$57FF for the rest of the session (the loader at \$5800 stays), and every transaction your program tries simply never completes. Chapter 3 shows the linker configuration that reserves the space, and the tool that writes it.

The same thing happens to an image whose mapper decodes \$5000-\$5FFF itself, since the two cannot share the bus.

MAPPERS

A FujiNet program can use any mapper the cartridge implements. The simplest, and the one every program in this manual uses, is mapper 0, NROM: 32K of PRG at \$8000-\$FFFF and 8K of CHR, no banking at all.

No.	Board	Notes
0	NROM	32K PRG, 8K CHR ROM or RAM
1	MMC1 (SxROM)	with the consecutive-write rule, and SUROM's 512K
2	UxROM	16K switchable + 16K fixed
3	CNROM	8K CHR banks
4	MMC3 (TxROM)	with the scanline IRQ
7	AxROM	32K banks, one-screen mirroring
11	Color Dreams	
30	UNROM 512	
34	BNROM / NINA-001	
66	GxROM	
71	Camerica / BF9097	
206	Namco 108	

The SRAMs set the ceiling: 512K of PRG, 512K of CHR, and up to 32K of work RAM. What the design cannot do it says plainly: the MMC5's extended attributes and split screen and the MMC2 and MMC4 tile latches

need the PPU address bus at full speed, which this cartridge does not carry; a 72-pin NES has no path for expansion audio; and the lockout chip is an external clone.

WHAT IS NOT FINISHED

- * No cartridge has been built. The schematic exists and passes its checks; the bus timing is still a datasheet claim until a scope says otherwise.
- * Pressing Reset while an image is loading restarts the CPU through half-written vectors. Turn the power off and on; the cartridge reloads CONFIG.
- * Battery-backed saves live in the cartridge's work RAM and are lost at power off. Use an app key (Chapter 7) for anything that must be kept.
- * Four-screen mirroring is not yet wired.

3. SETTING UP YOUR TOOLS

A FujiNet NES program is an ordinary NES program with three extra facts: where the mailbox is, a reserved place for the claim, and a check that nothing in it breaks the mailbox's one rule.

THE COMPILER

Install cc65 2.19. You need ca65 and ld65 for assembly, and cl65, which drives the whole chain, for C. Everything in this manual is built with `-t nes`, cc65's NES target, or `-t none` for plain assembly.

FUJINET-LIB

The C programs link against fujinet-lib. Its NES target lives on the add-nes branch of fujinet-lib-experimental:

BUILDING THE LIBRARY

```
git clone -b add-nes https://github.com/FujiNetWIFI/fujinet-lib-experimental
cd fujinet-lib-experimental
make nes # -> r2r/nes/fujinet.nes.lib
```

Besides the library it provides three files every NES program needs:

1. **makefiles/nes-fujinet.cfg**: cc65's stock NES linker configuration – NROM, 32K PRG, 8K CHR ROM, the cartridge's 8K work RAM at \$6000 – with \$FFF0-\$FFF9 carved out for the claim.
2. **makefiles/nes-romstamp.py**: run after the link: it writes FUJI at \$FFF0 and refuses an image that has a read-modify-write instruction aimed at the mailbox.
3. **makefiles/platforms/nes.mk**: does both for you if your project builds with the library's makefiles.

THIS MANUAL'S PROGRAMS

The `listings/` directory holds every program in Chapters 5 to 9 and the netcat of Appendix D. `make` builds them all into `listings/build/`:

LISTINGS/MAKEFILE, THE TWO RECIPES

```
build/c/%.nes: c/%.c
cl65 -t nes -O -I $(FUJINET_LIB)/include -C $(NESCFCG) \
-m build/c/%*.map -o $@ $< $(LIBFILE)
python3 $(STAMP) --stamp --map build/c/%*.map $@
python3 common/checkrom.py --claim --map build/c/%*.map $@

build/asm/%.nes: build/asm/%.o $(COMMON_O) common/nes.cfg
ld65 -C common/nes.cfg -m build/asm/%*.map -o $@ $< $(COMMON_O)
python3 common/checkrom.py --claim --map build/asm/%*.map $@
```

The assembly programs share five files in listings/common/:

File	What it holds
fujinet.inc	The mailbox's addresses and registers, and the library's zero page. Copied unchanged from the cartridge firmware's own test programs.
fujidefs.inc	Every other device and command number, from the firmware's headers.
fujilib.s	The mailbox library: begin, append, commit, wait. Also copied unchanged.
booklib.s	What this manual adds: a vblank NMI, a non-waiting commit, an unpadding string append, a number printer.
nesinit.s, fujidisp.s	The iNES header, the claim, the vectors, the cold start; a font in CHR RAM and a few text routines.
nes.cfg	The linker configuration: NROM-256, CHR RAM, the claim at \$FFF0.

LISTINGS/COMMON/NES.CFG

```
MEMORY {
    ZP:      start = $0000, size = $00E0, type = rw, define = yes;
    RAM:     start = $0300, size = $0500, type = rw, define = yes;
    WRAM:    start = $6000, size = $2000, type = rw, define = yes;
    HEADER:  start = $0000, size = $0010, file = %0, fill = yes;
    PRG:     start = $8000, size = $7FF0, file = %0, fill = yes, fillval = $FF;
    CLAIM:   start = $FFF0, size = $000A, file = %0, fill = yes, fillval = $FF;
    VECTORS: start = $FFFA, size = $0006, file = %0, fill = yes;
}
```

Notice two things. Zero page stops at \$E0, because the mailbox library owns \$E0-\$EF. And PRG stops at \$FFF0, ten bytes short of the vectors: the CLAIM segment, which nesinit.s fills with FUJI, sits there.

CHECKROM.PY

checkrom.py comes from the cartridge firmware's tools. It checks the iNES header the way the cartridge parses it, that the claim is there, that the reset vector lands in \$8000-\$FFFF, and – walking only the code segments named in the linker map – that no read-modify-write instruction is aimed at \$5500-\$57FF.

A CLEAN BUILD

```
$ make -C listings
...
build/c/hello.nes: ok
checkrom: build/c/hello.nes: ok
checkrom: build/asm/hello.nes: ok
```

A FUJINET TO TALK TO

The programs need a FujiNet on the other end of the cartridge's USB link. On a PC that is **fujinet-pc**, the FujiNet firmware built for Linux, macOS or Windows with the RS-232 bus. It listens for a cartridge on a TCP port – **bus over IP**, 127.0.0.1:9995 by default – and serves its web configuration page on another. Its SD card is a directory beside it.

AN NES WITH THE CARTRIDGE IN IT

There are two emulated NES machines with the FujiNet cartridge built in.

MAME, grafted. The cartridge firmware ships a MAME device, `nes_fujinet`, that compiles the cartridge's own mailbox, FujiBus and mapper sources into MAME and connects to fujinet-pc over bus-over-IP. Apply it once and rebuild:

GRAFTING THE CARTRIDGE INTO MAME

```
cd fujinet-firmware/pico/nes
./emu/apply.sh ~/mame           # copies the device into src/devices/bus/nes
cd ~/mame && make REGENIE=1
./mame nes -nes_slot fujinet -cart hello.nes
```

MAME's NES driver insists on a `-cart`; the device treats it as the image the cartridge would have loaded. This manual's `emu/run.sh` wraps that line, takes a screenshot after a few seconds, and prints the screen as text; its `emu/drive.lua` presses buttons on a script. Every screen in this manual came out of those two.

FujiNet Go NES Desktop. A self-contained NES – the MesenCE emulator core, with the cartridge modelled from the same firmware sources and fujinet-pc built in. Its FujiNet listens on port 11506 and serves its

configuration page at <http://localhost:11507>. Use **Open Cartridge...** to run your .nes in place of CONFIG, or **Import Cartridge to SD...** to copy it to the FujiNet's SD card and boot it from CONFIG, which exercises the real network load. **Reset Game** presses the console's Reset; **Reset to CONFIG** is a power cycle. Its debugger loads ld65 label files, with the mailbox registers already named.

NOTE: Reset Game resets the console but not the cartridge, just as on real hardware. It is the quickest way to test that your program takes its sequence numbers from the cartridge (Chapter 4).

4. THE CARTRIDGE MAILBOX

Everything your program says to the FujiNet, and everything the FujiNet says back, passes through 2K of addresses at \$5000. The console writes into some of them; the cartridge paints the others.

TWO DIRECTIONS

The cartridge edge carries the CPU's R/W line, so the mailbox can tell a read from a write, and it uses that:

1. **Console to cartridge:** is always a **write**, to one of three write-only pages. Reading those pages does nothing at all.
2. **Cartridge to console:** is always memory the cartridge has **painted**: the 1K reply window and the status page. Your program reads them like RAM.

Address	Direction	What it is
\$5000-\$53FF	read	The reply window: the whole of the last reply, up to 1024 bytes.
\$5400-\$54FF	read	The status page: sequence, error, length, boot progress.
\$5500-\$557F	write	Registers. A store of value V to \$5500+n sets register n to V.
\$5580-\$55FF	write	One-shot operations (only \$55FE, the loader's).
\$5600-\$56FF	write	Raw REGDATA, kept for the firmware's shared decoder. Programs never use it.
\$5700-\$57FF	write	The TX stream. A store anywhere in the page appends one byte.

THE ONE RULE

CAUTION: Only STA, STX and STY may touch \$5500-\$57FF, plain or indexed. Never a read-modify-write instruction.

INC, DEC, ASL, LSR, ROL and ROR with an absolute address – and the undocumented SLO, RLA, SRE, RRA, DCP and ISC – read the location, write the **old** value back, and then write the new one, on consecutive cycles. On a page where every write is an event, the old value lands as an event of its own: an INC \$5510 would launch two transactions.

An indexed store is safe. STA \$5700,X spends an extra cycle reading the target address before it writes, but reads of these pages are inert.

Three things stand guard. `checkrom.py` refuses an image with such an instruction aimed at the mailbox. `fujinet-lib's nes-romstamp.py` does the same. And the cartridge itself recognises the exact signature – two writes to one address on adjacent cycles – drops the first, and counts it in `DIAG_RMW` at \$5413, so a program can show the count on screen while you look for the culprit.

In C, the rule means never `++`, `--`, `|=` or the like on a mailbox address. `fujinet-lib` keeps every mailbox store inside two tiny functions, `fn_regwr()` and `fn_tx()`, so the compiler can only ever emit a plain `sta`.

THE STATUS PAGE

The cartridge paints these cells. Read them whenever you like.

Address	Name	Meaning
\$5400	ACKSEQ	The sequence number of the last transaction answered. Painted last .
\$5401	STATUS	Bit 0: the USB link to the FujiNet is up. Bit 1: busy.
\$5402	ERR	How the last transaction went: 0 OK, 1 no link, 2 timeout, 3 bad frame, 4 too big.
\$5403	REPLY_CMD	\$06 ACK, the FujiNet said yes; \$15 NAK, it said no.
\$5404-\$5405	RXLEN	Length of the reply in the window, low byte first.
\$5406	BOOT_STATE	0 idle, 1 transferring, 2 ready, \$80 failed (Chapter 8).
\$5407	BOOT_PCT	0-100 while an image is pushed.
\$5408	BOOT_ERR	Why a load failed: 1 too big, 2 truncated, 3 no such mapper, 4 store busy.
\$5409-\$540A	MAGIC	'F', 'N': a FujiNet cartridge is present.
\$540B	PROTO_VER	The mailbox protocol version, 1.
\$540C	SLICE_ECHO	Always 0: the reply is never sliced on the NES.
\$540D-\$5411	LOAD_*	The loader's half: state, destination, offset, sequence, percent.
\$5412	SRAM_STATE	1 once the SRAMs are switched on.
\$5413	DIAG_RMW	How many read-modify-write dummy writes the cartridge has dropped.
\$5414	MAPPER	The mapper number of the running image, low byte.
\$5415	LINK	1 once the FujiNet has ever been seen.
\$5416-\$5418	BOOT_GOT	Image bytes received so far, 24 bits.
\$5419-\$541B	BOOT_TOT	Image size, 24 bits; 0 until the image starts arriving.

On an ordinary cartridge these addresses are open bus, so checking for the magic bytes is how a program knows it has a FujiNet at all:

6502 ASSEMBLY (CA65)

```
lda    $5409          ; FN_MAG0
cmp    #'F'
bne    nocart
lda    $540A          ; FN_MAG1
cmp    #'N'
bne    nocart
```

C (CC65 + FUJINET-LIB)

```
if (!fuji_nes_present()) /* 'F','N' and version 1 */
no_cartridge();
```

THE REGISTERS

Register	Address	Name	Meaning
\$00	\$5500	DEVICE	The FujiBus device: \$70 Fuji, \$71-\$78 network, \$45 clock ...
\$01	\$5501	CMD	The command.
\$02	\$5502	NPARAM	How many parameters are in the TX stream.
\$05	\$5505	DATA_RST	Any value: rewind the TX stream to empty.
\$06	\$5506	RXSLICE	Which slice of the reply to show. Always 0 on the NES.
\$10	\$5510	SEQ	Nonzero and not ACKSEQ: send the transaction .
\$11	\$5511	BOOTLOCK	\$B5: arm the loader for a staged image.
\$12-\$13	\$5512-\$5513	BOOTSEL	\$B5 then \$4A: reboot the RP2354B into its USB bootloader.
\$14	\$5514	SLICE_ACK	The loader has copied a slice. Programs never use it.

A register write is one store, so the library's routine for it is two instructions long:

LISTINGS/COMMON/FUJILIB.S

```
; fn_rw -- register X = A.
.proc fn_rw
    sta    FN_RSEL,x
    rts
```

The register's number is in X and its value in A. The base address's low byte is zero, so \$5500,X never carries into the next page.

THE TX STREAM

The bytes you store at \$5700 make up, in order:

- a) NPARAM parameters, each a size byte – 1, 2 or 4 – followed by that many bytes of value, low byte first;
- b) then the payload, as raw bytes.

There is room for 320 bytes in all, parameters included; stores past that are silently dropped. Here is SET_DEVICE_FULLPATH, which takes three one-byte parameters and a path:

01	00	01	00	01	01	/nesbook/hello.bin	00 ... 00
size, slot		size, host		size, mode		the path, padded with NULs to 256 bytes	

With NPARAM = 3 the cartridge reads three parameters and treats the rest as the payload. Rewind the stream with DATA_RST before you start, and set NPARAM to match what you stored.

COMMITTING A TRANSACTION

When the registers and the stream are ready, one store to SEQ sends the transaction. The cartridge unpacks the stream, wraps it in a FujiBus packet, sends it to the FujiNet over USB, and waits for the answer. Then it paints:

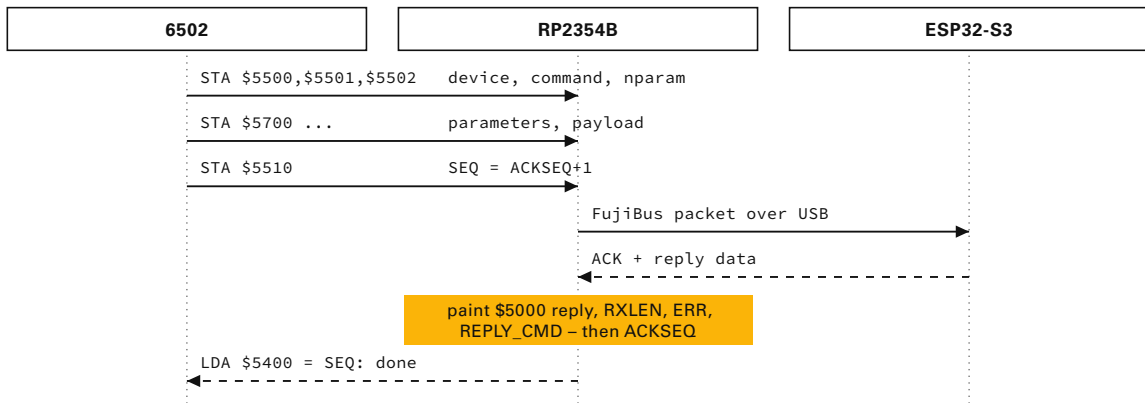


Fig. 3 – One transaction. ACKSEQ is painted last, so seeing it change means everything else is already in place.

Waiting is a loop that reads ACKSEQ until it equals the number you sent. Here is the library's, which also gives up after about eleven seconds:

LISTINGS/COMMON/FUJILIB.S: FN_GO

```
.proc fn_go
    lda    FN_ACKS
    clc
    adc     #1
    bne    have
    lda     #1                ; 0 is reserved as "never used"
have:    sta    FN_SEQ
        ldx    #FR_SEQ
        jsr    fn_rw         ; launching it is this single store
        lda    FN_TMO
        sta    FN_CNT
outer:   ldy    #0
mid:     ldx    #0
        ; ACKSEQ is published LAST, after the whole reply is in place.
inner:   lda    FN_ACKS
        cmp    FN_SEQ
        beq    got
        dex
        bne    inner
        dey
        bne    mid
        dec    FN_CNT
        bne    outer
        lda    #FNEWAIT
        rts
got:     lda    FN_ERR
        rts
```

WHERE THE SEQUENCE NUMBER COMES FROM

CAUTION: The next sequence number is the cartridge's own ACKSEQ plus one, wrapping from 255 to 1. Zero means "never used". Never keep a counter of your own.

Pressing Reset restarts the CPU, and your program clears its RAM – but the cartridge has no reset line on the edge and keeps going. A program that counts from 1 again after a Reset sends a sequence number the cartridge has already answered. The cartridge ignores it, ACKSEQ already matches, and the old reply still sitting in the window looks like a fresh success.

The cartridge ignores a SEQ of zero, and a SEQ equal to the last one it took. `hello.s` in Chapter 5 prints ACKSEQ on the screen; press Reset and watch it go up by one, not start again.

THE REPLY WINDOW

The reply is painted at \$5000 in one piece, RXLEN bytes long, up to 1024. A longer reply is cut off at 1024. The window holds still. The cartridge repaints it only when you commit the next transaction – or, while the loader is running, a slice. Until then you can read it as often as you like, draw from it straight to the PPU, or stream it back into the TX stream for the next transaction. CONFIG does exactly that: it reads a directory entry out of the window and sends it back as the path of the file to mount, without ever copying it into console RAM.

NOTE: Setting up the next transaction – the registers, DATA_RST, the TX stream – does not touch the window. Only the commit does. So it is safe to read a value out of one reply while you build the next request, as `json.s` does in Chapter 6.

WHAT CAN GO WRONG

After ACKSEQ matches, look at two cells:

1. **ERR at \$5402:** is the cartridge's own verdict on the trip. 0 means the FujiNet answered. 1 means the USB link was down (the cartridge waits up to three seconds for it). 2 means the FujiNet did not answer in time. 3 means the TX stream was malformed – a size byte other than 1, 2 or 4 – or the FujiNet's reply failed its checks. 4 means the request or the reply would not fit the cartridge's buffers.
2. **REPLY_CMD at \$5403:** is the FujiNet's verdict on the request: \$06 ACK or \$15 NAK. A NAK carries no data.

If ACKSEQ never matches at all, the cartridge never saw your SEQ store: you are running without the claim, on a cartridge that is not a FujiNet, or you wrote SEQ equal to ACKSEQ. The library calls that FNEWAIT, \$FF.

HOW LONG TO WAIT

The cartridge gives each transaction five seconds to come back from the FujiNet, and sixty for MOUNT_IMAGE and COPY_FILE, which do not answer until a whole file has been moved. It reports a TIMEOUT in ERR when the time runs out – so your own wait should be longer, or you will give up first and never see the cartridge's error code.

Who waits	How long
The cartridge, for an ordinary transaction	5 s

The cartridge, for MOUNT_IMAGE and COPY_FILE	60 s
The cartridge, for the USB link to come up	3 s
fn_go in fujilib.s (16 quanta of about 0.7 s)	about 11 s
fn_commit() in fujinet-lib	about 12 s
CONFIG, for a mount	65 s

CAUTION: Both libraries' waits are shorter than the cartridge's sixty seconds for MOUNT_IMAGE. For a large image, launch the mount without waiting and watch it yourself, as Chapter 8 shows.

THE NMI

Because the console's half of the mailbox is writes and the cartridge's half is reads, an interrupt can land anywhere in a transaction without harm. The vblank NMI handler may read the status page – or anything else – as long as it never **writes** to \$5500–\$57FF. cc65's own NMI handler touches only the PPU and RAM, so a C program is safe without thinking about it.

5. YOUR FIRST TRANSACTION

The first program asks the FujiNet one question – "how are you connected?" – and puts the answer on the screen.

OBJECT OF THIS CHAPTER

Build hello, in C and in assembly; run it; follow its one transaction through the mailbox; and press Reset to watch the sequence number carry on.

IN C

LISTINGS/C/HELLO.C

```
/* hello.c -- the first transaction, in C.
 *
 * Asks the FujiNet for its adapter configuration and puts the network name,
 * the IP address and the firmware version on the screen. */

#include <conio.h>
#include <fuji-net-fuji.h>
#include <fuji-net-nes.h>

static AdapterConfigExtended ac;

void main(void)
{
    clrscr();
    cputs("FUJINET NES\r\n\r\n");

    if (!fuji_nes_present()) {          /* 'F','N' and version 1? */
        cputs("NO FUJINET CARTRIDGE");
        for (;;) ;
    }

    if (!fuji_get_adapter_config_extended(&ac)) {
        cputs("NO ANSWER");
        for (;;) ;
    }

    cprintf("SSID %s\r\n", ac.ssid);
    cprintf("IP   %s\r\n", ac.sLocalIP);
    cprintf("FW   %.15s\r\n", ac.fn_version);
    for (;;) ;
}
```

1. **fuji_nes_present()**: checks the magic bytes and the protocol version in the status page. On an ordinary cartridge they are open bus.
2. **fuji_get_adapter_config_extended()**: is one transaction: device \$70, command \$C4, no parameters, no payload. The 240-byte reply is copied out of the reply window into ac.
3. **cputs, cprintf**: are cc65's console routines. On the NES they queue their writes for the vblank NMI, so they work with the screen on.

```
FUJINET NES
SSID Dummy Cafe
IP 127.0.0.1
FW V1.6-1c5dcb807
```

```
FUJINET NES
SSID DUMMY CAFE
IP 127.0.0.1
FW V1.6-1C5DCB807

ACKSEQ #01
```

Fig. 4 – hello in C (left) and in assembly (right). The FujiNet here is fujinet-pc, whose WiFi is a fixture called "Dummy Cafe".

IN ASSEMBLY

The assembly version does the same, with the transaction spelled out. Here is its heart; the whole program is Listing 1 in Appendix C.

LISTINGS/ASM/HELLO.S

```

have:  CALL    FNDEVF, FNCADPX, 0
        jsr     fn_go          ; commit, wait for ACKSEQ
        jne     fail           ; A = FN_ERR, or FNEWAIT
        jsr     fn_ack         ; did the FujiNet say ACK?
        jne     fail

        PUTS    5, 2, tssid
        ldx     #AC_SSID      ; window offset
        ldy     #24           ; at most 24 characters
        jsr     disp_rpl
        PUTS    7, 2, tip
        ldx     #AC_SIP
        ldy     #16
        jsr     disp_rpl
        PUTS    9, 2, tver
        ldx     #AC_VER
        ldy     #15
        jsr     disp_rpl
        jmp     show

```

1. **CALL FNDEVF, FNCADPX, 0:** is a macro from `book.inc`. It stores the device, the command and the parameter count in the library's zero page and calls `fn_beg`, which writes them to registers \$00-\$02 and rewinds the TX stream.
2. **fn_go:** commits: ACKSEQ plus one goes to SEQ, then it waits for ACKSEQ to match. It returns the cartridge's ERR in A, with the Z flag set for success, or FNEWAIT if the cartridge never answered.
3. **fn_ack:** looks at `REPLY_CMD`: zero for ACK, `FNENAK` for NAK.
4. **disp_rpl:** copies up to Y bytes from offset X of the reply window straight to the PPU, stopping at a NUL. The answer never touches console RAM.

The reply's layout is the firmware's `AdapterConfigExtended`, packed:

Offset	Bytes	Field
0	33	ssid – "NOT CONNECTED" when WiFi is down
33	64	hostname
97	4 each	localIP, gateway, netmask, dnsIP, as four binary bytes
113	6 each	macAddress, bssid
125	15	fn_version (no NUL if it fills the field)

140	16 each	sLocalIP, sGateway, sNetmask, sDnsIP, as text
204	18 each	sMacAddress, sBssid, as text; 240 in all

WHAT CROSSED THE MAILBOX

Store or load	Meaning
\$5500 ← \$70	DEVICE: the Fuji device
\$5501 ← \$C4	CMD: GET_ADAPTERCONFIG_EXTENDED
\$5502 ← \$00	NPARAM: none
\$5505 ← \$00	DATA_RST: empty TX stream
\$5510 ← \$01	SEQ: ACKSEQ was 0, nothing answered yet, so 1
\$5400 → \$01	ACKSEQ matches: done
\$5402 → \$00	ERR: the FujiNet answered
\$5403 → \$06	REPLY_CMD: ACK
\$5404 → \$F0	RXLEN: 240 bytes, now at \$5000-\$50EF

PRESS RESET



Fig. 5 – Before and after the console's Reset. The program started again from nothing, but the cartridge did not: the next sequence number was 2.

hello.s prints ACKSEQ at the bottom of the screen. Press Reset: the 6502 starts over, nesinit.s clears all of RAM, and fn_go reads ACKSEQ – still 1 – and sends 2. Had it counted from a variable of its own it would have sent 1 again, the cartridge would have ignored it, and the program would have printed the stale reply as if it were new.

THE ASSEMBLY LIBRARY

fujilib.s is the cartridge firmware's own test library, unchanged. It is short enough to know by heart:

Routine	Takes	Does
fn_chk		Z set if the magic bytes say a FujiNet is there.
fn_rw	X reg, A value	One register write.
fn_beg	fn_dev, fn_cmd, fn_npr	Begin a transaction: the three registers, then DATA_RST. Sets the timeout to 16 quanta.
fn_txb	A	Append one raw byte to the TX stream.
fn_pb	A	Append a one-byte parameter (size byte 1, then A).

fn_pw	A low, X high	Append a two-byte parameter.
fn_path	fn_ptr	Append the string at fn_ptr, padded with NULs to exactly 256 bytes.
fn_go	fn_tmo	Commit and wait. A = ERR (Z set if 0), or FNEWAIT.
fn_ack		A = 0 if the reply was ACK, else FENAK.
fn_blk		Arm the loader (BOOTLOCK = \$B5).
fn_boot		Interrupts and PPU off, jump to the loader at \$5800. Never returns.

booklib.s adds what this manual's programs need beyond that:

Routine	Does
nmi	The vblank handler: counts frames in frames, nothing else.
wait_vbl	Wait for the next vblank.
scroll0	After drawing in vblank: scroll home, NMI on.
fn_str	Append the string at fn_ptr without padding – for URLs.
fn_launch	Commit without waiting.
fn_done	C set once the launched transaction has been answered.
disp_d3	A as three decimal digits, right-aligned.

CAUTION: The library keeps its state in zero page \$E0-\$EF: the sequence it is waiting for, the timeout, the transaction setup and fn_ptr. Keep your own variables out of it. nes.cfg stops the ZEROPAGE segment at \$E0.

The display routines in fujidisp.s write straight to the PPU and must run with rendering off, or inside vblank; Chapter 11 explains why. hello.s draws everything first and turns the screen on last.

6. THE NETWORK DEVICE

Eight network channels, N1: to N8:, each able to hold one open connection to anything the FujiNet speaks – a web page, a TCP socket, a file on a TNFS server.

THE LIFE OF A CONNECTION

The network channels are FujiBus devices \$71 to \$78. N1: is \$71. A connection goes through five commands, always in this order:

1. **OPEN:** names the thing to connect to, as a URL with an N: in front: N:HTTP://example.com/file.txt, N:TCP://bbs.example.com:23/, N:TNFS://host/path. OPEN closes whatever the channel had open before.
2. **STATUS:** says how many bytes are waiting to be read, whether the connection is still up, and the last error. For HTTP, the first STATUS is what actually sends the request.
3. **READ:** takes up to the number of bytes you ask for – at most 1024, the size of the reply window.
4. **WRITE:** sends bytes – at most 317 in one transaction on the NES.
5. **CLOSE:** hangs up.

A file is over when STATUS says nothing is waiting and its error byte says 136, END OF FILE. A TCP connection is over when STATUS says it is no longer connected and nothing is waiting.

READING A FILE

Serve the two fixtures in listings/fixture/ with any web server – python3 -m http.server 8765 in that directory will do – and run netget:

LISTINGS/C/NETGET.C

```
/* netget.c -- read a text file over HTTP and print it. */

#include <conio.h>
#include <fujinet-network.h>

static const char url[] = "N:HTTP://127.0.0.1:8765/hello.txt";
static char buf[256];

void main(void)
{
    int16_t n, i;

    clrscr();
    cputs("NETGET\r\n\r\n");

    network_init();
    if (network_open(url, OPEN_MODE_READ, OPEN_TRANS_NONE) != FN_ERR_OK) {
        cputs("OPEN FAILED");
        for (;;) ;
    }

    /* network_read() returns once it has len bytes or the file has ended. */
    while ((n = network_read(url, buf, sizeof buf)) > 0) {
        for (i = 0; i < n; i++) {
            if (buf[i] == '\n')
                cputs("\r\n");
            else
                cputc(buf[i]);
        }
    }

    network_close(url);
    cputs("\r\n-- END --");
    for (;;) ;
}
```

`network_read()` loops inside the library, asking STATUS and READ until it has the bytes it was asked for or the file has ended. In assembly the loop is yours:

LISTINGS/ASM/NETGET.S: THE READ LOOP

```

next:  CALL    FNDEVN, NCSTAT, 0      ; how much is waiting?
      jsr     fn_go
      jne     fail
      lda     FN_RPLY+1              ; avail, high byte
      bne     big
      lda     FN_RPLY                ; avail, low byte
      bne     some
      lda     FN_RPLY+3              ; nothing waiting: an error (136 = EOF)
      cmp     #2                     ; means the file is over
      bcs     eof
      jmp     next
big:   lda     #255
some:  sta     count

      CALL    FNDEVN, NCREAD, 1      ; READ count bytes
      lda     count
      ldx     #0
      jsr     fn_pw                  ; one 2-byte parameter
      jsr     fn_go
      jne     fail

      ldx     #0                     ; print them out of the window
print: lda     FN_RPLY,x
      cmp     #$0A
      bne     :+
      stx     xsave                  ; disp_at uses X
      inc     row
      jsr     newline
      ldx     xsave
      jmp     adv
:      sta     PPU_DATA
adv:   inx
      cpx     FN_RXLO
      bne     print
      jmp     next

```

Asking for no more than 255 bytes at a time keeps the whole reply within reach of one index register. The bytes are drawn straight from the window; a \$0A moves the PPU cursor to the next line instead.

```
NETGET
HELLO FROM THE NETWORK.
THIS FILE CAME OVER HTTP
THROUGH THE ESP32 AND THE
CARTRIDGE MAILBOX, TO YOUR
NES.
-- END --
```

```
NETGET
HELLO FROM THE NETWORK.
THIS FILE CAME OVER HTTP
THROUGH THE ESP32 AND THE
CARTRIDGE MAILBOX, TO YOUR
NES.
-- END --
```

Fig. 6 – netget, C and assembly.

A JSON ROUND TRIP

The FujiNet can parse what it fetches. Open the document, set the channel's parser to JSON, PARSE, and then each QUERY with a path such as `/mailbox/base` leaves that one value waiting to be read:

LISTINGS/C/JSON.C

```
/* json.c -- fetch a JSON document and pick fields out of it. */

#include <conio.h>
#include <fujinet-network.h>

static const char url[] = "N:HTTP://127.0.0.1:8765/hello.json";
static const char *const paths[] = {
    "/name", "/console", "/year", "/mailbox/base", "/mailbox/reply"
};
static char value[128];

void main(void)
{
    uint8_t i;

    clrscr();
    cputs("JSON\r\n\r\n");

    network_init();
    if (network_open(url, OPEN_MODE_READ, OPEN_TRANS_NONE) != FN_ERR_OK
        || network_json_parse(url) != FN_ERR_OK) {
        cputs("OPEN/PARSE FAILED");
        for (;;) ;
    }

    for (i = 0; i < 5; i++) {
        if (network_json_query(url, paths[i], value) < 0)
            value[0] = '\0';
        cprintf("%-14s %s\r\n", paths[i], value);
    }

    network_close(url);
    for (;;) ;
}
```

LISTINGS/ASM/JSON.S: ONE FIELD

```
CALL    FNDEVN, NCQUERY, 0      ; QUERY: the path is the payload
jsr     fn_path                 ; NUL-padded to 256, as the library does
jsr     fn_go
jne     fail
CALL    FNDEVN, NCSTAT, 0       ; the value's length
jsr     fn_go
jne     fail
lda     FN_RPLY
beq     skip                   ; no such field
CALL    FNDEVN, NCREAD, 1
lda     FN_RPLY                 ; still the STATUS reply: fn_beg wrote
ldx     #0                     ; only registers, not the window
jsr     fn_pw
jsr     fn_go
jne     fail
lda     row
ldx     #17
jsr     disp_at
ldx     #0
ldy     FN_RXLO
show1:  lda     FN_RPLY,x
cmp     #$20                   ; stop at the trailing newline
bcc     skip
sta     PPU_DATA
inx
dey
bne     show1
```

A value comes back as text, ended by the parser's line ending – a \$0A unless you change it with SET_PARAMETER. Numbers come back as their digits; true, false and null as TRUE, FALSE and NULL. A path that does not exist leaves nothing waiting.

NOTE: The assembly version reads the value's length out of the STATUS reply **after** it has called fn_beg for the READ. That is safe: setting up a transaction never repaints the window. Only the commit does.

```

JSON
/name          FujiNet
/console       NES
/year          1985
/mailbox/base  $5000
/mailbox/reply 1024

JSON
/NAME          FUJINET
/CONSOLE       NES
/YEAR          1985
/MAILBOX/BASE  $5000
/MAILBOX/REPLY 1024

```

Fig. 7 – json, C and assembly. The paths are on the left, the values the FujiNet found for them on the right.

COMMAND REFERENCE

On every card, **params** are in the order they go into the TX stream, each with its size in bytes. A parameter is never optional unless the card says so: a missing one does not earn a NAK, it restarts the FujiNet. The device is N1:, \$71; use \$72-\$78 for N2: to N8:.

OPEN

DEV \$71 CMD \$4F 'O'

PARAMS	mode (1): 4 read / HTTP GET, 6 directory, 8 write / HTTP PUT, 9 append, 12 read-write / HTTP GET with headers, 13 HTTP POST, 5 HTTP DELETE. trans (1): 0 none, 1 CR, 2 LF, 3 CRLF, 4 PETSCII.
PAYLOAD	The URL, N: optional, up to 256 bytes. It ends at a NUL, or at CR LF.
REPLY	none; NAK if the URL is bad or the server refuses. STATUS then says why: 165 invalid devicespec, 144 general, or the protocol's own error.

OPEN closes whatever the channel had open. A URL with no scheme is taken relative to the channel's current directory (CHDIR). The schemes the FujiNet knows include HTTP, HTTPS, TCP, UDP, TELNET, TNFS, FTP, SMB, NFS, SSH, SFTP, WS and WSS.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVN, NCOPEN, 2
lda     #NMREAD      ; mode
jsr     fn_pb
lda     #0            ; no translation
jsr     fn_pb
lda     #<url
sta     fn_ptr
lda     #>url
sta     fn_ptr+1
jsr     fn_str        ; the URL, unpadded
jsr     fn_go
jne     fail
jsr     fn_ack
jne     fail
```

C (CC65 + FUJINET-LIB)

```
if (network_open(url, OPEN_MODE_READ,
                  OPEN_TRANS_NONE) != FN_ERR_OK)
    fail();
```

CLOSE

DEV \$71 CMD \$43 'C'

PARAMS	none
PAYLOAD	none
REPLY	none; NAK if the protocol's close failed. Once it has failed, CLOSE goes on NAKing until the next OPEN.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVN, NCCLOSE, 0
jsr     fn_go
```

C (CC65 + FUJINET-LIB)

```
network_close(url);
```

STATUS

DEV \$71 CMD \$53 'S'

PARAMS	none
PAYLOAD	none
REPLY	4 bytes: bytes waiting (2, low first, at most 65535), connected (1: 1 yes, 0 no), error (1: 1 OK, 136 end of file, others in Appendix A). With no connection open: 0, 0, 207.

On HTTP the first STATUS after OPEN is what sends the request and waits for the response headers, so it may take a while.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVN, NCSTAT, 0
jsr     fn_go
jne     fail
lda     FN_RPLY          ; bytes waiting, low
ldx     FN_RPLY+1        ; and high
ldy     FN_RPLY+3        ; the error
```

C (CC65 + FUJINET-LIB)

```
if (network_status(url, &avail, &conn,
                    &err) != FN_ERR_OK)
    fail();
```

READ

DEV \$71 CMD \$52 'R'

PARAMS	count (2), 1 or more.
PAYLOAD	none
REPLY	Up to count bytes; fewer if fewer are waiting. NAK if count is 0, nothing is open, or the read failed.

The reply window holds 1024 bytes, so ask for no more than that. fujinet-lib caps every READ at 1024 for you; network_read() keeps going until it has the whole len or the file ends.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVN, NCREAD, 1
lda     #<200
ldx     #>200
jsr     fn_pw
jsr     fn_go
jne     fail
; FN_RXLO/FN_RXHI bytes at FN_RPLY
```

C (CC65 + FUJINET-LIB)

```
/* waits until len bytes or the end */
n16 = network_read(url, buf, 200);
/* one STATUS, then one READ */
n16 = network_read_nb(url, buf, 200);
```

WRITE

DEV \$71 CMD \$57 'W'

PARAMS	count (2), 1 or more.
PAYLOAD	count bytes. Short? The rest are sent as zeros.
REPLY	none; NAK if nothing is open or the write failed.

The TX stream holds 320 bytes and the count parameter takes three of them, so one WRITE carries at most 317. network_write() splits a longer buffer for you.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVN, NCWRITE, 1
lda     #5                ; count
ldx     #0
jsr     fn_pw
ldx     #0
:       lda     data,x
jsr     fn_txb
inx
cpx     #5
bne     :-
jsr     fn_go
jne     fail
```

C (CC65 + FUJINET-LIB)

```
if (network_write(url, (uint8_t *) "HELLO",
                    5) != FN_ERR_OK)
    fail();
```

SET PARSER

DEV \$71 CMD \$FC

PARAMS	ignored (1), then the parser (1): 0 none, 1 JSON, 2 HTML (CSS selectors), 3 XML (a subset of XPath).
PAYLOAD	none
REPLY	none; NAK if the parser number is above 3.

Sets how PARSE and QUERY treat what the channel is reading. Parser 0 hands the bytes through untouched, and every QUERY NAKs.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVN, NCPARSER, 2
lda     #1
jsr     fn_pb
lda     #1                ; JSON
jsr     fn_pb
jsr     fn_go
```

C (CC65 + FUJINET-LIB)

```
network_set_parser(url, PARSER_JSON);
```

PARSE

DEV \$71 CMD \$50 'P'

PARAMS	none
PAYLOAD	none
REPLY	none; NAK if nothing is open or the document would not parse (STATUS error 213 for bad JSON).

Reads the whole response and parses it. After this the channel can no longer be written to or seeked.

6502 ASSEMBLY (CA65)

CALL	FNDEVN, NCPARSE, 0
jsr	fn_go
jne	fail
jsr	fn_ack
jne	fail

C (CC65 + FUJINET-LIB)

/* SET_PARSER(JSON) then PARSE */
if (network_json_parse(url) != FN_ERR_OK)
fail();

QUERY

DEV \$71 CMD \$51 'Q'

PARAMS	none
PAYLOAD	The path, such as /players/0/name for JSON; trailing NULs are harmless. An empty path is the whole document.
REPLY	none; NAK if there is no parser or the path is malformed. STATUS then gives the value's length, and READ fetches it.

6502 ASSEMBLY (CA65)

CALL	FNDEVN, NCQUERY, 0
lda	#<path
sta	fn_ptr
lda	#>path
sta	fn_ptr+1
jsr	fn_str
jsr	fn_go

C (CC65 + FUJINET-LIB)

/* QUERY, then STATUS and READ until done */
n16 = network_json_query(url, "/name",
(char *) buf);

SET PARAMETER

DEV \$71 CMD \$FB

PARAMS	which (1), value (1). Which 0: the query flags – \$01 remap characters, \$02 remap ATASCII international, \$04 delete SGML tags, \$10 plain ASCII. Which 1: the parser's line ending, one character.
PAYLOAD	none
REPLY	none; NAK if no parser is set, or for any other which.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVN, NCPARAM, 2
lda     #1                ; the line ending
jsr     fn_pb
lda     #0                ; NUL instead of LF
jsr     fn_pb
jsr     fn_go
```

C (CC65 + FUJINET-LIB)

```
network_set_line_ending(url, 0);
```

TRANSLATION

DEV \$71 CMD \$54 'T'

PARAMS	ignored (1), translation (1): 0 none, 1 CR, 2 LF, 3 CRLF, 4 PETSCII, \$FF force none.
PAYLOAD	none
REPLY	none

Overrides the end-of-line translation OPEN asked for. The FujiNet's own line ending on this bus is CR LF, so with translation 0 nothing changes.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVN, NCTrans, 2
lda     #0
jsr     fn_pb
lda     #2                ; CR LF -> LF
jsr     fn_pb
jsr     fn_go
```

C (CC65 + FUJINET-LIB)

```
network_set_translation(url,
                        OPEN_TRANS_LF);
```

SET EOL

DEV \$71 CMD \$4C 'L'

PARAMS	none
PAYLOAD	The end-of-line bytes, exactly; empty clears the override.
REPLY	none

6502 ASSEMBLY (CA65)

```
CALL    FNDEVN, NCEOL, 0
lda     #$0A
jsr     fn_txb
jsr     fn_go
```

C (CC65 + FUJINET-LIB)

```
network_set_eol(url, "\n");
```

HTTP CHANNEL MODE

DEV \$71 CMD \$4D 'M'

PARAMS	ignored (1), mode (1): 0 body, 1 collect headers, 2 get headers, 3 set headers, 4 set POST data.
PAYLOAD	none
REPLY	none; NAK if the channel is not HTTP or the mode is above 4.

A POST goes: OPEN with mode 13, mode 4, WRITE the body, mode 0, then STATUS and READ the response. Mode 3 lets you WRITE request headers as Name: value; mode 1 lets you WRITE the names of response headers to keep, and mode 2 READs their values. Each change of mode empties both buffers.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVN, NCHTTPM, 2
lda     #0
jsr     fn_pb
lda     #4                ; what I WRITE is the body
jsr     fn_pb
jsr     fn_go
```

C (CC65 + FUJINET-LIB)

```
/* OPEN mode 13, then: */
network_http_post(url, "name=fuji");
```

SEEK / TELL

DEV \$71 CMD \$25 '%' / \$26 '&'

PARAMS	SEEK: offset (4).
PAYLOAD	none
REPLY	TELL: the position, 4 bytes. With nothing open TELL returns four bytes of nonsense, and still ACKs.

6502 ASSEMBLY (CA65)

CALL	FNDEVN, NCTELL, 0
jsr	fn_go
jne	fail
lda	FN_RPLY ; position, low byte

C (CC65 + FUJINET-LIB)

network_seek(url, 1024UL);
network_tell(url, &r32);

CHDIR / GETCWD

DEV \$71 CMD \$2C ',' / \$30 '0'

PARAMS	none
PAYLOAD	CHDIR: the path. .. or < goes up, / or > to the host's root; a full URL starts over.
REPLY	GETCWD: the prefix, as text, no NUL.

6502 ASSEMBLY (CA65)

CALL	FNDEVN, NCGETCWD, 0
jsr	fn_go ; FN_RXL0 bytes at FN_RPLY

C (CC65 + FUJINET-LIB)

network_fs_pwd(url, (char *) buf);

USERNAME / PASSWORD

DEV \$71 CMD \$FD / \$FE

PARAMS	none
PAYLOAD	The name or the password, up to 256 bytes.
REPLY	none

Kept for the next OPEN on the channel.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVN, NCUSER, 0
lda     #<data
sta     fn_ptr
lda     #>data
sta     fn_ptr+1
jsr     fn_str
jsr     fn_go
```

C (CC65 + FUJINET-LIB)

```
network_set_username(url, "guest");
network_set_password(url, "secret");
```

FILE OPERATIONS

DEV \$71 CMD \$20 \$21 \$23 \$24 \$2A \$2B

PARAMS	mode (1), unused (1).
PAYLOAD	The URL; for RENAME, old,new.
REPLY	none; NAK if the protocol has no file system or the operation failed.

\$20 RENAME, \$21 '!' DELETE, \$23 '#' LOCK, \$24 '\$' UNLOCK, \$2A '*' MKDIR, \$2B '+' RMDIR. Each is done in one shot, and closes whatever the channel had open.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVN, NCDELETE, 2
lda     #0
jsr     fn_pb
lda     #0
jsr     fn_pb
lda     #<url
sta     fn_ptr
lda     #>url
sta     fn_ptr+1
jsr     fn_str
jsr     fn_go
```

C (CC65 + FUJINET-LIB)

```
network_fs_delete(url);
network_fs_mkdir("N:TNFS://host/new/");
```

ACCEPT / CLOSE CLIENT

DEV \$71 CMD \$41 'A' / \$63 'c'

PARAMS	none
PAYLOAD	none
REPLY	none; NAK if the channel is not a listening TCP socket, or no client is waiting.

A TCP URL with no host – N:TCP://:6502/ – listens. When STATUS shows a connection, ACCEPT takes it.

6502 ASSEMBLY (CA65)

CALL	FNDEVN, NCACCEPT, 0
jsr	fn_go

C (CC65 + FUJINET-LIB)

/* after OPEN N:TCP://:6502/ */ network_accept(url); network_close_client(url);

UDP DESTINATION / REMOTE

DEV \$71 CMD \$44 'D' / \$72 'r'

PARAMS	none
PAYLOAD	SET_DESTINATION: host:port.
REPLY	GET_REMOTE: 256 bytes, the sender as ip:port. On a real FujiNet (an ESP32) GET_REMOTE always NAKs; it works on fujinet-pc.

6502 ASSEMBLY (CA65)

CALL	FNDEVN, NCDEST, 0
lda	#<data
sta	fn_ptr
lda	#>data
sta	fn_ptr+1
jsr	fn_str
jsr	fn_go

C (CC65 + FUJINET-LIB)

network_udp_set_destination("N:UDP://10.0.0.5:6502/");
--

INTERRUPT RATE

DEV \$71 CMD \$5A 'Z'

PARAMS	milliseconds (1).
PAYLOAD	none
REPLY	none

Paces the FujiNet's "data waiting" interrupt line. The NES cartridge does not carry that line to the 6502, so an NES program polls STATUS instead.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVN, NCINTR, 1
lda     #100
jsr     fn_pb
jsr     fn_go
```

C (CC65 + FUJINET-LIB)

```
network_set_interrupt_rate(url, 100);
```

The network device NAKs \$FF GET_DSTATS, \$FA SET_UNIT, \$81 and \$80, and \$45 GET_ERROR – read the error byte of STATUS instead.

7. THE FUJI DEVICE

Device \$70 is the FujiNet itself: its WiFi, its eight host slots and eight device slots, its directories, its app keys, and a handful of jobs it will do for a 1.79 MHz CPU that would rather not.

HOSTS, DEVICES AND PATHS

The FujiNet keeps two tables that every FujiNet computer shares.

- 1. Eight host slots:** name places files come from: SD for the FujiNet's own SD card, or the name of a TNFS server such as `ec.tnfs.io`. `READ_HOST_SLOTS` returns all eight, 32 bytes each.
- 2. Eight device slots:** each name a file on one of those hosts, and a mode – 1 read, 2 write. On the NES the slot that matters is device slot 0: it is where a cartridge image is mounted to be booted (Chapter 8).

Before you can list a host's files you `MOUNT_HOST` it. Slot numbers on the wire are 0 to 7 – what `CONFIG` shows as host 1 is slot 0.

LISTING A DIRECTORY

LISTINGS/C/DIR.C

```
/* dir.c -- list a directory on a TNFS host. */

#include <conio.h>
#include <string.h>
#include <fujinet-fuji.h>

#define HOST 7 /* host slot 8: ec.tnfs.io */
#define ROWS 20

static HostSlot hosts[8];
static char path[256];
static char entry[32];

void main(void)
{
    uint8_t n;

    clrscr();
    if (!fuji_get_host_slots(hosts, 8)) {
        cputs("NO HOST SLOTS");
    }
}
```

```

    for (;;) ;
}
cprintf("DIR OF %s\r\n\r\n", (char *) hosts[HOST]);

/* The path and an optional filter travel as one 256-byte block:
   "/" NUL "*.nes" NUL. Here there is no filter. */
memset(path, 0, sizeof path);
path[0] = '/';

if (!fuji_mount_host_slot(HOST) || !fuji_open_directory(HOST, path)) {
    cputs("CANNOT OPEN");
    for (;;) ;
}

for (n = 0; n < ROWS; n++) {
    if (!fuji_read_directory(30, 0, entry))
        break;
    if (entry[0] == 0x7F)                /* $7F = end of directory */
        break;
    cprintf(" %s\r\n", entry);
}

fuji_close_directory();
for (;;) ;
}

```

OPEN_DIRECTORY takes the path and an optional filter as one 256-byte block – the path, a NUL, the filter, a NUL. READ_DIR_ENTRY then returns one name at a time, padded to the length you ask for, with a / on the end of a directory's name. The end of the directory is a name that begins with \$7F.

LISTINGS/ASM/DIR.S: THE LOOP

```

entry: CALL    FNDEVF, FNCRDIR, 2      ; READ_DIR_ENTRY(maxlen, flags)
      lda     #28
      jsr     fn_pb
      lda     #0
      jsr     fn_pb
      jsr     fn_go
      jne     fail
      jsr     fn_ack
      jne     close
      lda     FN_RPLY
      cmp     #$7F                    ; end of directory
      beq     close
      lda     row
      ldx     #3
      jsr     disp_at
      ldx     #0
      ldy     #28
      jsr     disp_rpl
      inc     row
      lda     row
      cmp     #4+ROWS
      bne     entry

```

```

DIR OF ec.tnfs.io
apple2/
atari/
c64/
coco/
intu/
msdos/
utils/
== New tnfs ==

```

```

DIR OF EC.TNFS.IO
APPLE2/
ATARI/
C64/
COCO/
INTU/
MSDOS/
UTILS/
== NEW TNFS ==

```

Fig. 8 – The top of `ec.tnfs.io`, in C and in assembly. The host's name comes straight from `READ_HOST_SLOTS`.

NOTE: `fuji_open_directory()` sends 256 bytes starting at the pointer you give it, so give it a 256-byte buffer. And pass an empty string, not `NULL`, as the filter to `fuji_open_directory_filter()`: it looks at the filter's first byte.

APP KEYS

An app key is up to 64 bytes the FujiNet keeps for your program on its SD card, under `/FujiNet/`, named by three numbers: a **creator** (two bytes; zero is not allowed), an **app** and a **key** (one byte each). Pick a creator number of your own – the FujiNet games use `$E41C` and `$BEBE`; the Lobby uses creator 1, app 1, with key 0 holding the player's name.

LISTINGS/C/APPKEY.C

```
/* appkey.c -- count how many times this program has been run. */

#include <conio.h>
#include <fujinet-fuji.h>

#define CREATOR 0x5E5E /* a scratch creator id */
#define APP 0x01
#define KEY 0x00

static uint8_t data[MAX_APPKEY_LEN + 2];

void main(void)
{
    uint16_t len = 0;
    uint16_t runs = 0;

    clrscr();
    cputs("APPKEY\r\n\r\n");

    fuji_set_appkey_details(CREATOR, APP, DEFAULT);

    /* A key that has never been written reads back empty. */
    if (fuji_read_appkey(KEY, &len, data) && len >= 2)
        runs = data[0] | (data[1] << 8);

    ++runs;
    data[0] = runs & 0xFF;
    data[1] = runs >> 8;

    if (!fuji_write_appkey(KEY, 2, data)) {
        cputs("WRITE FAILED");
        for (;;) ;
    }

    cprintf("THIS PROGRAM HAS RUN\r\n%u TIME%s.\r\n", runs, runs == 1 ? "" : "S");
    cputs("\r\nPRESS RESET TO COUNT AGAIN.");
    for (;;) ;
}
```

Underneath, a key is always opened first, with a six-byte record:

creator lo	creator hi	app	key	mode	0
---------------	---------------	-----	-----	------	---

mode 0 to read, 1 to write

then READ or WRITE. On this bus the READ reply starts with the key's length, two bytes, low first. A key that was never written reads back with a length of zero – READ_APPKEY never NAKs.

LISTINGS/ASM/APPKEY.S: OPEN, THEN READ

```
        lda     #0                      ; mode 0: read
        jsr     open_key
        jne     fail
        CALL    FNDEVF, FNCRKEY, 0
        jsr     fn_go
        jne     fail
        jsr     fn_ack
        bne     count                  ; never written: start from zero
        lda     FN_RPLY+1               ; length, high byte
        bne     have
        lda     FN_RPLY                ; length, low byte
        cmp     #2
        bcc     count
have:    lda     FN_RPLY+2               ; the two bytes after the length
        sta     runs
        lda     FN_RPLY+3
        sta     runs+1
```

```
APPKEY
THIS PROGRAM HAS RUN
2 TIMES.
PRESS RESET TO COUNT AGAIN.
```

```
APPKEY
THIS PROGRAM HAS RUN
#0003 TIMES.
PRESS RESET TO COUNT AGAIN.
```

Fig. 9 –The C program and then the assembly one, run one after the other: they share the same key.

WIFI

GET WIFI STATUS

DEV \$70 CMD \$FA

PARAMS none
PAYLOAD none
REPLY 1 byte: 3 connected, 6 not connected. Nothing else is ever sent.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVF, FNCWIFI, 0
jsr     fn_go
jne     fail
lda     FN_RPLY
cmp     #3                ; connected?
```

C (CC65 + FUJINET-LIB)

```
if (fuji_get_wifi_status(&status)
    && status == 3)
    cputs("ONLINE");
```

GET WIFI ENABLED

DEV \$70 CMD \$EA

PARAMS none
PAYLOAD none
REPLY 1 byte: 1 enabled, 0 disabled.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVF, FNCWIFEN, 0
jsr     fn_go
```

C (CC65 + FUJINET-LIB)

```
if (!fuji_get_wifi_enabled())
    cputs("WIFI IS OFF");
```

GET SSID

DEV \$70 CMD \$FE

PARAMS none
PAYLOAD none
REPLY 97 bytes: the network name (33), then its password (64), in the clear.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVF, FNCGSSID, 0
jsr     fn_go                ; name at FN_RPLY
```

C (CC65 + FUJINET-LIB)

```
fuji_get_ssid(&nc);
cputs(nc.ssid);
```

SCAN NETWORKS

DEV \$70 CMD \$FD

PARAMS	none
PAYLOAD	none
REPLY	1 byte: how many networks were found.

Takes several seconds – the FujiNet scans before it answers.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVF, FNCSCAN, 0
jsr     fn_go
jne     fail
lda     FN_RPLY          ; count
```

C (CC65 + FUJINET-LIB)

```
fuji_scan_for_networks(&count);
```

GET SCAN RESULT

DEV \$70 CMD \$FC

PARAMS	index (1), from 0.
PAYLOAD	none
REPLY	34 bytes: the name (33), then the signal strength as a signed byte, in dBm. NAK for an index past the last scan's count.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVF, FNCSCNR, 1
lda     #0                ; the first
jsr     fn_pb
jsr     fn_go
jne     fail
lda     FN_RPLY+33        ; rssi, negative
```

C (CC65 + FUJINET-LIB)

```
fuji_get_scan_result(0, &info);
printf("%s %d", info.ssid, info.rssi);
```

SET SSID

DEV \$70 CMD \$FB

PARAMS	none
PAYLOAD	97 bytes: the name, NUL-padded to 33, then the password, NUL-padded to 64.
REPLY	none, once the FujiNet has joined. NAK if it could not.

CAUTION: send all 97 bytes. The FujiNet copies what you send over a buffer it did not clear, so a short payload leaves garbage in the password. The FujiNet saves the network to its configuration.

6502 ASSEMBLY (CA65)

	CALL	FNDEVF, FNCSSSID, 0
	ldx	#0
:	lda	ssidpw,x ; 97 bytes, padded
	jsr	fn_txb
	inx	
	cpx	#97
	bne	:-
	jsr	fn_go

C (CC65 + FUJINET-LIB)

memset(&nc, 0, sizeof nc);
strcpy(nc.ssid, "MyNetwork");
strcpy(nc.password, "secret");
fuji_set_ssid(&nc);

HOSTS AND DEVICE SLOTS

READ HOST SLOTS

DEV \$70 CMD \$F4

PARAMS	none
PAYLOAD	none
REPLY	256 bytes: eight host names of 32 bytes.

6502 ASSEMBLY (CA65)

	CALL	FNDEVF, FNCRHST, 0
	jsr	fn_go ; slot n at FN_REPLY + n*32

C (CC65 + FUJINET-LIB)

fuji_get_host_slots(hosts, 8);

WRITE HOST SLOTS

DEV \$70 CMD \$F3

PARAMS	none
PAYLOAD	256 bytes: all eight names.
REPLY	none; the FujiNet saves its configuration.

Send all eight. To change one, read the table, change your copy, and write it all back – or, as CONFIG does, stream seven of them straight out of the reply window and the changed one from RAM.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVF, FNCRHST, 0
jsr     fn_go      ; read them first
CALL    FNDEVF, FNCWHST, 0
ldx     #0         ; and stream them back
:      lda     FN_RPLY,x
      jsr     fn_txb
      inx
      bne     :-
```

C (CC65 + FUJINET-LIB)

```
strcpy((char *) hosts[3], "ec.tnfs.io");
fuji_put_host_slots(hosts, 8);
```

MOUNT HOST

DEV \$70 CMD \$F9

PARAMS	host slot (1), 0-7.
PAYLOAD	none
REPLY	none; NAK if the slot is empty or the host cannot be reached.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVF, FNCMHST, 1
lda     #0         ; host 1, the SD card
jsr     fn_pb
jsr     fn_go
jne     fail
jsr     fn_ack
jne     fail
```

C (CC65 + FUJINET-LIB)

```
if (!fuji_mount_host_slot(0))
    fail();
```

UNMOUNT HOST

DEV \$70 CMD \$E6

PARAMS	host slot (1).
PAYLOAD	none
REPLY	none; NAK if that host was not mounted.

Also unmounts every device slot using that host.

6502 ASSEMBLY (CA65)

CALL	FNDEVF, FNCUHST, 1
lda	#7
jsr	fn_pb
jsr	fn_go

C (CC65 + FUJINET-LIB)

fuji_unmount_host_slot(7);

SET / GET HOST PREFIX

DEV \$70 CMD \$E1 / \$E0

PARAMS	host slot (1).
PAYLOAD	SET: the prefix, NUL-terminated, up to 256 bytes.
REPLY	GET: 256 bytes, the prefix.

A prefix is a directory that paths on that host are taken relative to.

6502 ASSEMBLY (CA65)

CALL	FNDEVF, FNCGPFX, 1
lda	#0
jsr	fn_pb
jsr	fn_go

C (CC65 + FUJINET-LIB)

fuji_get_host_prefix(0, path);

READ DEVICE SLOTS

DEV \$70 CMD \$F2

PARAMS	none
PAYLOAD	none
REPLY	304 bytes: eight slots of 38 – host slot (1, \$FF empty), mode (1: 1 read, 2 write, plus \$40 when mounted), file name (36).

6502 ASSEMBLY (CA65)

CALL	FNDEVF, FNCRDEV, 0
jsr	fn_go ; slot n at FN_RPLY + n*38

C (CC65 + FUJINET-LIB)

fuji_get_device_slots(slots, 8);

WRITE DEVICE SLOTS

DEV \$70 CMD \$F1

PARAMS	none
PAYLOAD	304 bytes: all eight slots.
REPLY	none

304 bytes is under the 320 the TX stream holds, so it goes in one transaction.

6502 ASSEMBLY (CA65)

CALL	FNDEVF, FNCRDEV, 0
jsr	fn_go ; then stream 304 back
CALL	FNDEVF, FNCWDEV, 0

C (CC65 + FUJINET-LIB)

slots[1].hostSlot = 0xFF; /* empty it */
fuji_put_device_slots(slots, 8);

SET DEVICE FULL PATH

DEV \$70 CMD \$E2

PARAMS	device slot (1), host slot (1), mode (1).
PAYLOAD	The path, NUL-terminated, up to 256 bytes.
REPLY	none; NAK for a bad slot. An empty path empties the slot.

fujinet-lib's `fuji_set_device_filename(mode, host, device, path)` takes its arguments in the opposite order to the wire, and always sends 256 bytes from path – so path must be a 256-byte buffer.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVF, FNCSDFP, 3
lda     #0           ; device slot
jsr     fn_pb
lda     #0           ; host slot
jsr     fn_pb
lda     #1           ; read
jsr     fn_pb
lda     #<path
sta     fn_ptr
lda     #>path
sta     fn_ptr+1
jsr     fn_path
jsr     fn_go
```

C (CC65 + FUJINET-LIB)

```
strcpy(path, "/nesbook/hello.bin");
/* note the order: mode, host, device */
fuji_set_device_filename(1, 0, 0, path);
```

GET DEVICE FULL PATH

DEV \$70 CMD \$DA

PARAMS	device slot (1).
PAYLOAD	none
REPLY	256 bytes: the path.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVF, FNCGDFP, 1
lda     #0
jsr     fn_pb
jsr     fn_go
```

C (CC65 + FUJINET-LIB)

```
fuji_get_device_filename(0, path);
```

UNMOUNT IMAGE

DEV \$70 CMD \$E9

PARAMS	device slot (1).
PAYLOAD	none
REPLY	none

6502 ASSEMBLY (CA65)

```
CALL    FNDEVF, FNCUIMG, 1
lda     #0
jsr     fn_pb
jsr     fn_go
```

C (CC65 + FUJINET-LIB)

```
fuji_unmount_disk_image(0);
```

MOUNT ALL

DEV \$70 CMD \$D7

PARAMS	none
PAYLOAD	none
REPLY	none; NAK if any slot fails. A cartridge image in a slot is pushed to the cartridge, as MOUNT_IMAGE would.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVF, FNCMALL, 0
jsr     fn_go
```

C (CC65 + FUJINET-LIB)

```
fuji_mount_all();
```

MOUNT_IMAGE, \$F8, which loads a cartridge image, has Chapter 8 to itself.

NEW DISK

DEV \$70 CMD \$E7

PARAMS	none
PAYLOAD	262 bytes: sector count (2), sector size (2), host slot (1), device slot (1), file name (256).
REPLY	none; NAK if the file exists or cannot be written.

Creates a blank disk image. Of little use to a NES program, but there.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVF, FNCNEWD, 0
lda     #<720          ; sectors
jsr     fn_txb
lda     #>720
jsr     fn_txb
; then size, host, slot, 256-byte name
```

C (CC65 + FUJINET-LIB)

```
/* no NES wrapper: NewDisk is not defined */
fuji_bus_call(FUJI_DEVICEID_FUJINET, 0xE7,
              FUJI_FIELD_DATA, 0, 0, 0, 0,
              buf, 262);
```

COPY FILE

DEV \$70 CMD \$D8

PARAMS	source host (1), destination host (1) – counted from 1.
PAYLOAD	source destination. A destination ending in / gets the source's file name – only if nothing follows the string.
REPLY	none, once the copy is done. The cartridge allows it 60 seconds.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVF, FNCCOPY, 2
lda     #8              ; from host 8
jsr     fn_pb
lda     #1              ; to host 1
jsr     fn_pb
lda     #<data
sta     fn_ptr
lda     #>data
sta     fn_ptr+1
jsr     fn_str          ; exact length
jsr     fn_go
```

C (CC65 + FUJINET-LIB)

```
/* sends 256 bytes: give a full name */
fuji_copy_file(8, 1, (char *)
              "/nes/x.nes|/nesbook/x.nes");
```

CONFIG BOOT

DEV \$70 CMD \$D9

PARAMS	enable (1).
PAYLOAD	none
REPLY	none

Whether the FujiNet serves CONFIG at boot. A successful MOUNT_IMAGE already turns it off.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVF, FNCCBOOT, 1
lda     #0
jsr     fn_pb
jsr     fn_go
```

C (CC65 + FUJINET-LIB)

```
fuji_set_boot_config(0);
```

SET BOOT MODE

DEV \$70 CMD \$D6

PARAMS	mode (1).
PAYLOAD	none
REPLY	none

Mounts one of the FujiNet's boot disks into device slot 0. Meant for disk computers; meaningless on the NES.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVF, FNCBMODE, 1
lda     #0
jsr     fn_pb
jsr     fn_go
```

C (CC65 + FUJINET-LIB)

```
fuji_set_boot_mode(0);
```

DIRECTORIES

OPEN DIRECTORY

DEV \$70 CMD \$F7

PARAMS	host slot (1).
PAYLOAD	Up to 256 bytes: the path, a NUL, and an optional filter such as *.nes, then a NUL.
REPLY	none; NAK if the host is not mounted or the path does not exist.

Opening a directory closes any other.

6502 ASSEMBLY (CA65)

CALL	FNDEVF, FNCODIR, 1
lda	#0
jsr	fn_pb
lda	#<path
sta	fn_ptr
lda	#>path
sta	fn_ptr+1
jsr	fn_path ; 256, no filter
jsr	fn_go

C (CC65 + FUJINET-LIB)

memset(path, 0, sizeof path);
path[0] = '/';
fuji_open_directory(0, path);

READ DIRECTORY ENTRY

DEV \$70 CMD \$F6

PARAMS	length (1), flags (1).
PAYLOAD	none
REPLY	Exactly length bytes: the name, NUL-padded, shortened with an ellipsis if it does not fit; a directory ends in /. \$7F \$7F is the end. NAK if no directory is open.

With flags bit 7 set, twelve bytes come first: year minus 1970, month, day, hour, minute, second; the size (4 bytes); flags (bit 0 directory, bit 1 name cut short); and a media type – 0 unknown, 1 disk image, 2 cartridge image (.nes, .bin, .rom and the like), 3 IMD.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVF, FNCRDIR, 2
lda     #28                ; length
jsr     fn_pb
lda     #0                  ; flags
jsr     fn_pb
jsr     fn_go
lda     FN_RPLY
cmp     #$7F                ; the end?
```

C (CC65 + FUJINET-LIB)

```
if (fuji_read_directory(28, 0, buf)
    && buf[0] != 0x7F)
    cputs((char *) buf);
```

CLOSE DIRECTORY

DEV \$70 CMD \$F5

PARAMS	none
PAYLOAD	none
REPLY	none

6502 ASSEMBLY (CA65)

```
CALL    FNDEVF, FNCCDIR, 0
jsr     fn_go
```

C (CC65 + FUJINET-LIB)

```
fuji_close_directory();
```

GET / SET DIRECTORY POSITION

DEV \$70 CMD \$E5 / \$E4

PARAMS	SET: position (2).
PAYLOAD	none
REPLY	GET: 2 bytes, the position.

How CONFIG pages: SET the position of the page's first entry, then sixteen READs.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVF, FNCSDPS, 1
lda     #<16          ; entry 16
ldx     #>16
jsr     fn_pw
jsr     fn_go
```

C (CC65 + FUJINET-LIB)

```
fuji_set_directory_position(16);
fuji_get_directory_position(&pos);
```

APP KEY COMMANDS

OPEN APP KEY

DEV \$70 CMD \$DC

PARAMS	none
PAYLOAD	6 bytes: creator (2), app (1), key (1), mode (1: 0 read, 1 write), reserved (1).
REPLY	none; NAK if creator is 0 or there is no SD card.

All six bytes, always: the FujiNet does not clear the record first.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVF, FNCOKEY, 0
lda     #$5E          ; creator, low
jsr     fn_txb
lda     #$5E          ; and high
jsr     fn_txb
lda     #1            ; app
jsr     fn_txb
lda     #0            ; key
jsr     fn_txb
lda     #0            ; mode: read
jsr     fn_txb
jsr     fn_txb        ; reserved
jsr     fn_go
```

C (CC65 + FUJINET-LIB)

```
/* fujinet-lib opens for you */
fuji_set_appkey_details(0x5E5E, 1, DEFAULT);
```

READ APP KEY

DEV \$70 CMD \$DD

PARAMS	none
PAYLOAD	none
REPLY	2 bytes of length, low first, then that many bytes, 0 to 64.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVF, FNCRKEY, 0
jsr     fn_go
jne     fail
lda     FN_RPLY          ; length, low
; the key from FN_RPLY+2
```

C (CC65 + FUJINET-LIB)

```
if (fuji_read_appkey(0, &len, buf))
    buf[len] = 0;
```

WRITE APP KEY

DEV \$70 CMD \$DE

PARAMS	none
PAYLOAD	The key's new value, up to 64 bytes.
REPLY	none; NAK unless it was opened for writing.

A write, good or bad, ends the open: open again before the next one.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVF, FNCWKEY, 0
lda     #42
jsr     fn_txb
jsr     fn_go
```

C (CC65 + FUJINET-LIB)

```
fuji_write_appkey(0, 5,
    (uint8_t *) "HELLO");
```

CLOSE APP KEY

DEV \$70 CMD \$DB

PARAMS none
PAYLOAD none
REPLY none

6502 ASSEMBLY (CA65)

```
CALL FNDEVF, FNCCKEY, 0  
jsr fn_go
```

C (CC65 + FUJINET-LIB)

```
/* no wrapper */  
fuji_bus_call(FUJI_DEVICEID_FUJINET, 0xDB,  
FUJI_FIELD_NONE, 0, 0, 0, 0,  
NULL, 0);
```

THE ADAPTER

GET ADAPTER CONFIG

DEV \$70 CMD \$E8 / \$C4

PARAMS none
PAYLOAD none
REPLY \$E8: 140 bytes. \$C4, the extended form: 240 bytes, the same 140 followed by the addresses as text. Laid out in Chapter 5.

6502 ASSEMBLY (CA65)

```
CALL FNDEVF, FNCADPX, 0  
jsr fn_go
```

C (CC65 + FUJINET-LIB)

```
fuji_get_adapter_config(&ac);  
fuji_get_adapter_config_extended(&acx);
```

STATUS

DEV \$70 CMD \$53

PARAMS none
PAYLOAD none
REPLY 4 bytes, always zero.

6502 ASSEMBLY (CA65)

```
CALL FNDEVF, FNCSTAT, 0  
jsr fn_go
```

C (CC65 + FUJINET-LIB)

```
fuji_status((FNStatus *) buf);
```

DEVICE READY

DEV \$70 CMD \$00

PARAMS	none
PAYLOAD	none
REPLY	512 bytes of 'A'.

A round-trip test of the link: a big, known reply.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVF, FNCREADY, 0
jsr     fn_go          ; FN_RXLO/HI = 512
```

C (CC65 + FUJINET-LIB)

```
fjui_bus_call(FUJI_DEVICEID_FUJINET, 0x00,
              FUJI_FIELD_REPLY, 0, 0, 0, 0,
              buf, 512);
```

RESET

DEV \$70 CMD \$FF

PARAMS	none
PAYLOAD	none
REPLY	none – the FujiNet restarts without answering, so the transaction times out and the library reports failure.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVF, FNCRESET, 0
jsr     fn_go          ; returns ERR 2, timeout
```

C (CC65 + FUJINET-LIB)

```
fjui_reset(); /* returns false */
```

JOBS FOR THE COPROCESSOR

RANDOM NUMBER

DEV \$70 CMD \$D3

PARAMS	none
PAYLOAD	none
REPLY	4 bytes, a random 32-bit number, low first.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVF, FNCRAND, 0
jsr     fn_go
lda     FN_RPLY          ; four random bytes
```

C (CC65 + FUJINET-LIB)

```
/* no wrapper */
fuji_bus_call(FUJI_DEVICEID_FUJINET, 0xD3,
              FUJI_FIELD_REPLY, 0, 0, 0, 0,
              &r32, 4);
```

GENERATE GUID

DEV \$70 CMD \$BB

PARAMS	none
PAYLOAD	none
REPLY	37 bytes: a version 4 UUID as text, lower-case, with its NUL.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVF, FNCGUID, 0
jsr     fn_go
```

C (CC65 + FUJINET-LIB)

```
fuji_generate_guid(guid);
```

PARAMS	INPUT and OUTPUT: length (2).
PAYLOAD	INPUT: the bytes.
REPLY	LENGTH: 4 bytes. OUTPUT: the bytes, taken from the front.

Encoding is \$D0 input, \$CF compute, \$CE length, \$CD output; decoding is \$CC, \$CB, \$CA, \$C9. Both use one buffer on the FujiNet: INPUT adds to it, COMPUTE replaces it with the result, OUTPUT takes from the front.

6502 ASSEMBLY (CA65)

```

CALL    FNDEVF, FNCB64EI, 1
lda     #5
ldx     #0
jsr     fn_pw
ldx     #0
lda     data,x
jsr     fn_txb
inx
cpx     #5
bne     :-
jsr     fn_go
CALL    FNDEVF, FNCB64EC, 0
jsr     fn_go

```

C (CC65 + FUJINET-LIB)

```

fuji_base64_encode_input("HELLO", 5);
fuji_base64_encode_compute();
fuji_base64_encode_length(&blen);
fuji_base64_encode_output(buf, 8);

```

HASH

DEV \$70 CMD \$C8-\$C2

PARAMS	INPUT: length (2). COMPUTE: algorithm (1): 0 MD5, 1 SHA-1, 2 SHA-256, 3 SHA-512, 4 SHA-224, 5 SHA-384. LENGTH and OUTPUT: 1 for hex, 0 for binary.
PAYLOAD	none
REPLY	LENGTH: 1 byte. OUTPUT: the digest.

\$C8 input, \$C7 compute (and forget the input), \$C3 compute and keep it, \$C6 length, \$C5 output, \$C2 clear.
fujinet-lib names only the first four algorithms.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVF, FNCHCMP, 1
lda     #2                ; SHA-256
jsr     fn_pb
jsr     fn_go
CALL    FNDEVF, FNCHOUT, 1
lda     #1                ; as hex
jsr     fn_pb
jsr     fn_go
```

C (CC65 + FUJINET-LIB)

```
fuji_hash_data(SHA256, (uint8_t *) "HELLO",
               5, true, buf);
```

PARAMS	INPUT and OUTPUT: length (2). ENCODE: version (1, 0 for the smallest), error correction (1: 0-3), shorten the URL (1). LENGTH: format (1): 0 binary, 1 ANSI, 2 bitmap, 3 SVG, 4 ATASCII, 5 PETSCII.
PAYLOAD	none
REPLY	LENGTH: 4 bytes. OUTPUT: the code, from the front.

\$BC input, \$BD encode, \$BE length, \$BF output; in C, include `fujinet-qrcode.h`. In binary format each module is one byte, row by row. An OUTPUT is limited by the 1K reply window.

6502 ASSEMBLY (CA65)

```

CALL    FNDEVF, FNCQENC, 3
lda     #0           ; version: auto
jsr     fn_pb
lda     #1           ; ecc: medium
jsr     fn_pb
lda     #0           ; don't shorten
jsr     fn_pb
jsr     fn_go

```

C (CC65 + FUJINET-LIB)

```

n16 = qrcreate(0, QR_ECC_MEDIUM, false,
               QR_OUTPUT_MODE_BINARY, "FUJINET", 7,
               buf, sizeof buf);

```

The Fuji device NAKs \$F0, \$EB, \$E3, \$DF, \$D5, \$D4, \$D2 (GET_TIME – use the clock device), \$D1, \$C1, \$A0-\$A9, \$90 and \$3F.

8. *BOOTING AN IMAGE*

A program can load another program: any NES cartridge image the FujiNet can reach, up to 512K of PRG and 512K of CHR, fetched off the network into the cartridge's own memory and started as if it had been plugged in.

FOUR STEPS

- a) **MOUNT_HOST** the host the image is on.
- b) **SET_DEVICE_FULLPATH** – device slot 0, that host, mode 1 (read), and the image's path.
- c) **MOUNT_IMAGE** device slot 0. The FujiNet sees from the file's extension that it is a cartridge image, and before it answers it pushes the whole file down the USB link into a staging store on the cartridge – 256K of RAM, or 1.5 MB of flash for bigger images. Then **BOOT_STATE** says **READY**.
- d) **Boot**: store \$B5 to **BOOTLOCK**, turn interrupts, the PPU and the APU off, and jump to the loader ROM at \$5800. It copies the staged image into the SRAMs, sets up its mapper, and starts it through its own reset vector.

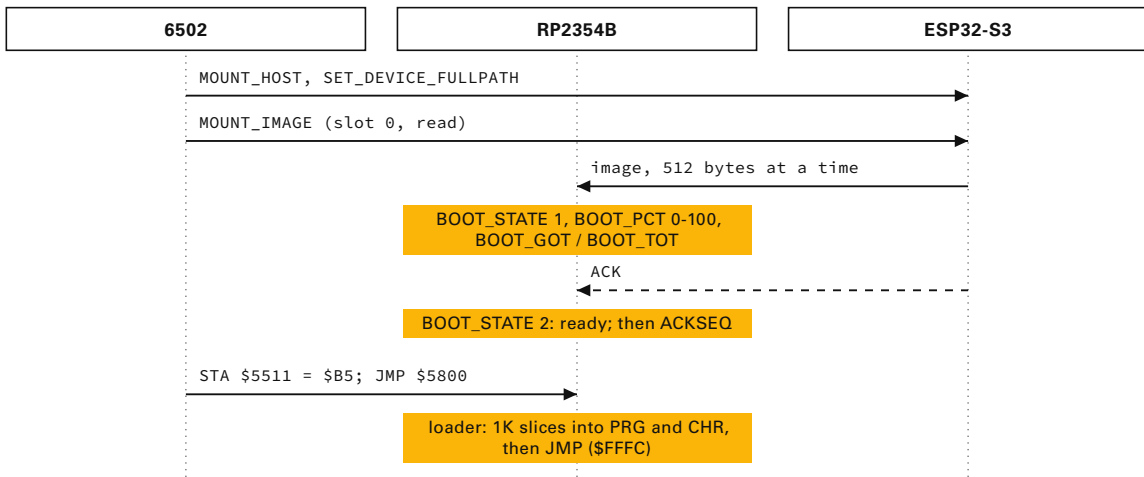


Fig. 10 – A network boot. The image crosses the link while MOUNT_IMAGE is still waiting for its answer.

IN C

LISTINGS/C/BOOT.C

```
/* boot.c -- fetch a cartridge image from the SD card and run it. */

#include <conio.h>
#include <fujiinet-fuji.h>
#include <fujiinet-nes.h>

#define HOST 0 /* host slot 1: the SD card */
#define SLOT 0 /* device slot 1 */
#define MODE_READ 1 /* disk access mode: read only */

static char path[256] = "/nesbook/hello.bin";

void main(void)
{
    clrscr();
    printf("BOOTING %s\r\n\r\n", path);

    if (!fuji_mount_host_slot(HOST)
        || !fuji_set_device_filename(MODE_READ, HOST, SLOT, path)) {
        cputs("CANNOT MOUNT");
        for (;;) ;
    }

    /* MOUNT_IMAGE does not answer until the whole image has crossed the
       link into the cartridge's staging store. */
    if (!fuji_mount_disk_image(SLOT, MODE_READ)
        || fuji_nes_boot_state() != FUJI_NES_BOOT_READY) {
        printf("FAILED, BOOT ERROR %u", fuji_nes_boot_error());
        for (;;) ;
    }

    printf("%lu BYTES STAGED\r\n", fuji_nes_boot_total());
    fuji_set_boot_config(0); /* don't come back to CONFIG */
    fuji_nes_boot(); /* never returns */
}
```

`fuji_mount_disk_image()` is one transaction, and it does not come back until the image is on the cartridge. `fuji_nes_boot()` does the rest: BOOTLOCK, the interrupts and the PPU and APU off, and the jump.

CAUTION: fujinet-lib waits about twelve seconds for any transaction. The cartridge gives MOUNT_IMAGE sixty. A large image on a slow network can take longer than the library will wait: the call returns false while the image is still arriving. For anything bigger than a few dozen kilobytes, start the mount yourself and watch it, as the assembly version does.

IN ASSEMBLY, WITH PROGRESS

boot.s launches MOUNT_IMAGE with fn_launch, which commits and returns at once, and then watches BOOT_PCT once a frame – writing to the PPU only in vblank, since the screen is on – until fn_done says the answer is in:

LISTINGS/ASM/BOOT.S: LAUNCH AND WATCH

```

CALL    FNDEVF, FNCMIMG, 2      ; MOUNT_IMAGE(slot, mode)
lda     #SLOT
jsr     fn_pb
lda     #MREAD
jsr     fn_pb
jsr     fn_launch                ; sent; do not wait here

lda     #LIMIT
sta     secs
lda     #60
sta     tick
watch:  jsr     wait_vbl          ; vblank: safe to touch the PPU
lda     #4
ldx     #10
jsr     disp_at
lda     FN_BPC                  ; 0-100, painted by the cart
jsr     disp_d3
jsr     scroll0
jsr     fn_done                 ; MOUNT_IMAGE answered?
bcs     answered
dec     tick
bne     watch
lda     #60
sta     tick
dec     secs
bne     watch
lda     #FNEWAIT
jmp     fail

answered:
lda     FN_ERR
jne     fail
jsr     fn_ack
jne     fail
lda     FN_BST
cmp     #FN_BRDY
jne     bootfail
jsr     fn_blk                  ; BOOTLOCK = $B5
jmp     fn_boot                 ; SEI, PPU off, JMP $5800

```



Fig. 11 – boot.s loading, and the program it loaded: hello from the SD card. A 40K image crosses the link in a quarter of a second in emulation.

It waits sixty-five seconds, so that a real timeout is reported by the cartridge's ERR rather than by its own loop. CONFIG does the same, and draws a bar from BOOT_PCT and a byte count from BOOT_GOT and BOOT_TOT.

From C, the same is done by starting the transaction by hand:

STARTING MOUNT_IMAGE WITHOUT WAITING, IN C

```
fn_regwr(FNR_DATA_RST, 0);
fn_regwr(FNR_DEVICE, 0x70);
fn_regwr(FNR_CMD, 0xF8);           /* MOUNT_IMAGE */
fn_regwr(FNR_NPARAM, 2);
fn_tx(1); fn_tx(0);               /* device slot 0 */
fn_tx(1); fn_tx(1);               /* mode: read */
want = FN_ACKSEQ + 1;
if (want == 0)
    want = 1;
fn_regwr(FNR_SEQ, want);          /* sent */
while (FN_ACKSEQ != want) {
    waitvsync();
    draw_bar(fuji_nes_boot_percent()); /* 0-100 */
}
```

THE COMMAND

MOUNT IMAGE

DEV \$70 CMD \$F8

PARAMS	device slot (1), mode (1): 1 read, 2 write.
PAYLOAD	none
REPLY	none – after the whole image is on the cartridge. NAK for a bad slot, a file that will not open, or a push that failed.

Starting a mount clears `BOOT_STATE`, `BOOT_PCT`, `BOOT_ERR` and the byte counts, so whatever an earlier attempt left there cannot be mistaken for this one's result. A mount that succeeds also turns off "boot CONFIG", so calling `fuji_set_boot_config(0)` afterwards, as the games do, is harmless but not needed.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVF, FNCMIMG, 2
lda     #0           ; device slot 0
jsr     fn_pb
lda     #1           ; read
jsr     fn_pb
jsr     fn_launch    ; then watch
```

C (CC65 + FUJINET-LIB)

```
if (fuji_mount_disk_image(0, 1)
    && fuji_nes_boot_state()
    == FUJI_NES_BOOT_READY)
    fuji_nes_boot();
```

WHICH FILES ARE IMAGES

The FujiNet decides by the file's extension. `.nes`, `.bin`, `.rom`, `.int`, `.itv` and `.chf` are cartridge images; anything else mounts as a disk, and `BOOT_STATE` never reaches `READY`.

NOTE: The `.nes` extension was added to the FujiNet firmware in October 2026. The `fujinet-pc` these examples ran against predates it, which is why boot loads `/nesbook/hello.bin` – the same iNES file, renamed.

An `.nes` image carries its own iNES header, and that is what the cartridge reads to choose the mapper. If it names a mapper the cartridge does not implement, the load fails.

WHEN IT FAILS

BOOT_ERR	Name	Meaning
1	TOOBIG	The image is bigger than the staging store, or than the SRAMs.

2	TRUNCATED	The stream ended before the size it declared.
3	NOMAP	The iNES header names a mapper this cartridge cannot do.
4	STOREBUSY	The staging store was busy.

BOOT_STATE \$80 means failed, and BOOT_ERR says why. A MOUNT_IMAGE that NAKs leaves BOOT_STATE at 0.

INSIDE THE LOADER

You never need to drive the loader yourself – `fn_boot` and `fujii_nes_boot()` simply jump to it – but it is short and instructive. It lives in the cartridge at \$5800–\$5FFF, served by the RP2354B like the mailbox, so the copy it performs cannot overwrite it, and nothing has to be moved into console RAM first.

- It waits two vblanks, then stores to \$55FE to ask for the load.
- The cartridge paints a 1K slice of the image into the reply window and then `LOAD_SEQ`. `LOAD_DST` says where it goes: 0 for PRG, to \$8000 plus `LOAD_OFF` kilobytes – the cartridge has moved the PRG bank under that window – or 1 for CHR, through \$2006 and \$2007.
- The loader copies it, about 9.5 cycles a byte, and stores `LOAD_SEQ` to `SLICE_ACK`, \$5514.
- At `LOAD_STATE` 2, done, it acks once more and jumps through the new image's reset vector at \$FFFC.

LISTINGS: LOADER.S, THE COPY LOOP

```
loop:  lda    FN_LSTATE
      cmp    #FN_LDONE
      bne    :+
      jmp    done
:      cmp    #FN_LFAIL
      bne    :+
      jmp    fail
:      cmp    #FN_LSLICE
      bne    loop
      lda    FN_LSEQ
      cmp    lseq
      beq    loop
      sta    lseq
      lda    FN_LOFF
      cmp    #8
      bcs    loop          ; not a value the cart publishes: a missed cycle
      ldy    FN_LDST
      beq    prg
      cpy    #1
      bne    loop
      jmp    chr
```

COMING BACK

There is no way back from a booted image except the power switch. When the console is off, the cartridge sees the CPU's clock stop for a quarter of a second, switches its SRAMs off, and on the next power-on loads CONFIG again. Pressing Reset only restarts the image that is loaded.

9. THE CLOCK AND OTHER DEVICES

The NES has no clock of its own. The FujiNet has one, set from the network, and it will tell you the time in any of seven formats.

WHAT TIME IS IT?

LISTINGS/C/CLOCK.C

```
/* clock.c -- the FujiNet's network time, once a second. */

#include <conio.h>
#include <time.h>
#include <fujinet-clock.h>

static uint8_t now[26];

/* One second of vblanks, counted by cc65's NMI handler in clock(). */
static void second(void)
{
    clock_t t = clock() + 60;

    while (clock() < t)
        ;
}

void main(void)
{
    clrscr();
    cputs("CLOCK\r\n\r\n");

    for (;;) {
        if (clock_get_time(now, TZ_ISO_STRING) == FN_ERR_OK) {
            gotoxy(0, 3);
            cprintf("%.10s\r\n%.8s", now, now + 11);    /* date, then time */
        }
        second();
    }
}
```

clock_get_time() writes a NUL after the reply, so give it a buffer one byte longer than the format. The ISO string is 25 characters – with its own NUL, 26 – so now is 26 bytes.

In assembly the time is drawn straight out of the reply window, in vblank, since this program keeps the screen on while it waits:

LISTINGS/ASM/CLOCK.S: ONCE A SECOND

```
loop:  CALL    FNDEVC, CLKISO, 0
      jsr     fn_go                ; runs with the screen on: the NMI
      bne     wait                ; may land anywhere in it
      jsr     fn_ack
      bne     wait

      jsr     wait_vbl
      lda     #5
      ldx     #2
      jsr     disp_at
      ldx     #0                  ; "2026-10-04"
      ldy     #10
      jsr     disp_rpl
      lda     #6
      ldx     #2
      jsr     disp_at
      ldx     #11                 ; "18:20:00"
      ldy     #8
      jsr     disp_rpl
      jsr     scroll0
```



Fig. 12 – The FujiNet's clock, in C and in assembly. This FujiNet's zone is UTC.

THE CLOCK DEVICE

Every read command takes one optional parameter: 1 to give the time in the **alternate** time zone set by SETTZ, \$99, rather than the FujiNet's own.

GET TIME, ISO 8601

DEV \$45 CMD \$49 'I' / \$5A 'Z'

PARAMS	alternate zone (1), optional.
PAYLOAD	none
REPLY	25 bytes: 2026-10-04T18:20:00-0500 and a NUL. \$49 in the FujiNet's zone, \$5A in UTC.

6502 ASSEMBLY (CA65)

CALL	FNDEVC, CLKISO, 0
jsr	fn_go
jne	fail ; the string at FN_RPLY

C (CC65 + FUJINET-LIB)

clock_get_time(now, TZ_ISO_STRING);
clock_get_time(now, UTC_ISO_STRING);

GET TIME, BINARY

DEV \$45 CMD \$93 / \$9A

PARAMS	alternate zone (1), optional.
PAYLOAD	none
REPLY	6 bytes: day, month, year minus 2000, hour, minute, second. \$9A always uses the alternate zone.

6502 ASSEMBLY (CA65)

CALL	FNDEVC, CLKTIME, 0
jsr	fn_go
jne	fail
lda	FN_RPLY+3 ; the hour

C (CC65 + FUJINET-LIB)

clock_get_time(now, APETIME_BINARY);

GET TIME, SIMPLE

DEV \$45 CMD \$54 'T' / \$4D 'M'

PARAMS	alternate zone (1), optional.
PAYLOAD	none
REPLY	\$54: 7 bytes – century (20), year, month, day, hour, minute, second. \$4D: the same and hundredths of a second, 8 bytes.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVC, CLKSIMP, 0
jsr     fn_go
```

C (CC65 + FUJINET-LIB)

```
clock_get_time(now, SIMPLE_BINARY);
```

GET TIME, PRODOS AND SOS

DEV \$45 CMD \$50 'P' / \$53 'S'

PARAMS	alternate zone (1), optional.
PAYLOAD	none
REPLY	\$50: ProDOS's 4-byte date and time. \$53: Apple III SOS, 19 bytes, YYYYMMDD0HHMMSS000 and a NUL.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVC, CLKPRODOS, 0
jsr     fn_go
```

C (CC65 + FUJINET-LIB)

```
clock_get_time(now, PRODOS_BINARY);
```

GET TIME ZONE

DEV \$45 CMD \$47 'G' / \$4C 'L'

PARAMS	none
PAYLOAD	none
REPLY	\$47: the zone string and a NUL, UTC if none is set. \$4C: one byte, its length plus one.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVC, CLKGTZ, 0
jsr     fn_go
```

C (CC65 + FUJINET-LIB)

```
clock_get_tz((char *) buf);
```

SET TIME ZONE

DEV \$45 CMD \$74 't' / \$99

PARAMS	none
PAYLOAD	A POSIX time zone such as EST5EDT or CET-1CEST.
REPLY	none; NAK for an empty payload.

\$74 sets the FujiNet's own zone and saves it. \$99 sets the alternate zone, in memory only, for reads that ask for it. A zone that does not make sense is ignored – but still answered with an ACK.

6502 ASSEMBLY (CA65)

	CALL	FNDEVC, CLKSETS, 0
	ldx	#0
:	lda	data,x
	beq	:+
	jsr	fn_txb
	inx	
	bne	:-
:	jsr	fn_go

C (CC65 + FUJINET-LIB)

<pre>clock_set_tz("EST5EDT"); clock_get_time_tz(now, "CET-1CEST", TZ_ISO_STRING);</pre>

The clock NAKs \$7A, \$73, \$70, \$69, \$61 and \$41: those are other computers' spellings. Use the alternate-zone parameter instead.

DISKS

Devices \$31 to \$38 are the eight device slots seen as disk drives, in 512-byte sectors. A NES program has little use for them, but they work. A slot holding a cartridge image has no sectors: every command to it NAKs.

READ SECTOR

DEV \$31 CMD \$52 'R'

PARAMS	sector (4), from 0.
PAYLOAD	none
REPLY	512 bytes; NAK past the end, or if nothing is mounted.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVD, DKREAD, 1
lda     #4          ; size: 4 bytes
jsr     fn_txb
lda     #0          ; sector 0
jsr     fn_txb
jsr     fn_txb
jsr     fn_txb
jsr     fn_txb
jsr     fn_txb
jsr     fn_go
```

C (CC65 + FUJINET-LIB)

```
fuji_bus_call(FUJI_DEVICEID_DISK, 'R',
              FUJI_FIELD_C1234 | FUJI_FIELD_REPLY,
              0, 0, 0, 0, buf, 512);
```

WRITE SECTOR

DEV \$31 CMD \$57 'W' / \$50 'P'

PARAMS	sector (4).
PAYLOAD	512 bytes.
REPLY	none. \$57 verifies the write; \$50 does not.

CAUTION: a sector is bigger than the TX stream's 320 bytes. On the NES a disk is in practice read-only.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVD, DKWRITE, 1
; 4-byte sector, then 512 bytes:
; more than the TX stream holds
```

C (CC65 + FUJINET-LIB)

```
/* 4 + 1 + 512 bytes: too many for one
   NES transaction */
```

The other disk commands – \$53 STATUS, which this firmware wrongly treats as a write; \$21 and \$22 FORMAT, which return a sector of status; and \$4E and \$4F, the PERCOM block – are of no use to an NES program.

THE PRINTER

PRINT

DEV \$40 CMD \$57 'W' / \$50 'P'

PARAMS	none
PAYLOAD	The bytes to print, up to 320.
REPLY	none. If the printer is switched off in the FujiNet's configuration there is no reply at all, and the transaction times out.

The FujiNet's printer writes what it receives to a file, which you fetch from its web page. STATUS, \$53, takes one parameter and answers four bytes.

6502 ASSEMBLY (CA65)

```
CALL    FNDEVP, PRWRITE, 0
:      ldx    #0
      lda    data,x
      beq    :+
      jsr    fn_txb
      inx
      bne    :-
:      jsr    fn_go
```

C (CC65 + FUJINET-LIB)

```
fjui_bus_call(FUJI_DEVICEID_PRINTER, 'W',
              FUJI_FIELD_DATA, 0, 0, 0, 0,
              "HELLO\r\n", 7);
```

THE MODEM

The FujiNet's modem, device \$50, is there – but its replies are sent as raw, unframed bytes, the way a real modem talks, and a command whose reply has no data sends nothing at all. The cartridge waits for a FujiBus frame that never comes, and every modem command ends as a timeout. Use the network device's TCP and Telnet channels instead.

10. THE LIBRARIES

Two libraries stand between a program and the mailbox. One is a page of assembly you have already read. The other is `fujinet-lib`, and its NES half is a little over two hundred lines of C.

FUJINET-LIB ON THE NES

`fujinet-lib` is the same library on every FujiNet computer. Its common code – `network_open()`, `fuji_read_appkey()`, `clock_get_time()` and the rest – turns every call into one or more **bus calls**:

THE ONE FUNCTION EVERY PLATFORM IMPLEMENTS

```
bool fuji_bus_call(uint8_t device, uint8_t fuji_cmd, uint8_t fields,
                  uint8_t aux1, uint8_t aux2, uint8_t aux3, uint8_t aux4,
                  const void *buf, size_t buf_length);
```

`fields` says how the four aux bytes are to be sent – none, one to four single bytes, one or two 16-bit values, or one 32-bit value – and whether `buf` is a payload going out (`FUJI_FIELD_DATA`) or a place for the reply (`FUJI_FIELD_REPLY`). The NES implementation maps that straight onto the mailbox:

FUJINET-LIB BUS/NES/FUJINET-BUS-NES.C

```
numbytes = fuji_field_numbytes(fields);
numfields = fuji_field_numfields(fields);
size = numfields ? (uint8_t) (numbytes / numfields) : 0;

/* Each parameter costs its size byte plus its value bytes. Refuse rather
   than truncate: a short SET_DEVICE_FULLPATH is rejected on the ESP32 side
   anyway, and silently dropping the tail would be far harder to see. */
if ((fields & FUJI_FIELD_DATA)
    && (uint16_t) (numfields + numbytes) + buf_length > FN_TX_MAX)
    return false;

fn_regwr(FNR_DATA_RST, 0); /* rewind the TX write pointer */
fn_regwr(FNR_DEVICE, device);
fn_regwr(FNR_CMD, fuji_cmd);
fn_regwr(FNR_NPARAM, 0);

idx = 0;
for (field = 0; field < numfields; field++) {
    fn_tx(size);
    for (i = 0; i < size; i++)
        fn_tx(aux[idx++]);
}
```

```

}
if (numfields)
    fn_regwr(FNR_NPARAM, numfields);

if (fields & FUJI_FIELD_DATA) {
    const uint8_t *data = (const uint8_t *) buf;
    size_t n = buf_length;

    while (n--)
        fn_tx(*data++);
}

if (fn_commit() != FN_OK)
    return false;

if (FN_REPLYCMD != FUJICMD_ACK)
    return false;

rlen = (uint16_t) FN_RXLEN_LO | ((uint16_t) FN_RXLEN_HI << 8);
if (rlen > FN_REPLY_MAX)
    rlen = FN_REPLY_MAX;

if (fields & FUJI_FIELD_REPLY) {
    uint8_t *reply = (uint8_t *) buf;
    volatile uint8_t *src = FN_REPLY;
    uint16_t n;

    if (rlen > buf_length)
        rlen = buf_length;
    for (n = rlen; n; n--)
        *reply++ = *src++;
}

fuji_bus_call_rlen = rlen;

```

Three things to notice. It refuses, rather than truncates, a request that would not fit the 320-byte TX stream. It sets NPARAM only after the parameters are in. And a call with FUJI_FIELD_REPLY copies the reply out of the window; a call without it leaves the reply in the window, where fuji_bus_call_rlen says how long it is – which is how CONFIG reads directory entries without copying them.

FN_COMMIT()

```
uint8_t fn_commit(void)
{
    uint8_t want = (uint8_t) (FN_ACKSEQ + 1);
    uint16_t outer, inner;

    if (want == 0)
        want = 1; /* 0 means "never used" */
    fn_regwr(FNR_SEQ, want);

    for (outer = 0; outer < 4000u; outer++) {
        for (inner = 0; inner < 250u; inner++) {
            if (FN_ACKSEQ == want)
                return FN_ERRCODE;
        }
    }

    return FN_EWAIT;
}
```

THE CAPPED CALLS

1. **network_bus_read()**: asks for at most 1024 bytes: the reply window. `network_read()` loops over it until it has what you asked for.
2. **network_bus_write()**: sends at most 317: the TX stream, less the three bytes of the count parameter. `network_write()` loops.
3. **fuji_bus_appkey_read()**: reads into a 66-byte buffer of its own, because on this bus the reply starts with two bytes of length, and gives you only the key.

THE NES'S OWN CALLS

`fujinet-nes.h` adds what no other platform has:

Call	Does
<code>fuji_nes_present()</code>	True if 'F', 'N' and protocol version 1 are in the status page.
<code>fuji_nes_boot_state()</code>	BOOT_STATE: FUJI_NES_BOOT_IDLE, _XFER, _READY or _FAILED.
<code>fuji_nes_boot_percent()</code>	BOOT_PCT, 0-100.
<code>fuji_nes_boot_error()</code>	BOOT_ERR (Chapter 8).

<code>fuji_nes_boot_got(), _total()</code>	The image's byte counts. They are 24 bits painted a byte at a time, so the library reads them twice until two reads agree.
<code>fuji_nes_boot()</code>	BOOTLOCK, interrupts and PPU and APU off, jump to the loader. Never returns.
<code>fuji_nes_kbd_detect()</code>	Look for a keyboard on the expansion port: FUJI_NES_KBD_NONE, _FAMILY_BASIC or _SUBOR. Call once.
<code>fuji_nes_kbd_getc()</code>	Once a frame: the character of a key that has just gone down, or 0.
<code>fuji_nes_kbd_scan()</code>	The raw key matrix, for a program that wants it.
<code>fn_regwr(), fn_tx(), fn_commit()</code>	The mailbox itself, for a transaction built by hand.

KEYBOARDS

The Famicom's two keyboards – Nintendo's Family BASIC keyboard and the Subor – sit on the expansion port and are scanned through \$4016 and \$4017. `fuji_nes_kbd_detect()` tells them apart by what an idle port reads: a disabled Subor still drives \$1E, a Family BASIC keyboard reads 0. Letters come in lower case, and upper case with Shift; the arrow keys, Enter, Escape, Backspace and Home come as the FUJI_NES_KEY_ codes.

6502 ASSEMBLY (CA65)

```
; there is no assembly keyboard
; driver: link the C library's, or
; scan $4016/$4017 as kbd_nes.c does
```

C (CC65 + FUJINET-LIB)

```
uint8_t kbd = fuji_nes_kbd_detect();

for (;;) {
    char c;
    waitvsync();
    c = kbd ? fuji_nes_kbd_getc() : 0;
    if (c == FUJI_NES_KEY_ENTER)
        break;
}
```

The netcat in Appendix D takes typing from a keyboard if one is there and from an on-screen keyboard if not, the same as CONFIG.

BUILDING A TRANSACTION BY HAND

When a payload is built from pieces – part from RAM, part out of the last reply – there is no single buffer to give `fuji_bus_call()`. CONFIG has five such transactions in `fujiraw.c`. This one rewrites one host slot by streaming the other seven straight out of the reply window:

CONFIG FUJIRAW.C: ONE HOST SLOT

```
bool fnraw_write_host_slot(unsigned char slot, const char *name)
{
    volatile unsigned char *r;
    unsigned char i, j;

    /* Fresh window first: the write streams the other seven slots straight
     * back out of it. The reply is only repainted by a commit, so it holds
     * still while the TX stream is built. */
    if (!FUJICALL(FUJICMD_READ_HOST_SLOTS))
        return false;

    raw_begin(FUJICMD_WRITE_HOST_SLOTS);
    for (i = 0; i < 8; i++) {
        if (i == slot) {
            raw_tx_padded(name, 32);
        } else {
            r = FN_REPLY + (unsigned int)i * 32;
            for (j = 0; j < 32; j++)
                fn_tx(r[j]);
        }
    }
    return raw_finish();
}
```

It works because the reply window holds still until the next commit. Nothing else may run between the READ and the commit that ends the WRITE – not even another of the program's own transactions.

BUILDING WITH THE LIBRARY'S MAKEFILES

A project that builds with fujinet-lib's own makefiles, as the games do, gets the NES configuration, the map file and the claim for free from `makefiles/platforms/nas.mk`:

MAKEFILES/PLATFORMS/NES.MK, THE PART THAT MATTERS

```
NES_CFG ?= $(MWD)/nes-fujinet.cfg
NES_MAP ?= $(EXECUTABLE:.nes=.map)
LD_FLAGS += -C $(NES_CFG) -m $(NES_MAP)

$(PLATFORM)/executable-post::
$(MWD)/nes-romstamp.py --stamp --map $(NES_MAP) $(EXECUTABLE)
```

A game that wants a different layout – its own CHR bank, a sprite page – copies `nes-fujinet.cfg`, keeps the CLAIM memory area, and sets `NES_CFG`.

11. PICTURES, PADS AND SOUND

A network program spends most of its time waiting. On the NES it has to wait without disturbing a picture processor that will only take new data for about two thousand cycles in every frame.

THE PPU'S RULE

The PPU draws the screen out of its own memory – the pattern tables in CHR, the nametables, the palettes – and the CPU reaches that memory only through two registers, \$2006 (address) and \$2007 (data). While the PPU is drawing, those registers are its own: a write lands in the wrong place and knocks the scroll askew. A program may write to the PPU only

- * with rendering off (\$2001 = 0) – the screen goes black meanwhile; or
- * during vblank, the 20 scan lines between frames, announced by the NMI.

The programs in Chapters 5 to 9 take the first road for everything they draw at the start, and turn the screen on last. Those that draw while running – `boot.s` and `clock.s` – wait for the NMI with `wait_vbl`, write a few dozen bytes, and put the scroll back with `scroll0`, because every \$2006 write disturbs it.

CC65'S QUEUE

cc65's NES runtime takes the second road for you. `cputc()` and every other conio routine does not write to the PPU at all: it puts an (address, value) pair in a queue in \$0200-\$04FF, and cc65's NMI handler writes up to 70 of them in each vblank, then resets the scroll. A C program can therefore draw whenever it likes.

Three consequences:

- 1. NMI first:** The queue is drained only by the NMI. Draw before NMI is on and a full queue waits forever. cc65's start-up turns NMI on; a game that sets up the PPU itself must turn it on before its first `cputc`.
- 2. About 68 cells a frame:** A whole 32×24 screen is over 700 writes: eleven frames. The games keep a copy of the screen in RAM and queue only the cells that change; for a whole-screen redraw they turn rendering off and write straight to \$2007.
- 3. No scrolling:** cc65's NMI sets the scroll back to 0,0 every frame. The netcat in Appendix D scrolls its text by redrawing it from a copy in RAM.

CAUTION: cc65's own `waitvsync()` polls the `vblank` flag in `$2002`. A read of `$2002` in the very cycle the flag rises suppresses that frame's NMI – and with it that frame's queue flush and frame count. Wait instead for `clock()`, which cc65's NMI handler increments every frame, to change. Battleship, Fujitzee and every C program in this manual do; 5 Card Stud and Texas Hold'em still call cc65's.

NOTE: The NMI may land in the middle of a mailbox transaction; cc65's handler writes only the PPU and RAM, so that is harmless. Your own handler must not write `$5500–$57FF`.

PATTERN TABLES AND THE CLAIM ON CHR

cc65's NES runtime always links its 8K console font into CHR, whether a program uses `conio` or not. The games keep it at `$0000` and put their own tiles in the other pattern table at `$1000`, pointing the background there with `$2000`. The NES has no colour per character, only a palette per 16×16-pixel block, so the games bake each glyph-and-colour pair into a tile of its own with a Python script, `mkchr.py`, that writes the tile set as ca65 source. The budget is 256 tiles: 5 Card Stud uses 233, Fujitzee 238, Battleship all 256.

THE CARTRIDGE'S WORK RAM

The console has 2K of RAM, and cc65 takes most of it: zero page, the stack, the PPU queue at `$0200–$04FF`, and its own parameter stack. The FujiNet cartridge serves 8K more at `$6000–$7FFF`, and `nes-fujinet.cfg` puts all of a C program's data and BSS there. That is where a game keeps its state, its screen copy and its network buffers.

CAUTION: That 8K is lost when the power goes off. Keep anything that must survive in an app key.

THE CONTROLLER

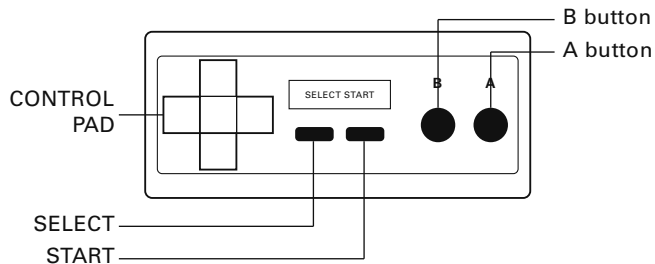


Fig. 13 – The NES controller. The FujiNet programs follow the same conventions.

- Control Pad** : Moves the cursor or the highlight.
- A button** : Chooses: the key under the cursor, the table, the move.
- B button** : Goes back, or deletes.
- START button** : Done; in the games, the menu.
- SELECT button** : Shift on the on-screen keyboard; in netcat, hang up.

Reading the controller is a write of 1 and then 0 to \$4016, then eight reads, one bit each, in the order A, B, Select, Start, Up, Down, Left, Right:

LISTINGS/NETCAT/PAD.C

```
static uint8_t read_pad(void)
{
    uint8_t i, b = 0;

    JOY1 = 1;
    JOY1 = 0;
    for (i = 0; i < 8; i++)
        b = (b << 1) | (JOY1 & 1);
    return b;
}
```

/* latch the buttons ... */
/* ... and shift them out */
/* A, B, Select, Start, Up ... Right */

\$4016 is not a mailbox page, so nothing here worries the one rule. A program that also reads a Family BASIC keyboard shares the port with it; the keyboard routines put it back the way they found it.

TYPING WITHOUT A KEYBOARD

Every FujiNet program on the NES that needs text – a player's name, a password, a URL – puts a keyboard on the screen. The games use a ten-wide grid of upper-case letters and digits; CONFIG and the netcat add lower case, the punctuation a URL needs, and SHIFT, SPACE, DEL and DONE. The pad moves a highlight, A presses the key under it, B deletes, Start is DONE.



Fig. 14 – The netcat's on-screen keyboard, with the URL it will connect to above it.

SOUND

The games make their sounds on the 2A03's two pulse channels, playing in unison for volume, with the triangle and noise channels for explosions. A pulse channel's 11-bit timer is the CPU clock over sixteen times the frequency, less one – $111861 / \text{Hz} - 1$ – which bottoms out about 55 Hz.

CAUTION: Leave the pulse channels' sweep units with the negate bit **set** ($\$4001 = \08), even when you are not sweeping. With it clear, the sweep unit's overflow check silences low notes.

12. GAME ONE: 5 CARD STUD

Five-card stud poker against bots, or against anybody else on the Internet who sits down at the same table. The NES version is the same game that runs on the Atari, the Apple II, the CoCo and a dozen more, with a thousand lines of NES underneath.

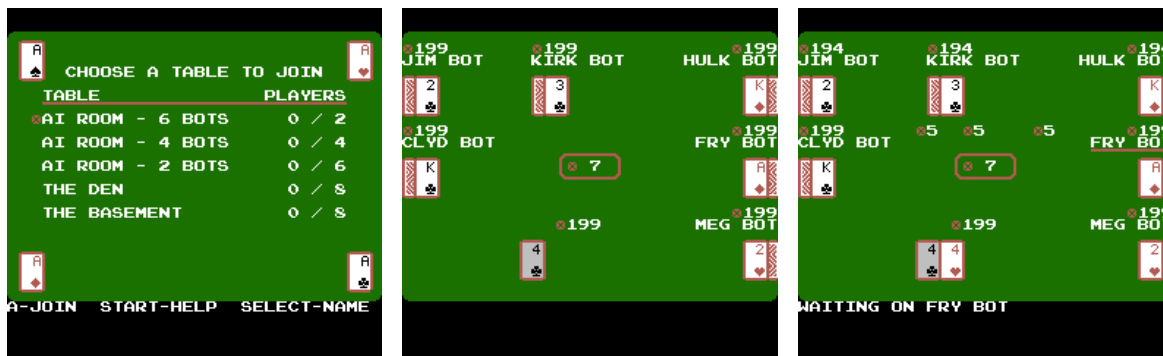


Fig. 15 – Choosing a table; the deal; a few seconds later, waiting on Fry Bot. Taken against the live server.

OBJECT OF THE GAME / GAME DESCRIPTION

Every client polls one web server for the state of its table, draws it, and sends the player's move. The server deals the cards, runs the bots, keeps the time and decides who won. All the client has to do is ask, draw and answer – which on the NES means ask through the mailbox, draw through the PPU queue, and answer with the pad.

THE SHAPE OF THE CLIENT

The game lives in `fujinet-5cardstud` (cc65 C). The shared code in `src/` – `main.c`, `screens.c`, `gamelogic.c`, `stateclient.c` – is the same on every computer. Each platform supplies a directory of its own that implements a fixed set of functions; for the NES that is `src/nas/`:

File	Lines	What it provides
graphics.c	575	initGraphics, drawCard, drawText, drawChip, resetScreen and the rest: the table, in tiles
ppu.s	23	ppu_put: one queued PPU write (Listing 10)
network.c	36	getResponse: one HTTP GET (Listing 9)
appkey.c	35	read_appkey, write_appkey
input.c	79	readJoystick, getPlatformKey: the pad (Listing 11)
osk.c	227	platformNameEntry: the on-screen keyboard (Listing 14)
sound.c	143	the beeps (Listing 13)
util.c	141	the timer, and quit to the Lobby (Listing 12)
mkchr.py	152	generates chr.s and tiles.h, the tile set
nes.cfg	66	the linker configuration

main() loads the player's preferences from an app key, sets up the screen and the sound, shows the welcome and table screens, and then goes round one loop for the rest of the game: get the state from the server, draw it, and if it is the player's turn, ask for a move.

MEMORY

The build is plain NROM – 32K of PRG, 8K of CHR ROM – linked with a copy of fujinet-lib's nes-fujinet.cfg that splits the CHR in two:

SRC/NES/NES.CFG

```
# 8k CHR Bank: cc65's font, then the game's own pattern table
ROM2:  file = %0, start = $0000, size = $1000, fill = yes;
ROM3:  file = %0, start = $1000, size = $1000, fill = yes;

# standard 2k SRAM (-zeropage)
# $0100-$0200 cpu stack
# $0200-$0500 3 pages for ppu memory write buffer
# $0500-$0800 3 pages for cc65 parameter stack
SRAM:  file = "", start = $0500, size = __STACKSIZE__, define = yes;

# additional 8K SRAM Bank: the cartridge serves this
RAM:   file = "", start = $6000, size = $2000, define = yes;
}
```

cc65's console font has to be linked whether the game uses it or not, so it gets \$0000; the game's 233 tiles get \$1000, and `initGraphics()` points the background there. All the game's data – its state, a 768-byte copy of the screen, its buffers – lives in the cartridge's 8K at \$6000.

ONE REQUEST, ONE GET

The NES half of the network code is one function:

SRC/NES/NETWORK.C

```
#ifdef BUILD_NES

/**
 * @brief   Network functions
 * @author   Thomas Cherryhomes
 * @email    thom dot cherryhomes at gmail dot com
 * @license  gpl v. 3, see LICENSE for details
 */

#include <stdint.h>
#include <fujinet-network.h>

static uint8_t initialized = 0;

uint8_t getResponse(char *url, unsigned char *buffer, uint16_t max_len)
{
    int16_t count;

    if (!initialized) {
        if (network_init() != FN_ERR_OK)
            return 0;
        initialized = 1;
    }

    // Don't read from a channel that never opened - network_read() would
    // otherwise poll status on a dead unit and hand back whatever it got.
    if (network_open(url, OPEN_MODE_HTTP_GET_H, OPEN_TRANS_NONE) != FN_ERR_OK)
        return 0;

    count = network_read(url, buffer, max_len);
    network_close(url);

    return count > 0;
}

#endif /* BUILD_NES */
```

Every poll is an OPEN, a READ and a CLOSE. Mode 12, OPEN_MODE_HTTP_GET_H, is a GET; network_read() loops until it has the whole reply or the server is done. The reply lands in clientState in the cartridge's work RAM.

The URL is put together by the shared apiCall() from the server's address, the path – tables, state, move/CA, leave – the table and the player's name, and bin=1:

A POLL, AND A CALL

```
N:https://5card.carr-designs.com/state?table=ai2&player=THOM&bin=1
N:https://5card.carr-designs.com/move/CA?table=ai2&player=THOM&bin=1
```

WHAT COMES BACK

bin=1 asks for the state as packed binary, laid out exactly as the C structures the client reads it into – fixed-length strings padded with NULs, numbers low byte first, which suits a 6502:

Offset	Bytes	Field
0	81	lastResult: what just happened
81	1	round: 0 no game, 1-4 betting, 5 showdown
82	2	pot
84	1	activePlayer: 0 is you, -1 nobody
85	1	moveTime: seconds left, on your turn
86	1	viewing: 1 if you are only watching
87	1	validMoveCount
88	65	validMoves[5]: a move code (3) and its name (10)
153	1	playerCount
154	33 each	players: name (9), status, bet (2), move (8), purse (2), hand (11)

Only the seated players are sent, so a reply is 154 + 33 bytes per player. The server turns the table around so that you are always player 0. A hand is two characters a card – rank then suit, kd for the king of diamonds, ?? for a card face down. The strings arrive in lower case; the NES font has only capitals, so textGlyph() folds them.

DRAWING WITHOUT A FRAME BUFFER

The shared code was written for computers that can read their screens back. The NES cannot read the nametable while it is drawing, so the NES layer keeps its own copy – shadow, one glyph per cell – and draws through it:

SRC/NES/GRAPHICS.C: PUT

```
static void put(unsigned char x, unsigned char y, unsigned char glyph, unsigned char tile)
{
    if (x >= WIDTH || y >= HEIGHT)
        return;
    shadow[y][x] = glyph;
    ppu_put(NT_BASE + ((unsigned int)(y + ROW0) << 5) + x, tile);
}

static unsigned char peek(unsigned char x, unsigned char y)
{
    if (x >= WIDTH || y >= HEIGHT)
        return ' ';
    return shadow[y][x];
}
```

The glyph goes in the shadow, where the card-drawing code can ask what is already there; the tile, which carries its colour, goes to the PPU queue. `ppu_put()` (Listing 10) is nine instructions of assembly that hand the address and value to cc65's `ppbuf_put` in the registers it wants.

Colour is the harder part. Each 16×16-pixel block of the screen has one palette, chosen by two bits of an attribute byte, so greying the player's own hole card means rewriting attribute bits:

SRC/NES/GRAPHICS.C: SETBLOCKPALETTE

```
static void setBlockPalette(unsigned char x, unsigned char y, unsigned char pal)
{
    unsigned char row = y + ROW0;
    unsigned char i = ((row >> 2) << 3) | (x >> 2);
    unsigned char shift = ((row & 2) << 1) | (x & 2);
    unsigned char v = (attr[i] & ~(3 << shift)) | (pal << shift);

    if (v != attr[i])
    {
        attr[i] = v;
        ppu_put(AT_BASE + i, v);
    }
}
```

Only an even column lines up with the attribute grid, so seat 0 sits on one.

YOUR TURN

The pad goes through cc65's joystick driver. Every **button** is reported as a key – A is Return, B is Escape – so that all of them pass through one edge detector (Listing 11 [5](#)). Start and Select report on **release**, so the code can tell either one alone from both together: Select plus Start leaves the table for the Lobby.

When it is your turn, the status line shows the moves the server allows; the pad moves an underline between them and A sends move/ and the move's code.

THE SERVER

The server is Go, in `servers/fujinet-game-system/5cardstud/server`. It keeps every table in memory, each behind its own lock. A request copies the table, works on the copy, and saves it only if nothing went wrong.

The game moves forward only when somebody asks: each `/state` request runs the game logic – deal, time out a slow player, let a bot move – before it answers. A human has 39 seconds to move; a bot takes 3. Tables `ai2`, `ai4` and `ai6` seat two, four and six bots; `den` and `basement` start empty.

LEAVING FOR THE LOBBY

`quit()` (Listing 12) is how every FujiNet game on the NES goes back to the FujiNet Lobby: find the host slot named `ec.tnfs.io`, mount `nes/lobby.nes` from it, and boot it (Listing 12 [6](#)).

NOTE: The comment in `quit()` says the image is pushed after `MOUNT_IMAGE` has been answered. It is not: `MOUNT_IMAGE` answers only once the image has arrived (Chapter 8), so the percentage loop that follows sees `READY` at once. And at the time of writing there is no NES Lobby image on `ec.tnfs.io` yet.

13. GAME TWO: BATTLESHIP

Fuji Battleship: up to four players, each with five ships and a ten-by-ten sea, every shot fired at every opponent at once.

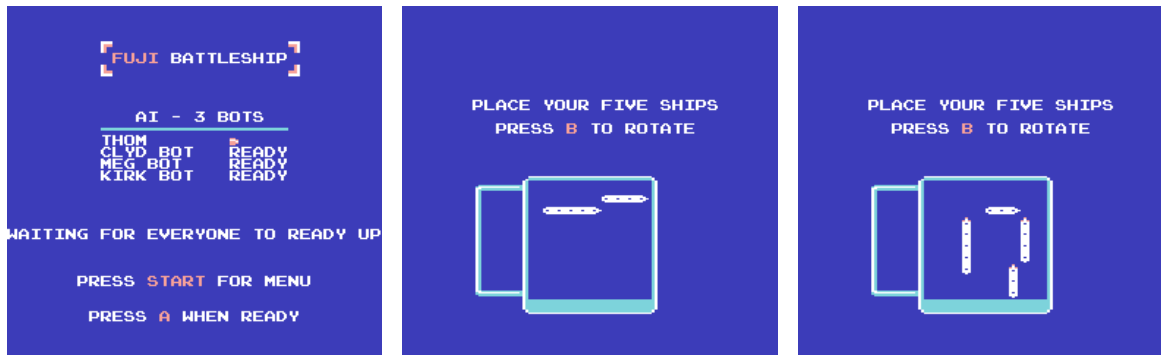


Fig. 16 – Ready to start against three bots; placing the fleet; four of five placed, the B button turning the fifth.

OBJECT OF THE GAME / GAME DESCRIPTION

Place five ships, then take turns choosing one square. Every opponent's sea is hit at that square at once. The last fleet afloat wins. As with 5 Card Stud, the server does the work and the client asks, draws and answers.

WHAT IS DIFFERENT HERE

Battleship's NES layer has the same shape as 5 Card Stud's – a `src/nas/` directory behind the shared code, the same `ppu.s`, the same split `CHR` – with three things worth a closer look: how it draws a whole screen, how it waits for a frame, and `fujinet.c`, the one file through which it talks to the FujiNet.

NOTE: At the time of writing the NES port of Battleship is not yet committed to its repository; the listings here are taken from the working tree.

ONE FILE FOR THE FUJINET

On the NES, most of fujinet-lib's `fuji_` calls are macros over the bus call. The shared code's own prototypes for them would link against nothing, so everything that talks to the cartridge is in one file that includes only the library's headers, and the shared code reaches it through three hooks:

SRC/NES/FUJINET.C: THE HOOKS

```
int16_t custom_network_call(char *url, uint8_t *buffer, uint16_t max_len)
{
    int16_t read;

    if (network_open(url, OPEN_MODE_HTTP_GET, OPEN_TRANS_NONE) != FN_ERR_OK)
        return -1;

    read = network_read(url, buffer, max_len);
    network_close(url);
    return read;
}

uint16_t custom_read_appkey(uint16_t creator_id, uint8_t app_id, uint8_t key_id, char *destination)
{
    uint16_t read = 0;

    fuji_set_appkey_details(creator_id, app_id, DEFAULT);
    if (!fuji_read_appkey(key_id, &read, (uint8_t *)destination))
        read = 0;
    return read;
}

void custom_write_appkey(uint16_t creator_id, uint8_t app_id, uint8_t key_id, uint16_t count, char *data)
{
    fuji_set_appkey_details(creator_id, app_id, DEFAULT);
    fuji_write_appkey(key_id, count, (uint8_t *)data);
}
```

The rest of `fujinet.c` (Listing 15) is the trip back to the Lobby, the same as 5 Card Stud's.

WIRE FORMAT

Requests are state, ready, place/ with the five ships' squares, attack/ with one square, and leave, with bin=1&v=2. The reply has two shapes, chosen by its status byte: in the lobby, the server's name and each player's name and status; in play, the last shot, the player's own ships, and for each player a 100-byte sea – the squares hit and missed – and how many of each ship is still afloat.

A WHOLE SCREEN AT ONCE

A full 32×24 screen through cc65's queue takes eleven frames – a third of a second, plainly visible as the picture is painted in. So for the two operations that change everything, Battleship stops the picture and writes the nametable directly from its shadow copy:

SRC/NES/GRAPHICS.C: BLITSHADOW

```
static void blitShadow(void)
{
    unsigned char x, y;

    ppu_drain();
    waitvsync();
    PPU_CTRL = CTRL_OFF;
    PPU_MASK = 0;

    PPU_ADDR = (unsigned char)((NT_BASE + (ROW0 << 5)) >> 8);
    PPU_ADDR = (unsigned char)(NT_BASE + (ROW0 << 5));
    for (y = 0; y < HEIGHT; y++)
        for (x = 0; x < WIDTH; x++)
            PPU_DATA = shadow[y][x];

    // Back on at the top of a frame, so the picture never starts mid-screen.
    while (!(PPU_STATUS & 0x80))
        ;
    PPU_ADDR = 0;
    PPU_ADDR = 0;
    PPU_SCROLL = 0;
    PPU_SCROLL = 0;
    PPU_CTRL = CTRL_ON;
    PPU_MASK = MASK_ON;
}
```

It drains the queue first, or the queue would land stale cells on top afterwards. It turns the NMI off for the duration, because the NMI handler's flush and scroll reset write \$2006 too. And it turns the picture back on at the top of a frame, so the screen never starts halfway down.

Everything else goes through `put()`, which queues a cell only if it has changed:

SRC/NES/GRAPHICS.C: PUT

```
static void put(unsigned char x, unsigned char y, unsigned char t)
{
    if (x >= WIDTH || y >= HEIGHT || shadow[y][x] == t)
        return;

    shadow[y][x] = t;
    ppu_put(NT_BASE + ((unsigned int)(y + ROW0) << 5) + x, t);
}
```

NOTE: Battleship's art is the MS-DOS client's CGA tile sheet, converted tile for tile: CGA's four colours are exactly one NES palette, so Battleship never writes an attribute byte after start-up. It uses all 256 tiles.

WAITING FOR A FRAME

SRC/NES/UTIL.C: WAITVSYNC

```
cc65's own waitvsync() polls PPUSTATUS for the vblank flag. With NMI on that
races the NMI: a read in the same cycle the flag rises suppresses the NMI,
and with it that frame's ring-buffer flush. Waiting for the NMI's frame
count to move instead is race-free, and needs NMI on, which it always is
outside graphics.c's blitShadow().
*/
void waitvsync(void)
{
    uint8_t t = (uint8_t)clock();
    while ((uint8_t)clock() == t)
        ;
}
```

BUTTONS

Button	Does
A	Choose; fire
B	Rotate the ship being placed; refresh
START	The menu

SELECT	Change your name
SELECT + A / B / START	Help / sound / quit to the Lobby

As in 5 Card Stud every button is a key to the shared code, and Select is a shift that reports on release only if nothing else was pressed with it (Listing 17).

THE SERVER

The server is Go, in `servers/fujinet-game-system/battleship`. There is no "join": a player who asks for a table's state is seated at it. Tables `ai1` to `ai3` have one to three bots, which take three seconds a move. The bots do not cheat: each builds a map of where an enemy ship could still be from what it has hit and missed – a single hit, a gap between two hits, a line of hits – and prefers squares that could hit more than one player's ship at once. When a human sits down at a full table, a bot gets up for them.

14. GAME THREE: FUJITZEE

Five dice, three rolls, thirteen boxes to fill. Fujitzee is the one game here that uses the NES's sprites – and the one whose whole state arrives in a single 599-byte structure.

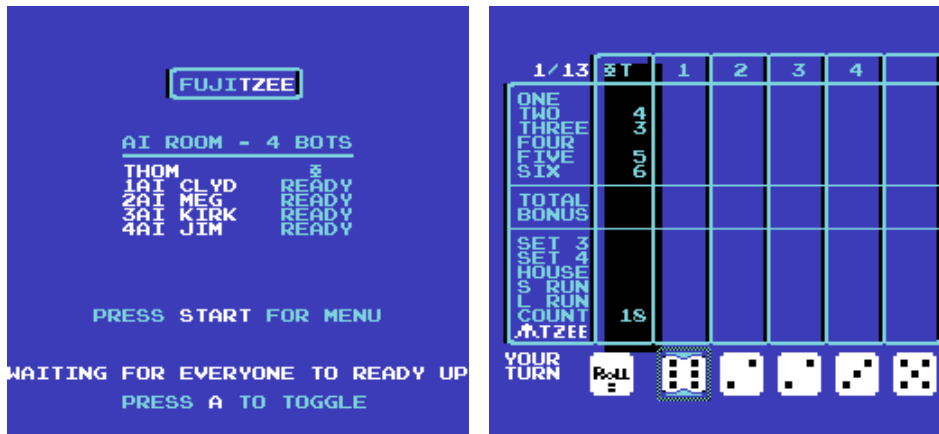


Fig. 17 – Waiting for the others to get ready; the scorecard, the dice and the cursor – the dotted frame round the second die is twelve sprites.

OBJECT OF THE GAME / GAME DESCRIPTION

Each turn, roll up to three times, keeping any dice you like between rolls, then score the result in one of the thirteen boxes. Up to six players sit at a table; more can watch.

REQUESTS AND REPLIES

The requests are state, ready, roll/ followed by five 0s and 1s – which dice to keep – score/ and a box number, and leave. With bin=1 the reply is the table's name and prompt, the round, the rolls left, whose

turn it is, the dice, the dice kept, which boxes are open, and then for every player a name and sixteen scores of two bytes each.

The client declares the same layout as a C structure and the server's bytes go straight into it. If cc65 ever padded that structure, every field after the padding would be wrong – so the NES layer refuses to compile if it is not exactly the size the server sends:

SRC/NES/UTIL.C

```
/* The server sends the packed wire layout straight into clientState.game,
 * so cc65 must not pad Game. Fails the build if it ever does. */
typedef char game_wire_size_check[(sizeof(Game) == 599) ? 1 : -1];
```

599 is 95 bytes of table and 42 for each of twelve players. A reply for three players is 221 bytes.

THE ACTIVE COLUMN

The scorecard has a column per player, and the player whose turn it is gets a black column instead of a blue one. Each column is exactly one attribute byte wide, and background palette 1 is palette 0 with blue made black – so the whole highlight is five attribute bytes:

SRC/NES/GRAPHICS.C: COLUMNATTR

```
static void columnAttr(int8_t player, unsigned char value)
{
    unsigned char r, i;

    if (player < 0)
        return;

    for (r = 1; r <= 5; r++)
    {
        i = (r << 3) + 2 + player;
        if (attr[i] != value)
        {
            attr[i] = value;
            if (!batch)
                ppu_put(AT_BASE + i, value);
        }
    }
}
```

SPRITES FOR THE CURSOR

The cursor round a die is a checkerboard frame a pixel outside it – twelve sprites, so moving it never touches the nametable. Their 256-byte page has to be page-aligned, so the linker configuration shrinks the work RAM by a page and puts the sprites at \$7F00. A sprite DMA, like any PPU access, has to happen in vblank:

SRC/NES/GRAPHICS.C: OAMFLUSH

```
static void oamFlush(void)
{
    ppu_drain();
    waitvsync();
    OAM_ADDR = 0;
    OAM_DMA = (unsigned char)((unsigned int)oam >> 8);
}
```

The queue is drained first, so the NMI that waitvsync() waits for has nothing to flush and leaves most of the vblank for the DMA. And because the PPU forgets its sprites while rendering is off, the whole-screen redraw does the DMA again before it turns the picture back on.

THE KEYBOARD AS A KEY SOURCE

Battleship's on-screen keyboard runs its own editing loop. Fujitzee's feeds the shared code's ordinary text-entry routine instead, one key at a time, through kbhit() and cgetc() (Listings 18 and 19). The shared code does the editing; the keyboard only has to say which key was pressed:

SRC/NES/INPUT.C

```
unsigned char kbhit(void)
{
    if (!pendingKey)
    {
        if (oskActive)
        {
            pendingKey = oskPoll();
            // Whatever the grid used up is not a press once it closes.
            lastButtons = joy_read(JOY_1) & ~DIRS;
            shifted = 1;
        }
        else
            pendingKey = poll();
    }
}
```

ONE PLAYER

The other clients let two or more people play from one computer. The NES has one controller here, so the NES build defines `SINGLE_LOCAL_PLAYER` and the screen that groups local players is compiled out.

THE SERVER

The server is Go, in `servers/fujinet-game-system/fujitzee`. Its bots – numbered, so "1AI CLYD" – chase a large straight when one is close, otherwise keep their biggest set, and fill the upper boxes first when that will earn the bonus. Tables `ai2` and `ai4` have bots; `bar` and `kit` start empty.

15. GAME FOUR: TEXAS HOLD'EM

Hold'em is 5 Card Stud's younger brother: the same client, the same NES layer almost line for line, and a server that knows how to deal a flop.

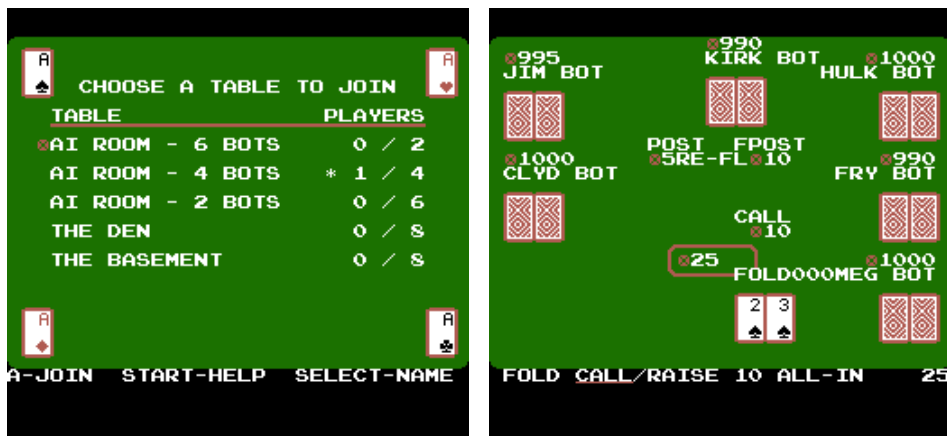


Fig. 18 –The table list, and a hand in progress against five bots: two hole cards each, the pot in the middle, the moves on the status line.

OBJECT OF THE GAME / GAME DESCRIPTION

Two cards each, face down; five shared cards dealt three, one and one; four rounds of betting with blinds of 5 and 10 and a purse of 1000. The best five cards of the seven win.

A CHAPTER OF DIFFERENCES

fujinet-texasHoldEm's src/nas/ is 5 Card Stud's with four changes. The tiles that mkchr.py generates are byte for byte the same. The differences:

1. **The seats:** move to keep the middle of the table clear for the community cards. Hold'em never greys a hidden hole card, so seat 0 no longer has to sit on an even column.
2. **The pot:** moves down three rows, below the board.
3. **A face-down card:** already on the felt is left alone.
4. **The title:** says Texas Hold'em.

SRC/NES/VARS.C: THE SEATS

```
const unsigned char playerXMaster[] = { 17, 1, 1, 1, 15, 29, 29, 29 };
const unsigned char playerYMaster[] = { 17, 17, 10, 3, 2, 3, 10, 17 };
const char playerDirMaster[] = { 1, 1, 1, 1, 1, -1, -1, -1 };
const char playerBetXMaster[] = { 1, 10, 10, 10, 3, -8, -8, -8 };
const char playerBetYMaster[] = { -2, -1, 2, 4, 5, 4, 2, -1 };
```

NOT REDRAWING WHAT HAS NOT CHANGED

The third difference, and a change to the shared `main.c`, come from the same lesson. Every tile goes through `cc65`'s queue at about 68 a frame. A poll that redraws the whole table – every opponent's two card backs, every purse – keeps the queue full, and the next real change waits behind it. So the card-drawing code ships nothing when a card back is already there:

SRC/NES/GRAPHICS.C

```
if (peek((x>WIDTH-3)?x-1:x, y+1) == UDG_BACK_L_TOP)
    return;
```

and the main loop skips the redraw entirely when the reply is the same as the last one, by keeping a checksum of it:

SRC/MAIN.C

```
static unsigned int lastDrawnSum = 1; // != any first real sum
unsigned int sum = 0;
unsigned int n; // Game is 429 bytes: not a char
unsigned char *sp = &clientState.firstByte;
for (n = 0; n < sizeof(Game); n++)
    sum += sp[n] ^ (unsigned char)n; // position-weighted vs byte swaps
if (sum != lastDrawnSum) {
    lastDrawnSum = sum;
    showGameScreen();
}
}
```

Each byte is mixed with its position, so two bytes trading places still changes the sum. A rare collision only means one poll goes undrawn.

THE WIRE

Hold'em's reply is 5 Card Stud's with one field added after viewing: the community cards, eleven bytes, askh2d for the ace of spades, king of hearts and two of diamonds. Everything after it moves down eleven, so a reply is 165 bytes and 33 for each player; the whole structure is 429. The moves gain RL and RH, the smallest and a bigger raise, and AI, all in. A player who is all in is reported to the 8-bit clients as merely playing.

The server – `servers/fujinet-game-system/texasholdem` – also offers a `WebSocket`, and a hash parameter that answers just 1 when nothing has changed. The NES client uses neither; its checksum does the same job without a second request.

THE SERVER'S BOTS

Each bot has a personality: how often it plays a hand, how often it raises before the flop, how often it bluffs. And the bots only play while a human is seated – an empty table waits.

NOTE: The table in the second picture shows a few cells of an earlier status line left over beside the seats – "FPOST", "OOOMEG". It is as the emulator drew it, against the live server, and is not a fault of the camera.

16. CONFIG AND THE LOBBY

CONFIG is the program the cartridge runs when you turn the console on: WiFi, host slots, a file browser and a loader, in 32K, dressed as the Famicom's Family BASIC.

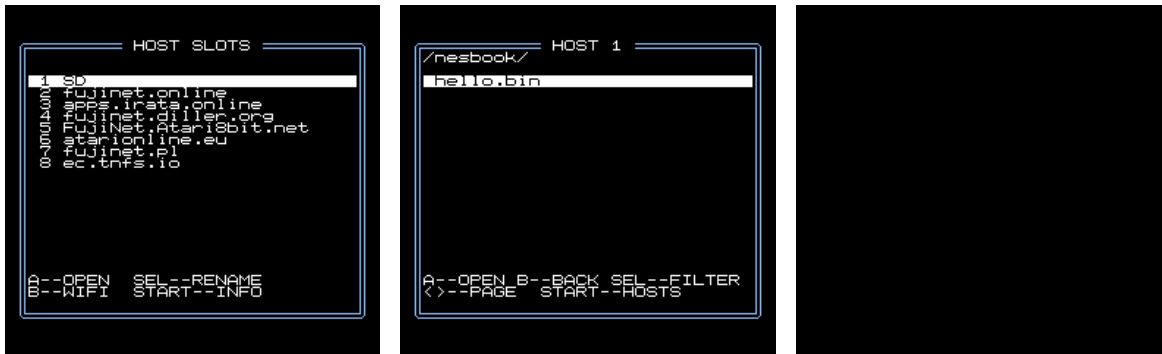


Fig. 19 –The host slots; a directory on the SD card; and loading `hello.bin` from it, with the bar filled by `BOOT_PCT` and the count by `BOOT_GOT` and `BOOT_TOT`.

WHERE IT LIVES

CONFIG is in `fujinet-config`, in its `nes/` directory, written in cc65 C against `fujinet-lib`. It is built like any other program in this manual – the library's linker configuration, the claim, `checkrom.py` – and then **baked**: the cartridge firmware's `build-cart.sh` turns `build/config.nes` into a C array inside the RP2354B's firmware. At power-on the loader copies it from there into the SRAMs, which takes about a sixth of a second.

BUILDING CONFIG AND BAKING IT INTO THE CARTRIDGE

```
cd fujinet-config/nes
make                               # build/config.nes
make PICO_NES=~/.fujinet-firmware/pico/nes cart
                                   # copy it in, build fujines.uf2
```

THE SCREENS

CONFIG is a state machine with one source file per state:

State	File	What it does
CHECK_WIFI	st_wifi.c	WiFi on? Connected (status 3)? A network saved? Then CONNECT, or SET.
CONNECT_WIFI	st_wifi.c	Asks for the WiFi status every two seconds, twenty times; B gives up.
SET_WIFI	st_wifi.c	SCAN_NETWORKS, then one GET_SCAN_RESULT per row; pick one, type its password.
HOSTS	st_hosts.c	The eight host slots, drawn straight from the reply window.
FILES	st_files.c	Sixteen entries a page; open a directory, or boot a file.
INFO	st_info.c	GET_ADAPTERCONFIG_EXTENDED.



Fig. 20 – Choosing a network, and the adapter. fujinet-pc pretends to see fifteen networks, all called "Dummy Cafe".

Screen	Pad	A	B	SELECT	START
HOSTS	move	open host	WiFi	rename	info

FILES	move; left/right page	open / boot	up a level	filter	hosts
WIFI	move	pick	hosts		rescan
keyboard	grid	pick	delete	case	OK

PAGING A DIRECTORY

A page is one SET_DIRECTORY_POSITION – to the page's first entry – and then up to sixteen READ_DIR_ENTRY calls of 27 bytes, each drawn from the reply window. When you choose a file, CONFIG seeks back to it and reads it again at 120 bytes, to have the whole name. Then it builds SET_DEVICE_FULLPATH from the directory's path in RAM and the file's name **out of the reply window**, without copying it (Chapter 10).

NOTE: CONFIG treats .. and . as the end of a directory, as well as \$7F. An SD card's root on fujinet-pc answers .. for ever, and an entry that was not there spoils the next seek.

BOOTING, WITH A BAR

CONFIG ST_BOOT.C: BOOT_MOUNT_SWAP

```
status_line("LOADING...");
want = fnraw_mount_start(DEVICE_SLOT, MODE_READ);
for (;;) {
    unsigned char now;

    st = fuji_nes_boot_state();
    if (st == FUJI_NES_BOOT_FAILED) {
        load_error(fuji_nes_boot_error());
        return;
    }
    if (!acked && FN_ACKSEQ == want) {
        acked = true;
        if (!fnraw_reply_ok()) {
            fail("MOUNT");
            return;
        }
    }
    if (acked && st == FUJI_NES_BOOT_READY)
        break;
    if (++frames > MOUNT_FRAMES) {
        load_error(st);
        return;
    }
    now = fuji_nes_boot_percent();
    if (now > 100)
        now = 100;
    if (now != pct) {
        pct = now;
        boot_pct(pct);
    }
    boot_bytes(fuji_nes_boot_got(), fuji_nes_boot_total());
    wait_frames(1);
}
```

It starts the mount without waiting (fnraw_mount_start() returns the sequence number it sent), then once a frame looks at three things: whether the boot has FAILED, whether the reply has come and was an ACK, and whether the image is READY. It needs both of the last two. It gives up after 3900 frames – 65 seconds – so that the cartridge's own 60-second timeout is the one you see. Errors appear the way Family BASIC prints them: ?MOUNT ERROR 02, ?LOAD ERROR 03.

TYPING

CONFIG's text editor takes characters from its on-screen keyboard and, if `fujii_nes_kbd_detect()` found one, from a Family BASIC or Subor keyboard at the same time. On the keyboard the arrows move, Return is A, Escape and Backspace are B, and in a text field every printable key types.

THE LOBBY

On other FujiNet computers there is one more program, the **Lobby**, which lists the game servers that are running and who is playing, and boots the right client for the game you pick. The NES clients are ready for it: each one's `quit()` mounts `nes/lobby.nes` from `ec.tnfs.io` and boots it, and each one keeps the player's name and the server it was sent to in the Lobby's app keys – creator 1, app 1, key 0 for the name, and one key per game for the server.

Key	Game	Holds
0	all	the player's name
1	5 Card Stud	server address and query
3	Fujitzee	server address and query
5	Battleship	server address and query
8	Texas Hold'em	server address and query

What is missing, at the time of writing, is the other end: there is no NES Lobby yet, `ec.tnfs.io` has no `nes/` directory, and the game servers do not yet list an NES client to the Lobby. CONFIG does not offer the Lobby either. When they arrive, the clients will need no change.

APPENDIX A. ERROR CODES

THE MAILBOX

After ACKSEQ matches, ERR at \$5402 is the cartridge's verdict on the trip and REPLY_CMD at \$5403 is the FujiNet's on the request.

ERR	Name	Meaning
0	OK	The FujiNet answered. Look at REPLY_CMD.
1	NOLINK	No USB link to the FujiNet, even after waiting three seconds.
2	TIMEOUT	No answer in 5 seconds (60 for MOUNT_IMAGE and COPY_FILE). Also: RESET, a disabled printer, and every modem command.
3	BADFRAME	A size byte other than 1, 2 or 4 in the TX stream, or a reply that failed its checks.
4	TOOBIG	The request or the reply would not fit the cartridge's buffers.

REPLY_CMD	Name	Meaning
\$06	ACK	Yes. RXLEN bytes of reply are in the window.
\$15	NAK	No. A NAK never carries data.

THE LIBRARIES

Value	Name	Meaning
\$FF	FNEWAIT, FN_WAIT	The cartridge never answered: no claim, no FujiNet cartridge, or SEQ equal to ACKSEQ. Returned by <code>fn_go</code> and <code>fn_commit()</code> .
\$EE	FNENAK	<code>fn_ack</code> : the FujiNet said NAK.
\$FD	FNENOC	The bring-up programs' "no cartridge": the magic bytes are missing.
0	FN_ERR_OK	fujinet-lib: success.
1	FN_ERR_IO_ERROR	fujinet-lib: the transaction failed or was NAKed.
2	FN_ERR_BAD_CMD	fujinet-lib: called with bad arguments.
3-5	OFFLINE, WARNING, NO_DEVICE	fujinet-lib: not used on the NES.

BOOTING

BOOT_STATE	Name	Meaning
0	IDLE	Nothing staged; also left by a MOUNT_IMAGE that NAKed.
1	XFER	The image is arriving.
2	READY	Staged. Arm BOOTLOCK and jump to \$5800.
\$80	FAILED	See BOOT_ERR.

BOOT_ERR	Name	Meaning
1	TOOBIG	Too big for the staging store or for the SRAMs.
2	TRUNCATED	The stream ended early.
3	NOMAP	The cartridge does not implement the image's mapper.
4	STOREBUSY	The staging store was in use.

THE NETWORK DEVICE

The fourth byte of the network STATUS reply.

Code	Meaning	Code	Meaning
1	Success	200	Connection refused
131	Write only	201	Network unreachable
132	Invalid command	202	Socket timeout
135	Read only	203	Network down
136	End of file	204	Connection reset
138	General timeout	205	Connection already in progress
144	General error	206	Address in use
146	Not implemented	207	Not connected
151	File exists	208	Server not running
162	No space on device	209	No connection waiting
165	Invalid devicespec	210	Service not available
166	Invalid point	211	Connection aborted

167	Access denied	212	Invalid username or password
170	File not found	213	Could not parse JSON
		214	Client error
		215	Server error
		255	Could not allocate buffers

An HTTP server's status code becomes one of these:

HTTP	Error	HTTP	Error	HTTP	Error
2xx	1	404, 410	170	423, 451	167
401-403, 407	212	405	146	other 4xx	214
408	138	5xx	215	no connection	207

APPENDIX B. QUICK REFERENCE

A TRANSACTION

- a) \$5505 ← anything: rewind the TX stream.
 - b) \$5500 ← device, \$5501 ← command, \$5502 ← number of parameters.
 - c) \$5700 ← for each parameter, its size (1, 2 or 4) and its bytes, low first; then the payload. At most 320 bytes.
 - d) \$5510 ← ACKSEQ + 1, skipping 0.
 - e) Wait until \$5400 reads the same.
 - f) \$5402 = 0 and \$5403 = \$06? The reply is \$5404–\$5405 bytes at \$5000.
- STA, STX and STY only. Never INC, DEC or a shift on \$5500–\$57FF. Claim the mailbox with FUJI at \$FFF0.

DEVICES

Device	What	Device	What
\$70	The Fuji device (Chapter 7)	\$45	The clock (Chapter 9)
\$71–\$78	Network N1:-N8: (Chapter 6)	\$40	The printer
\$31–\$38	Disks: the device slots	\$50	The modem: no reply reaches the NES

NETWORK COMMANDS

Cmd	Name	Params	Cmd	Name	Params
\$4F	OPEN	mode, trans + URL	\$54	TRANSLATION	x, trans
\$43	CLOSE		\$4C	SET_EOL	1. bytes
\$53	STATUS	→ 4 bytes	\$4D	HTTP MODE	x, mode
\$52	READ	count(2)	\$25/\$26	SEEK/TELL	offset(4) / →4
\$57	WRITE	count(2) + bytes	\$2C/\$30	CHDIR/GETCWD	1. path / →text
\$FC	SET_PARSER	x, parser	\$FD/\$FE	USER/PASS	1. text
\$50	PARSE		\$20–\$2B	file ops	mode, x + URL
\$51	QUERY	1. path	\$41/\$63	ACCEPT/CLOSE CLIENT	
\$FB	SET_PARAMETER	which, value	\$44/\$72	UDP DEST/REMOTE	1. host:port

FUJI COMMANDS

Cmd	Name	Cmd	Name	Cmd	Name
\$FF	RESET (no reply)	\$E8	GET_ADAPTERCONFIG	\$D3	RANDOM_NUMBER
\$FE	GET_SSID	\$E7	NEW_DISK	\$D0-\$CD	BASE64 ENCODE
\$FD	SCAN_NETWORKS	\$E6	UNMOUNT_HOST	\$CC-\$C9	BASE64 DECODE
\$FC	GET_SCAN_RESULT	\$E5/\$E4	GET/SET DIR POSITION	\$C8-\$C2	HASH
\$FB	SET_SSID	\$E2	SET_DEVICE_FULLPATH	\$C4	GET_ADAPTERCONFIG_EXT
\$FA	GET_WIFISTATUS	\$E1/\$E0	SET/GET HOST PREFIX	\$BC-\$BF	QR CODE
\$F9	MOUNT_HOST	\$DE	WRITE_APPKEY	\$BB	GENERATE_GUID
\$F8	MOUNT_IMAGE	\$DD	READ_APPKEY	\$53	STATUS
\$F7	OPEN_DIRECTORY	\$DC	OPEN_APPKEY	\$00	DEVICE_READY
\$F6	READ_DIR_ENTRY	\$DB	CLOSE_APPKEY		
\$F5	CLOSE_DIRECTORY	\$DA	GET_DEVICE_FULLPATH		
\$F4/\$F3	READ/WRITE HOST SLOTS	\$D9	CONFIG_BOOT		
\$F2/\$F1	READ/WRITE DEVICE SLOTS	\$D8	COPY_FILE		
\$EA	GET_WIFI_ENABLED	\$D7	MOUNT_ALL		
\$E9	UNMOUNT_IMAGE	\$D6	SET_BOOT_MODE		

CLOCK COMMANDS

Cmd	Reply	Cmd	Reply
\$49 I	ISO 8601, FujiNet's zone (25)	\$50 P	ProDOS (4)
\$5A Z	ISO 8601, UTC (25)	\$53 S	SOS (19)
\$93	D M Y H M S (6)	\$47 G	the zone
\$9A	the same, alternate zone (6)	\$4C L	the zone's length + 1
\$54 T	century Y M D h m s (7)	\$74 t	set the zone (saved)
\$4D M	the same + hundredths (8)	\$99	set the alternate zone

SIZES

What	Bytes	What	Bytes
TX stream, everything	320	Reply window	1024
One network WRITE	317	One network READ	1024
Host slot name	32	Device slot	38
All host slots	256	All device slots	304
Path, OPEN_DIRECTORY, SET_DEVICE_FULLPATH	256	NetConfig (SSID + password)	97
App key	64	App key open record	6
AdapterConfig	140	AdapterConfigExtended	240

PROGRAMS IN THIS MANUAL

Program	Shows	Chapter
hello	presence, GET_ADAPTERCONFIG_EXTENDED, ACKSEQ across Reset	5
netget	OPEN, STATUS, READ, CLOSE over HTTP	6
json	SET_PARSER, PARSE, QUERY	6
dir	READ_HOST_SLOTS, MOUNT_HOST, the directory commands	7
appkey	OPEN, READ and WRITE an app key	7
boot	MOUNT_IMAGE, progress, the loader	8
clock	the clock, drawn in vblank	9
netcat	a terminal: TCP, an on-screen keyboard, a scrolling pane	D

APPENDIX C. PROGRAM LISTINGS

The programs of Chapters 5 to 9 in assembly – their C versions are printed whole in those chapters – and then the NES layers of the four games, with the callouts their chapters refer to. Every listing here is the file that was built and run.

THIS MANUAL'S PROGRAMS

LISTING 1 HELLO.S -- THE FIRST TRANSACTION

```
1 ; hello.s -- the first transaction, in assembly.
2 ;
3 ; Asks the FujiNet for GET_ADAPTERCONFIG_EXTENDED and puts the network name,
4 ; the IP address and the firmware version on the screen, straight out of the
5 ; reply window.
6
7     .include "book.inc"
8     .export main
9
10    AC_SSID = 0           ; char[33]
11    AC_VER  = 125         ; char[15]
12    AC_SIP  = 140         ; char[16], the IP already as text
13
14    .segment "CODE"
15 .proc main
16     PUTS    2, 2, title
17     jsr     fn_chk      ; 'F','N' in the status page? ❶
18     beq     have
19     PUTS    5, 2, nocart
20     jmp     show
21
22 have: CALL    FNDEVF, FNCADPX, 0 ❷
23
24     jsr     fn_go        ; commit, wait for ACKSEQ ❸
25     jne     fail        ; A = FN_ERR, or FNEWAIT
26     jsr     fn_ack       ; did the FujiNet say ACK?
27     jne     fail
28
29     PUTS    5, 2, tssid
30     ldx     #AC_SSID     ; window offset ❹
31     ldy     #24          ; at most 24 characters
32     jsr     disp_rpl
33     PUTS    7, 2, tip
34     ldx     #AC_SIP
35     ldy     #16
36     jsr     disp_rpl
37     PUTS    9, 2, tver
38     ldx     #AC_VER
39     ldy     #15
40     jsr     disp_rpl
41     jmp     show
42
43 fail: pha
44     PUTS    5, 2, tfail
```

```

44     pla
45     jsr    disp_hex
46
47 show: PUTS    12, 2, tseq    ; persists across a console Reset
48     lda    FN_ACKS 5
49     jsr    disp_hex
50     jsr    disp_on
51     jmp    :-
52 .endproc
53
54 .segment "RODATA"
55 title: .byte "FUJINET NES", 0
56 nocart: .byte "NO FUJINET CARTRIDGE", 0
57 tssid: .byte "SSID ", 0
58 tip: .byte "IP ", 0
59 tver: .byte "FW ", 0
60 tfail: .byte "FAILED: $", 0
61 tseq: .byte "ACKSEQ $", 0
62

```

LISTING 2 NETGET.S -- A FILE OVER HTTP

```

1 ; netget.s -- read a text file over HTTP and print it.
2 ; OPEN, then STATUS / READ until the server has nothing more,
3 ; then CLOSE.
4 ; Each READ asks for at most 255 bytes so one index register
5 ; walks the reply.
6     .include "book.inc"
7     .export main
8
9     .segment "ZEROPAGE"
10 row: .res 1
11 col: .res 1
12 count: .res 1
13 xsave: .res 1
14
15 .segment "CODE"
16 .proc main
17     PUTS    2, 2, title
18     lda    #4
19     sta    row
20     jsr    newline
21
22     ; OPEN N1: mode 4 (read), translation 0, the URL as the
23     payload CALL    FNDEVN, NCOPEN, 2 1
24     lda    #NMREAD
25     jsr    fn_pb
26     lda    #0
27     jsr    fn_pb
28     lda    #<url
29     sta    fn_ptr
30     lda    #>url
31     sta    fn_ptr+1
32     jsr    fn_str
33     jsr    fn_go
34     jsr    fail
35     jsr    fn_ack
36     jsr    fail
37
38 next: CALL    FNDEVN, NCSTAT, 0    ; how much is waiting?
39     2
40     jsr    fn_go
41     jsr    fail
42     lda    FN_RPLY+1    ; avail, high byte
43     bne    big
44     lda    FN_RPLY    ; avail, low byte
45     bne    some
46     lda    FN_RPLY+3    ; nothing waiting: an
47     error (136 = EOF)
48     cmp    #2    ; means the file is over
49     3
50     bcs    eof
51     jmp    next
52     lda    #255
53     some: sta    count
54
55     CALL    FNDEVN, NCREAD, 1    ; READ count bytes 4
56     lda    count
57     ldx    #0
58     jsr    fn_pw    ; one 2-byte parameter
59     jsr    fn_go
60     jsr    fail
61     ldx    #0    ; print them out of the
62
63     window
64     print: lda    FN_RPLY,x
65     cmp    #$0A
66     bne    :+
67     stx    xsave    ; disp_at uses X
68     inc    row
69     jsr    newline
70     ldx    xsave
71     jmp    adv
72     : sta    PPU_DATA
73     adv:    inx

```

```

70      cpx      FN_RXL0
71      bne     print
72      jmp     next
73
74 eof:   CALL    FNDEVN, NCCLOSE, 0
75      jsr     fn_go
76      inc     row
77      jsr     newline
78      lda     #<tend
79      sta     fn_ptr
80      lda     #>tend
81      sta     fn_ptr+1
82      jsr     disp_str
83      jmp     show
84
85 fail:  pha
86      PUTS     20, 2, tfail
87      pla
88      jsr     disp_hex
89 show:  jsr     disp_on
90      jmp     :-
91 .endproc
92
93 ; newline -- PPU cursor to column 2 of `row`.
94 .proc newline
95     lda     row
96     ldx     #2
97     jmp     disp_at
98 .endproc
99
100 .segment "RODATA"
101 title: .byte "NETGET", 0
102 url:   .byte "N:HTTP://127.0.0.1:8765/hello.txt", 0
103 tend:  .byte "-- END --", 0
104 tfail: .byte "FAILED: $", 0
105

```

LISTING 3 JSON.S -- A JSON ROUND TRIP

<pre> 1 ; json.s -- fetch a JSON document and pick fields out of it. 2 ; 3 ; OPEN as for any file; SET_PARSER to JSON and PARSE; then, per field, QUERY 4 ; a path and READ the value the query leaves waiting. 5 6 .include "book.inc" 7 .export main 8 9 .segment "ZEROPAGE" 10 row: .res 1 11 idx: .res 1 12 13 .segment "CODE" 14 .proc main 15 PUTS 2, 2, title 16 17 CALL FNDEVN, NCOPEN, 2 18 lda #NMREAD 19 jsr fn_pb 20 lda #0 21 jsr fn_pb 22 lda #<url 23 sta fn_ptr </pre>	<pre> 24 lda #>url 25 sta fn_ptr+1 26 jsr fn_str 27 jsr fn_go 28 jne fail 29 30 CALL FNDEVN, NCPARSE, 2 ; channel mode: JSON 1 31 lda #1 32 jsr fn_pb 33 lda #1 34 jsr fn_pb 35 jsr fn_go 36 jne fail 37 CALL FNDEVN, NCPARSE, 0 ; the FujiNet parses the whole body 2 38 jsr fn_go 39 jne fail 40 jsr fn_ack 41 jne fail 42 43 lda #4 44 sta row </pre>
---	---

```

45      lda      #0
46      sta      idx
47      field:   lda      paths,x          ; fn_ptr = paths[idx]
48              ldx      paths,x
49              sta      fn_ptr
50              lda      paths+1,x
51              sta      fn_ptr+1
52              lda      row
53              ldx      #2
54              jsr      disp_at
55              jsr      disp_str          ; the path itself, as a

56      label
57      CALL      FNDEVN, NCQUERY, 0      ; QUERY: the path is the
payload 3
58      jsr      fn_path                  ; NUL-padded to 256, as
the library does
59      jsr      fn_go
60      jne      fail
61      CALL      FNDEVN, NCSTAT, 0        ; the value's length
62      jsr      fn_go
63      jne      fail
64      lda      FN_RPLY
65      beq      skip
66      CALL      FNDEVN, NCREAD, 1
67      lda      FN_RPLY
reply: fn_beg wrote 4
68      ldx      #0                      ; only registers, not
the window
69      jsr      fn_pw
70      jsr      fn_go
71      jne      fail
72      lda      row
73      ldx      #17
74      jsr      disp_at
75      ldx      #0
76      ldy      FN_RXLO
77      show1:   lda      FN_RPLY,x
78              cmp      #$20            ; stop at the trailing
newline
79      bcc      skip
80      sta      PPU_DATA
81      inx
82      dey
83      bne      show1
84      skip:   inc      row
85              inc      idx
86              inc      idx
87              lda      idx
88              cmp      #2*5
89              jne      field
90
91      CALL      FNDEVN, NCCLOSE, 0
92      jsr      fn_go
93      jmp      show
94
95      fail:   pha
96              PUTS      20, 2, tfail
97              pla
98              jsr      disp_hex
99      show:   jsr      disp_on
100      :      jmp      :-
101      .endproc
102
103      .segment "RODATA"
104      title:   .byte "JSON", 0
105      url:     .byte "N:HTTP://127.0.0.1:8765/hello.json", 0
106      p0:      .byte "/name", 0
107      p1:      .byte "/console", 0
108      p2:      .byte "/year", 0
109      p3:      .byte "/mailbox/base", 0
110      p4:      .byte "/mailbox/reply", 0
111      paths:   .addr p0, p1, p2, p3, p4
112      tfail:   .byte "FAILED: $", 0
113

```

LISTING 4 DIR.S -- A DIRECTORY ON A TNFS HOST

```

1 ; dir.s -- list a directory on a TNFS host.
2 ;
3 ; READ_HOST_SLOTS for the host's name, then MOUNT_HOST,
  OPEN_DIRECTORY and one
4 ; READ_DIR_ENTRY per line until the $7F end marker.
5
6     .include "book.inc"
7     .export main
8
9 HOST    = 7                ; host slot 8: ec.tnfs.io
10 ROWS    = 20
11
12     .segment "ZEROPAGE"
13 row:     .res 1
14
15     .segment "CODE"
16 .proc main
17     CALL    FNDEVF, FNCRHST, 0        ; 8 x 32-byte host names
18     jsr     fn_go
19     jne     fail
20     PUTS    2, 2, title
21     ldx     #HOST*32 ①
22     ldy     #32
23     jsr     disp_rpl                ; this slot's name, from
  the window
24
25     CALL    FNDEVF, FNCMHST, 1        ; MOUNT_HOST(host)
26     lda     #HOST
27     jsr     fn_pb
28     jsr     fn_go
29     jne     fail
30     jsr     fn_ack
31     jne     fail
32
33     CALL    FNDEVF, FNCODIR, 1        ; OPEN_DIRECTORY(host),
  path "/"
34     lda     #HOST
35     jsr     fn_pb
36     lda     #<root
37     sta     fn_ptr
38     lda     #>root
39     sta     fn_ptr+1
40     jsr     fn_path                ; exactly 256 bytes,
  NUL-padded ②
41     jsr     fn_go
42     jne     fail
43     jsr     fn_ack
44     jne     fail
45
46     lda     #4
47     sta     row
48     CALL    FNDEVF, FNCRDIR, 2        ; READ_DIR_ENTRY(maxlen,
  entry: flags)
49     lda     #28
50     jsr     fn_pb
51     lda     #0
52     jsr     fn_pb
53     jsr     fn_go
54     jne     fail
55     jsr     fn_ack
56     jne     close
57     lda     FN_RPLY
58     cmp     #$7F                    ; end of directory ③
59
60     beq     close
61     lda     row
62     ldx     #3
63     jsr     disp_at
64     ldx     #0
65     ldy     #28
66     jsr     disp_rpl
67     inc     row
68     lda     row
69     cmp     #4+ROWS
70     bne     entry
71 close: CALL    FNDEVF, FNCCDIR, 0
72     jsr     fn_go
73     jmp     show
74
75 fail:  pha
76     PUTS    26, 2, tfail
77     pla
78     jsr     disp_hex
79     jsr     disp_on
80     jmp     :-
81 .endproc
82
83     .segment "RODATA"
84 title: .byte "DIR OF ", 0
85 root:  .byte "/", 0
86 tfail: .byte "FAILED: $", 0
87

```

LISTING 5 BOOT.S -- LOADING AN IMAGE, WITH PROGRESS

```

1 ; boot.s -- fetch a cartridge image from the SD card and run it,
2 ; progress count drawn while MOUNT_IMAGE is still outstanding.
3 ;
4 ; MOUNT_HOST -> SET_DEVICE_FULLPATH -> MOUNT_IMAGE. The last one
5 ; does not
6 ; answer until the whole image has crossed into the cartridge's
7 ; store, so it
8 ; is launched with fn_launch and watched, one vblank at a time,
9 ; through
10 ; BOOT_PCT. Then BOOTLOCK and a jump into the loader ROM at
11 ; $5800.
12
13 .include "book.inc"
14 .export main
15
16 HOST = 0 ; host slot 1: the SD card
17 SLOT = 0 ; device slot 1
18 MREAD = 1 ; disk access mode: read
19 LIMIT = 65 ; seconds: outlasts the cart's
20 own 60
21
22 .segment "ZEROPAGE"
23 .res 1
24 .res 1
25
26 .segment "CODE"
27 .proc main
28 PUTS 2, 2, title
29 lda #<path
30 sta fn_ptr
31 lda #>path
32 sta fn_ptr+1
33 jsr disp_str
34 PUTS 4, 2, tload
35 jsr disp_on
36
37 CALL FNDEVF, FNCMHST, 1 ; MOUNT_HOST(host)
38 lda #HOST
39 jsr fn_pb
40 jsr fn_go
41 jne fail
42 jsr fn_ack
43 jne fail
44
45 CALL FNDEVF, FNCSDFP, 3 ;
46 SET_DEVICE_FULLPATH(slot, host, mode)
47 lda #SLOT
48 jsr fn_pb
49 lda #HOST
50 jsr fn_pb
51 lda #MREAD
52 jsr fn_pb
53 lda #<path
54 sta fn_ptr
55 lda #>path
56 sta fn_ptr+1
57 jsr fn_path ; exactly 256 bytes
58 jsr fn_go
59 jne fail
60 jsr fn_ack
61
62 jne fail
63 CALL FNDEVF, FNCMIMG, 2 ; MOUNT_IMAGE(slot,
64 mode)
65 lda #SLOT
66 jsr fn_pb
67 lda #MREAD
68 jsr fn_pb
69 jsr fn_launch ; sent; do not wait here
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105

```

```

106 .endproc
107
108 .segment "RODATA"
109 title: .byte "BOOTING ", 0
110 tload: .byte "LOADING ", %, 0
111 tfail: .byte "FAILED: $", 0
112 path: .byte "/nesbook/hello.bin", 0
113

```

LISTING 6 APPKEY.S -- A RUN COUNTER

```

1 ; appkey.s -- count how many times this program has been run.
2 ;
3 ; An app key is opened with a 6-byte record -- creator (2 bytes,
4 ; LE), app,
5 ; key, mode (0 read, 1 write), reserved -- and then read or
6 ; written. On the
7 ; RS-232 Fujitsu the READ reply starts with the key's length, 2
8 ; bytes LE.
9
10 .include "book.inc"
11 .export main
12
13 CREATOR = $SE5E ; a scratch creator id
14 APP = $01
15 KEY = $00
16
17 .segment "ZEROPAGE"
18 runs: .res 2
19
20 .proc main
21     PUTS 2, 2, title
22     lda #0
23     sta runs
24     sta runs+1
25
26     lda #0 ; mode 0: read
27     jsr open_key
28     jne fail
29     CALL FNDEVF, FNCRKEY, 0
30     jsr fn_go
31     jne fail
32     jsr fn_ack
33     count ; never written: start
34
35     from zero
36     lda FN_RPLY+1 ; length, high byte 1
37     bne have
38     lda FN_RPLY ; length, low byte
39     cmp #2
40     bcc count
41     have: lda FN_RPLY+2 ; the two bytes after
42     the length
43     sta runs
44     lda FN_RPLY+3
45     sta runs+1
46
47     count: inc runs ; RMW on console RAM:
48     fine 2
49
50     bne :+
51     inc runs+1
52
53
54 :
55     lda #1 ; mode 1: write
56     jsr open_key
57     jne fail
58     CALL FNDEVF, FNCWKEY, 0 ; the payload is the
59     key's new value
60     lda runs
61     jsr fn_txb
62     runs+1
63     lda fn_txb
64     jsr fn_go
65     jne fail
66     jsr fn_ack
67     jne fail
68
69     PUTS 5, 2, tran
70     PUTS 6, 2, tdollar
71     lda runs+1
72     jsr disp_hex
73     lda runs
74     jsr disp_hex
75     PUTS 6, 8, ttimes
76     PUTS 9, 2, treset
77     jmp show
78
79 fail: pha
80     PUTS 12, 2, tfail
81     pla
82     jsr disp_hex
83     show: jsr disp_on
84     : jmp :-
85     .endproc
86
87 ; open_key -- OPEN_APPKEY with mode A. Returns as fn_go / fn_ack
88 do.
89 .proc open_key 3
90     pha
91     CALL FNDEVF, FNCOWKEY, 0
92     lda #<CREATOR
93     jsr fn_txb
94     lda #>CREATOR
95     jsr fn_txb
96     lda #APP
97     jsr fn_txb
98     lda #KEY
99     jsr fn_txb
100     pla
101     jsr fn_txb ; mode
102     lda #0
103     jsr fn_txb ; reserved
104     jsr fn_go

```

```

94         bne     done
95         jsr     fn_ack
96 done:    rts
97 .endproc
98
99         .segment "RODATA"
100 title:   .byte "APPKEY", 0
101 tran:    .byte "THIS PROGRAM HAS RUN", 0
102 tdollar: .byte "$", 0
103 times:   .byte "TIMES.", 0
104 treset:  .byte "PRESS RESET TO COUNT AGAIN.", 0
105 tfail:   .byte "FAILED: $", 0
106

```

LISTING 7 CLOCK.S -- THE TIME, ONCE A SECOND

```

1 ; clock.s -- the FujiNet's network time, once a second.
2 ;
3 ; The clock device answers 'I' with a 25-byte ISO 8601 string in the
4 ; FujiNet's time zone: 2026-10-04T18:20:00-0500. The date and the time are
5 ; copied straight from the reply window to the screen during vblank.
6
7         .include "book.inc"
8         .export main
9
10
11 tick:   .segment "ZEROPAGE"
12         .res 1
13
14         .segment "CODE"
15 .proc main
16     PUTS    2, 2, title
17     jsr     disp_on
18
19 loop:    CALL    FNDEVC, CLKISO, 0
20         jsr     fn_go           ; runs with the screen on: the NMI
21         bne     wait           ; may land anywhere in it
22         jsr     fn_ack
23         bne     wait
24
25         jsr     wait_vbl ❶
26         lda     #5
27         ldx     #2
28         jsr     disp_at
29         ldx     #0             ; "2026-10-04"
30         ldy     #10
31         jsr     disp_rpl
32         lda     #6
33         ldx     #2
34         jsr     disp_at
35         ldx     #11           ; "18:20:00" ❷
36         ldy     #8
37         jsr     disp_rpl
38         jsr     scroll0
39
40 wait:    lda     #60
41         sta     tick
42 :        jsr     wait_vbl
43         dec     tick
44         bne     :-
45         jmp     loop

```

```

46 .endproc
47
48 .segment "RODATA"
49 title: .byte "CLOCK", 0
50

```

LISTING 8 BOOKLIB.S -- WHAT THIS MANUAL ADDS TO FUJILIB.S

```

1 ; booklib.s -- the few routines this book adds to the bring-up's
  fujilib.s.
2 ;
3 ; nmi          a vblank handler that only counts frames
4 ; wait_vbl    wait for the next vblank (rendering may be on)
5 ; scroll0     after drawing in vblank: scroll home, NMI on
6 ; fn_str      append the NUL-terminated string at fn_ptr,
  unpadding
7 ; fn_launch   commit the transaction and return at once
8 ; fn_done     has the launched transaction been answered? (C
  set if so)
9 ; disp_d3     A as three decimal digits, right-aligned with
  spaces
10
11 .include "fujinet.inc"
12
13 fn_done, export nmi, wait_vbl, scroll0, fn_str, fn_launch,
  fn_done, disp_d3
14 .exportzp frames
15
16 .segment "ZEROPAGE"
17 frames: .res 1 ; bumped once per vblank by the
  NMI
18 want: .res 1 ; the sequence fn_launch is
  waiting for
19
20 .segment "CODE"
21
22 ; The whole vblank handler. It never writes to $5500-$57FF, so
  it can land in
23 ; the middle of a transaction without disturbing it.
24 .proc nmi 1
25     inc     frames ; RMW on console RAM: harmless
26     rti
27 .endproc
28
29 .proc wait_vbl
30     lda     frames
31     cmp     frames
32     beq     :-
33     rts
34 .endproc
35
36 .proc scroll0
37     lda     #0
38     sta     PPU_SCROLL
39     sta     PPU_SCROLL
40     lda     %100000000 ; NMI on, nametable 0
41     sta     PPU_CTRL
42     rts
43 .endproc
44
45 ; fn_str -- append the string at FN_PTR without its NUL. A URL
  needs no
46 ; padding: the payload is as long as the bytes you store.
47 .proc fn_str
48     ldy     #0
49     lda     (FN_PTR),y
50     beq     done
51     sta     FN_TX
52     iny
53     bne     loop
54 done: rts
55 .endproc
56
57 ; fn_launch -- like fn_go, but does not wait. The sequence still
  comes from
58 ; the cart's ACKSEQ + 1, skipping 0.
59 .proc fn_launch 2
60     lda     FN_ACKS
61     clc
62     adc     #1
63     bne     :+
64     lda     #1
65     sta     want
66     sta     FN_RSEL+FR_SEQ ; the one store that sends it
67     rts
68 .endproc
69
70 ; fn_done -- C set once ACKSEQ echoes the launched sequence.
71 .proc fn_done 3
72     lda     FN_ACKS
73     cmp     want
74     beq     yes
75     clc
76     rts
77 yes: sec
78     rts
79 .endproc
80
81 ; disp_d3 -- A (0-255) as three characters, leading zeros as
  spaces.
82 .proc disp_d3
83     ldx     #'0'-1
84 h:     inx
85     sec
86     sbc     #100
87     bcs     h
88     adc     #100
89     cpx     #'0'
90     bne     :+
91     ldx     #' '
92     stx     PPU_DATA
93     ldy     #'0'-1
94 t:     iny
95     sec
96     sbc     #10

```

```

97         bcs     t
98         adc     #10
99         cpy     #'0'
100        bne     :+
101        cpx     #' '           ; a zero tens digit is blank
    only after a
102        bne     :+           ; blank hundreds digit
103        ldy     #' '
104        :       sty     PPU_DATA
105        ora     #'0'
106        sta     PPU_DATA
107        rts
108    .endproc
109

```

5 CARD STUD

LISTING 9 SRC/NES/NETWORK.C

```

1  #ifdef BUILD_NES
2
3  /**
4   * @brief   Network functions
5   * @author   Thomas Cherryhomes
6   * @email    thom dot cherryhomes at gmail dot com
7   * @license  gpl v. 3, see LICENSE for details
8   */
9
10 #include <stdint.h>
11 #include <fujinet-network.h>
12
13 static uint8_t initialized = 0;
14
15 uint8_t getResponse(char *url, unsigned char *buffer, uint16_t max_len)
16 {
17     int16_t count;
18
19     if (!initialized) {
20         if (network_init() != FN_ERR_OK)
21             return 0;
22         initialized = 1;
23     }
24
25     // Don't read from a channel that never opened - network_read() would
26     // otherwise poll status on a dead unit and hand back whatever it got.
27     if (network_open(url, OPEN_MODE_HTTP_GET_H, OPEN_TRANS_NONE) != FN_ERR_OK) ❶
28         return 0;
29
30     count = network_read(url, buffer, max_len); ❷
31     network_close(url);
32     return count > 0;
33 }
34
35 #endif /* BUILD_NES */
36
37

```

LISTING 10 SRC/NES/PPU.S

```
1 ; ppu.s -- one PPU write through cc65's NMI-flushed ring buffer.
2 ;
3 ; The game draws whenever it likes; the writes queue here and the NES runtime's
4 ; vblank handler (crt0.s nmi -> ppubuf_flush) lands them while rendering is
5 ; off. ppubuf_put blocks when the queue is full, so this needs NMI enabled --
6 ; initGraphics() turns it on before the first call.
7
8     .export      _ppu_put
9     .import      popax, ppubuf_put
10    .importzp     tmp1
11
12 ; void __fastcall__ ppu_put(unsigned addr, unsigned char val)
13 .proc _ppu_put
14     sta         tmp1
15     jsr         popax          ; A = addr low, X = addr high 3
16
17     pha
18     txa
19     tay         ; Y = high
20     pla
21     tax         ; X = low
22     lda         tmp1
23     jmp         ppubuf_put     ; A = value, X = low, Y = high 4
24 .endproc
```

LISTING 11 SRC/NES/INPUT.C

```
1 #ifdef BUILD_NES
2
3 /**
4  * @brief NES input routines
5  * @author Thomas Cherryhomes
6  * @license gpl v.3
7  */
8
9 #include <joystick.h>
10 #include "../platform-specific/input.h"
11
12 /*
13  * An NES controller is a d-pad and four buttons, read through
14  * cc65's joystick
15  * driver (installed by initGraphics()) -- see graphics.c, which
16  * needs it for
17  * the same reason it needs the PPU: both are platform start-up).
18  * Every button is reported as a KEY rather than as a stick bit.
19  * readCommonInput()
20  * returns early whenever readJoystick()'s value changes and only
21  * reaches
22  * getPlatformKey() when it has not, so a button that appeared in
23  * both places
24  * would be seen twice: once as the joystick edge, and again on
25  * the next pass as
26  * a key, because the button is still held. Keeping every button
27  * in
28  * getPlatformKey() puts all of them behind one edge detector.
29
30     A      RETURN  (select / join / confirm)
31     B      ESCAPE  (back / the in-game menu)
32     START  'h'     how to play
33     SELECT 'n'     change name
34     SELECT+START 'q' quit the table
35
36     START and SELECT are reported on RELEASE so that the chord can
37     be told from
38     either button alone: whatever set of the two was down when the
39     last of them
40     came up is the key. A and B report on press.
41 */
42
43 #define DIRS (JOY_UP_MASK | JOY_DOWN_MASK | JOY_LEFT_MASK |
44             JOY_RIGHT_MASK)
45 #define MODS (JOY_SELECT_MASK | JOY_START_MASK)
46
47 static unsigned char lastButtons;
48 static unsigned char modsSeen;
49
50 unsigned char readJoystick()
51 {
52     return joy_read(JOY_1) & DIRS;
53 }
54
55 void initPlatformKeyboardInput(void)
56 {
57     lastButtons = 0;
58     modsSeen = 0;
59 }
60
61 #endif
```

```

51 int getPlatformKey(void)
52 {
53     unsigned char buttons = joy_read(JOY_1) & ~DIRS;
54     unsigned char pressed = buttons & ~lastButtons;
55     unsigned char released = lastButtons & ~buttons;
56
57     lastButtons = buttons;
58     modsSeen |= buttons & MODS; 5
59
60     if (pressed & JOY_BTN_A_MASK)
61         return KEY_RETURN;
62     if (pressed & JOY_BTN_B_MASK)
63         return KEY_ESCAPE;
64
65     if ((released & MODS) && !(buttons & MODS)) {
66         unsigned char chord = modsSeen;
67         modsSeen = 0;
68         if (chord == MODS)
69             return 'q';
70         if (chord == JOY_START_MASK)
71             return 'h';
72         if (chord == JOY_SELECT_MASK)
73             return 'n';
74     }
75     return 0;
76 }
77
78 #endif /* BUILD_NES */
79
80

```

LISTING 12 SRC/NES/UTIL.C

```

1  #ifdef BUILD_NES
2
3  /**
4   * @brief Utility Functions
5   * @author Thomas Cherrymhomes
6   * @email thom dot cherrymhomes at gmail dot com
7   * @license gpl v. 3, see LICENSE for details
8   */
9
10 #include <stdbool.h>
11 #include <stdint.h>
12 #include <string.h>
13 #include <stdlib.h>
14 #include <time.h>
15 #include <fujinet-fuji.h>
16 #include <fujinet-nes.h>
17 #include "vars.h"
18 #include "../platform-specific/graphics.h"
19 #include "../platform-specific/util.h"
20
21 // The firmware's WRITE HOST SLOTS expects all 8 slots.
22 #define FUJI_HOST_SLOT_COUNT 8
23 #define LOBBY_DEVICE_SLOT 0
24 #define LOBBY_MODE_READ 1
25
26 // cc65's NMI handler counts frames in clock(); the game timer
27 // is frames since
28 // the last resetTimer().
29 static clock_t timerBase;
30
31 void resetTimer(void)
32 {
33     timerBase = clock();
34 }
35
36 int getTime(void)
37 {
38     return (int) (clock() - timerBase);
39 }
40
41 static bool sameHost(const char *a, const char *b)
42 {
43     while (*a && *b)
44     {
45         if ((*a | 0x20) != (*b | 0x20))
46             return false;
47         a++;
48         b++;
49     }
50     return *a == *b;
51 }
52
53 /*
54  SET_DEVICE_FULLPATH is a fixed 256-byte payload -- a short one
55  is rejected on
56  the ESP32 side -- so the path is padded here rather than at
57  the call site.
58 */
59 static const char lobbyPath[MAX_FILENAME_LEN] = "nes/lobby.nes";

```

```

57 static const char lobbyHost[] = "ec.tnfs.io";
58
59 static HostSlot slots[FUJI_HOST_SLOT_COUNT];
60
61 static uint8_t findLobbyHost(void)
62 {
63     uint8_t i;
64
65     if (!fuji_get_host_slots(slots, FUJI_HOST_SLOT_COUNT))
66         return FUJI_HOST_SLOT_COUNT;
67     for (i = 0; i < FUJI_HOST_SLOT_COUNT; i++)
68         if (sameHost(lobbyHost, (const char *) slots[i]))
69             return i;
70     return FUJI_HOST_SLOT_COUNT;
71 }
72
73 // The slot is used as found, never created: a player who
74 // reached this client
75 // through the FujiNet Lobby already has the host, and anyone
76 // else is told to
77 // add it in CONFIG.
78 static bool mountLobby(uint8_t slot)
79 {
80     if (!fuji_mount_host_slot(slot))
81         return false;
82     if (!fuji_set_device_filename(LOBBY_MODE_READ, slot,
83         LOBBY_DEVICE_SLOT,
84         (char *) lobbyPath))
85         return false;
86     // Only starts the transfer. The image is pushed to the
87     // cartridge 7
88     // asynchronously, after this has already been answered.
89     return fuji_mount_disk_image(LOBBY_DEVICE_SLOT,
90         LOBBY_MODE_READ) != 0;
91 }
92
93 void quit(void)
94 {
95     // Not `state`: src/misc.h defines that as a macro for the
96     // game state.
97     uint8_t bootState, slot, pct = 0xFF;
98     char pctText[5];
99
100     drawStatusText("LOADING LOBBY...");
101
102     if (!fuji_nes_present())
103     {
104         drawStatusText("NO FUJINET CARTRIDGE");
105         return;
106     }
107
108     slot = findLobbyHost();
109     if (slot == FUJI_HOST_SLOT_COUNT)
110     {
111         drawStatusText("ADD EC.TNFS.IO IN CONFIG");
112         return;
113     }
114
115     if (!mountLobby(slot))
116     {
117         drawStatusText("LOBBY NOT AVAILABLE");
118         return;
119     }

```

```

115 // Watch the cartridge's own progress counter rather than
116 // guessing a delay.
117 for (;;)
118 {
119     bootState = fuji_nes_boot_state();
120     if (bootState == FUJI_NES_BOOT_READY)
121         break;
122     if (bootState == FUJI_NES_BOOT_FAILED)
123     {
124         drawStatusText("LOBBY LOAD FAILED");
125         return;
126     }
127     if (fuji_nes_boot_percent() != pct)
128     {
129         pct = fuji_nes_boot_percent();
130         itoa(pct, pctText, 10);
131         strcat(pctText, "%");
132         drawStatusTextAt(18, pctText);
133     }
134 }
135 // Stop the firmware serving CONFIG at boot, then hand the
136 // console to the
137 // cartridge's loader ROM. It does not return.
138 fuji_set_boot_config(0);
139 fuji_nes_boot(); 6
140 }
141 #endif /* BUILD_NES */
142

```

LISTING 13 SRC/NES/SOUND.C

```

1  #ifdef BUILD_NES
2
3  /**
4   * @brief NES Sound Routines (2A03 APU, pulse channels 1+2)
5   * @author Thomas Cherryhomes
6   * @email thom dot cherryhomes at gmail dot com
7   * @license gpl v. 3, see LICENSE for details
8   */
9
10 #include <stdint.h>
11
12 void waitvsync(void);
13
14 #define PULSE1_CTRL (*(volatile uint8_t *) 0x4000)
15 #define PULSE1_SWEEP (*(volatile uint8_t *) 0x4001)
16 #define PULSE1_LO (*(volatile uint8_t *) 0x4002)
17 #define PULSE1_HI (*(volatile uint8_t *) 0x4003)
18 #define PULSE2_CTRL (*(volatile uint8_t *) 0x4004)
19 #define PULSE2_SWEEP (*(volatile uint8_t *) 0x4005)
20 #define PULSE2_LO (*(volatile uint8_t *) 0x4006)
21 #define PULSE2_HI (*(volatile uint8_t *) 0x4007)
22 #define APU_STATUS (*(volatile uint8_t *) 0x4015)
23
24 #define PULSE_ON 0xBF /* 50% duty, length halted,
25 constant volume 15 */
26 #define PULSE_OFF 0xB0 /* same, volume 0 */
27
28 #define MIN_GATE 3 /* shorter blips are barely
29 audible */
30
31 /**
32 * @brief Brain dead beep routine
33 * @param hz Frequency in Hz
34 * @param gate Delay in vertical blanks
35 * @param postGate Delay after tone off in vertical blanks
36 */
37 void beep(int hz, int gate, int postGate)
38 {
39     // 11-bit timer from the 1.79MHz CPU clock; the lowest
40     // reachable tone is
41     // ~55Hz, so the sub-bass beeps clamp there.
42     uint16_t t = 11861UL / (uint16_t)hz - 1; 8
43
44     if (t > 0x7FF)
45         t = 0x7FF;
46
47     // Stretch short blips, taking the extra out of the rest so
48     // the cue
49     // keeps its rhythm.
50     if (gate < MIN_GATE)
51     {
52         postGate -= MIN_GATE - gate;
53         if (postGate < 0)
54             postGate = 0;
55         gate = MIN_GATE;
56     }
57
58     // Both pulses in unison: one alone at volume 15 is quiet.
59     // The high
60     // writes restart the phase, so back to back they stay in
61     // step.
62
63     PULSE1_LO = PULSE2_LO = t & 0xFF;
64     PULSE1_HI = t >> 8;
65     PULSE2_HI = t >> 8;
66     PULSE1_CTRL = PULSE_ON;
67
68     while (gate-->0)
69         waitvsync();
70
71     PULSE1_CTRL = PULSE2_CTRL = PULSE_OFF;
72
73     while (postGate-->0)
74         waitvsync();
75 }
76
77 void soundDealCard()
78 {
79     beep(150,1,5);
80 }
81
82 void soundJoinGame()
83 {
84     beep(430,5,8);
85     beep(340,5,0);
86     beep(500,5,0);
87 }
88
89 void soundPlayerLeft()
90 {
91     uint8_t i;
92     for (i=80;i>=50;i-=10)
93         beep(i,2,15);
94 }
95
96 void soundPlayerJoin()
97 {
98     uint8_t i;
99     for (i=50;i<=80;i+=10)
100         beep(i,2,15);
101 }
102
103 void soundGameDone()
104 {
105     beep(311,10,0);
106     beep(330,20,0);
107     beep(392,10,0);
108     beep(415,20,0);
109 }
110
111 void soundMyTurn()
112 {
113     beep(430,4,2);
114     beep(430,4,2);
115 }
116
117 void soundTick()
118 {
119     beep(50,2,0);
120 }
121
122 void soundCursor()
123 {
124     beep(300,2,0);
125 }

```

```

118 }
119
120 void soundCursorInvalid()
121 {
122     beep(100,2,0);
123 }
124
125 void soundSelectMove()
126 {
127     beep(300,3,1);
128     beep(350,3,0);
129 }
130
131 void initSound()
132 {
133     APU_STATUS = 0x03;          /* pulse 1 and 2 */
134     PULSE1_SWEEP = PULSE2_SWEEP = 0x08; /* sweep off, negate set
so low notes are not muted */ 9
    PULSE1_CTRL = PULSE2_CTRL = PULSE_OFF;
135 }
136
137 void soundTakeChip(uint16_t counter)
138 {
139     beep(50+counter*20,2,2);
140 }
141
142 #endif /* BUILD_NES */
143
144

```

LISTING 14 SRC/NES/OSK.C

```

1  #ifdef BUILD_NES
2
3  /**
4   * @brief NES on-screen keyboard
5   * @author Thomas Cherryhomes
6   * @email thom dot cherrymhomes at gmail dot com
7   * @license gpl v. 3, see LICENSE for details
8   */
9
10 #include <joystick.h>
11 #include <string.h>
12 #include "vars.h"
13 #include "../platform-specific/graphics.h"
14 #include "../platform-specific/sound.h"
15
16 /*
17  The same grid as the ColecoVision client (src/coleco/osk.c),
18  the Intellivision
19  client and the NES CONFIG's fujiedit.c: the cursor position IS
20  the character,
21  so there is no shift key and no paging. Cancel is the OK
22  cell's absence --
23  the caller only reaches this screen when a name is required --
24  and the grid
25  never runs a mailbox transaction. Uppercase only: drawText()
26  uppercases
27  everything it prints, so lowercase is not representable
28  anywhere here.
29
30  d-pad move      A take the cell      B backspace
31  START OK
32
33  */
34
35 #define OSK_COLS 10
36 #define OSK_ROWS 4
37 #define OSK_PITCH 3 /* marker column + two
label columns */
38 #define OSK_X 1
39 #define OSK_Y 17
40
41 /* The last row is one short: its tenth cell does not exist. */
42 #define OSK_LAST_ROW_COLS 9
43
44 #define CELL_SPACE (OSK_COLS * 3 + 6) /* index 36 */
45 #define CELL_DEL (CELL_SPACE + 1)
46 #define CELL_OK (CELL_SPACE + 2)
47
48 static const char cells[] =
49 {
50     "ABCDEFGHIJ",
51     "KLMNOPQRST",
52     "UVWXYZ0123",
53     "456789";
54
55 static const char *labelSpace = "SP";
56 static const char *labelDel = "<";
57 static const char *labelOk = "OK";
58
59 #define REPEAT_FIRST 18 /* vblanks before a held
direction repeats */
60 #define REPEAT_NEXT 5
61
62 #define DIRS (JOY_UP_MASK | JOY_DOWN_MASK | JOY_LEFT_MASK |
JOY_RIGHT_MASK)

```

```

54 #define BUTTONS (JOY_BTN_A_MASK | JOY_BTN_B_MASK |
55 JOY_START_MASK)
56 static unsigned char curX, curY;
57 static unsigned char cellCols(unsigned char row)
58 { return row == OSK_ROWS - 1 ? OSK_LAST_ROW_COLS : OSK_COLS;
59 }
60
61 static void drawCell(unsigned char col, unsigned char row,
62 unsigned char marked)
63 {
64     unsigned char idx = row * OSK_COLS + col;
65     unsigned char x = OSK_X + col * OSK_PITCH;
66     char glyph[3];
67     drawText(x, OSK_Y + row, marked ? ">" : " ");
68
69     if (idx == CELL_SPACE)
70         drawText(x + 1, OSK_Y + row, labelSpace);
71     else if (idx == CELL_DEL)
72         drawText(x + 1, OSK_Y + row, labelDel);
73     else if (idx == CELL_OK)
74         drawText(x + 1, OSK_Y + row, labelOk);
75     else
76     {
77         glyph[0] = cells[idx];
78         glyph[1] = ' ';
79         glyph[2] = 0;
80         drawText(x + 1, OSK_Y + row, glyph);
81     }
82 }
83
84 static void drawGrid(void)
85 {
86     unsigned char row, col;
87     for (row = 0; row < OSK_ROWS; row++)
88         for (col = 0; col < cellCols(row); col++)
89             drawCell(col, row, row == curY && col == curX);
90 }
91
92 static void clearGrid(void)
93 {
94     unsigned char row;
95     for (row = 0; row < OSK_ROWS; row++)
96         drawText(0, OSK_Y + row, " ");
97 }
98
99 /* Repaint the field and its trailing cursor chip, the same
100 shape
101 inputFieldCycle() draws on the platforms that have a
102 keyboard. */
103 static void drawField(unsigned char x, unsigned char y, unsigned
104 char max,
105 const char *buffer)
106 {
107     unsigned char len = (unsigned char) strlen(buffer);
108     drawText(x, y, " ");
109     drawText(x, y, buffer);
110     if (len < max)
111         drawChip(x + len, y);
112 }
113
114 }
115
116 static void appendChar(char *buffer, unsigned char *len,
117 unsigned char max,
118 char ch)
119 {
120     if (*len >= max)
121     {
122         soundCursorInvalid();
123         return;
124     }
125     buffer[*len++] = ch;
126     buffer[*len] = 0;
127     soundCursor();
128 }
129
130 void platformNameEntry(unsigned char x, unsigned char y,
131 unsigned char max,
132 char *buffer)
133 {
134     unsigned char len = (unsigned char) strlen(buffer);
135     unsigned char hold = 0, lastDir = 0, lastButtons = 0;
136     unsigned char pad, dir, move, buttons, pressed, idx;
137
138     curX = 0;
139     curY = 0;
140     drawGrid();
141     drawField(x, y, max, buffer);
142     for (;;)
143     {
144         waitvsync();
145         pad = joy_read(JOY_1);
146         dir = pad & DIRS;
147         buttons = pad & BUTTONS;
148
149         /* Auto-repeat is counted in vblanks: this loop runs once
150 per frame. */
151         move = 0;
152         if (dir != lastDir)
153         {
154             lastDir = dir;
155             hold = REPEAT_FIRST; 10
156             move = dir;
157         }
158         else if (dir && --hold == 0)
159         {
160             hold = REPEAT_NEXT;
161             move = dir;
162         }
163         if (move)
164         {
165             drawCell(curX, curY, 0);
166             if ((move & JOY_LEFT_MASK) && curX)
167                 curX--;
168             else if ((move & JOY_RIGHT_MASK) && curX + 1 <
169 cellCols(curY))
170                 curX++;
171             else if ((move & JOY_UP_MASK) && curY)
172                 curY--;
173             else if ((move & JOY_DOWN_MASK) && curY + 1 < OSK_ROWS)
174                 curY++;
175         }
176     }
177 }

```

```

175         if (curX >= cellCols(curY))
176             curX = cellCols(curY) - 1;
177
178         drawCell(curX, curY, 1);
179         soundCursor();
180     }
181 }
182
183 pressed = buttons & ~lastButtons;
184 lastButtons = buttons;
185 if (!pressed)
186     continue;
187
188 /* B and START are shortcuts over the grid; A takes whatever
the cursor is
189    sitting on. */
190 idx = curY * OSK_COLS + curX;
191 if (pressed & JOY_BTN_B_MASK)
192     idx = CELL_DEL;
193 else if (pressed & JOY_START_MASK)
194     idx = CELL_OK;
195 else if (!(pressed & JOY_BTN_A_MASK))
196     continue;
197
198 if (idx == CELL_OK)
199 {
200     if (!len)
201     {
202         soundCursorInvalid();
203         continue;
204     }
205     break;
206 }
207
208 if (idx == CELL_DEL)
209 {
210     if (!len)
211     {
212         soundCursorInvalid();
213         continue;
214     }
215     buffer[--len] = 0;
216     soundCursor();
217 }
218 else
219     appendChar(buffer, &len, max, idx == CELL_SPACE ? ' ' :
cells[idx]);
220
221 drawField(x, y, max, buffer);
222 }
223
224 clearGrid();
225 }
226
227 #endif /* BUILD_NES */
228

```

BATTLESHIP

LISTING 15 SRC/NES/FUJINET.C

```

3 /**
4  * @brief  FujiNet calls - NES (CUSTOM_FUJINET_CALLS, and the
5  * lobby boot)
6  * @author  Thomas Cherryhomes
7  * @email  thom dot cherrymhomes at gmail dot com
8  * @license  gpl v. 3, see LICENSE for details
9  *
10 * Everything that talks to the cartridge through fujinet-lib-
11 experimental is
12 * in this one file, because it has to see the LIBRARY's
13 <fujinet-fuji.h>:
14 * the nes target implements most fuji_* calls as macros over
15 its bus call,
16 * so the prototypes in src/fujinet-fuji.h (which src/misc.h and
17 the platform
18 * headers pull in) link against nothing. This file therefore
19 includes none
20 * of the game's headers, and the shared code reaches the
21 cartridge through
22 * the CUSTOM_FUJINET_CALLS hooks instead.
23 */
24
25 #include <stdbool.h>
26 #include <stdint.h>
27 #include <string.h>
28 #include <stdlib.h>
29 #include <fujinet-fuji.h>
30 #include <fujinet-network.h>
31 #include <fujinet-nes.h>
32 #include "vars.h"
33
34 // graphics.c - declared here rather than through ../platform-
35 specific, which
36 // would bring in src/fujinet-fuji.h
37 void drawText(uint8_t x, uint8_t y, const char *s);
38 void drawSpace(uint8_t x, uint8_t y, uint8_t w);
39
40 // The firmware's WRITE HOST SLOTS expects all 8 slots.
41 #define FUJI_HOST_SLOT_COUNT 8
42 #define LOBBY_DEVICE_SLOT 0
43 #define LOBBY_MODE_READ 1
44
45 int16_t custom_network_call(char *url, uint8_t *buffer, uint16_t
46 max_len)
47 {
48     int16_t read;
49     if (network_open(url, OPEN_MODE_HTTP_GET, OPEN_TRANS_NONE) !
50 = FN_ERR_OK)
51         return -1;
52     read = network_read(url, buffer, max_len);
53     network_close(url);
54     return read;
55 }
56
57 uint16_t custom_read_appkey(uint16_t creator_id, uint8_t app_id,
58 uint8_t key_id, char *destination)
59 {
60     uint16_t read = 0;
61     fuji_set_appkey_details(creator_id, app_id, DEFAULT);
62     if (!fuji_read_appkey(key_id, &read, (uint8_t
63 *)destination))
64         read = 0;
65     return read;
66 }
67
68 void custom_write_appkey(uint16_t creator_id, uint8_t app_id,
69 uint8_t key_id, uint16_t count, char *data)
70 {
71     fuji_set_appkey_details(creator_id, app_id, DEFAULT);
72     fuji_write_appkey(key_id, count, (uint8_t *)data);
73 }
74
75 static void quitStatus(const char *s)
76 {
77     drawSpace(0, HEIGHT - 1, WIDTH);
78     drawText((WIDTH - (unsigned char)strlen(s)) / 2, HEIGHT - 1,
79 s);
80 }
81
82 static bool sameHost(const char *a, const char *b)
83 {
84     while (*a && *b)
85     {
86         if ((*a | 0x20) != (*b | 0x20))
87             return false;
88         a++;
89         b++;
90     }
91     return *a == *b;
92 }
93
94 /* SET_DEVICE_FULLPATH is a fixed 256-byte payload -- a short one
95 is rejected on
96 the ESP32 side -- so the path is padded here rather than at
97 the call site.
98 */
99 static const char lobbyPath[MAX_FILENAME_LEN] = "nes/lobby.nes";
100
101 static const char lobbyHost[] = "ec.tnfs.io";
102 static HostSlot slots[FUJI_HOST_SLOT_COUNT];
103
104 static uint8_t findLobbyHost(void)
105 {
106     uint8_t i;
107     if (!fuji_get_host_slots(slots, FUJI_HOST_SLOT_COUNT))
108         return FUJI_HOST_SLOT_COUNT;
109     for (i = 0; i < FUJI_HOST_SLOT_COUNT; i++)
110         if (sameHost(lobbyHost, (const char *)slots[i]))
111             return i;
112     return FUJI_HOST_SLOT_COUNT;
113 }
114
115 /*
116 The slot is used as found, never created: a player who reached
117 this client
118 through the FujiNet Lobby already has the host, and anyone
119 else is told to
120 add it in CONFIG.
121 */
122 static bool mountLobby(uint8_t slot)
123 {
124     if (!fuji_mount_host_slot(slot))
125         return false;
126 }

```

```

114     if (!fuji_set_device_filename(LOBBY_MODE_READ, slot,
LOBBY_DEVICE_SLOT,
115                                     (char *)lobbyPath))
116         return false;
117     // Only starts the transfer. The image is pushed to the
cartridge
118     // asynchronously, after this has already been answered.
119     return fuji_mount_disk_image(LOBBY_DEVICE_SLOT,
LOBBY_MODE_READ) != 0; ③
120 }
121
122 void quit(void)
123 {
124     uint8_t bootState, slot, pct = 0xFF;
125     char pctText[5];
126
127     quitStatus("LOADING LOBBY...");
128
129     if (!fuji_nes_present())
130     {
131         quitStatus("NO FUJINET CARTRIDGE");
132         return;
133     }
134
135     slot = findLobbyHost();
136     if (slot == FUJI_HOST_SLOT_COUNT)
137     {
138         quitStatus("ADD EC.TNFS.IO IN CONFIG");
139         return;
140     }
141
142     if (!mountLobby(slot))
143     {
144         quitStatus("LOBBY NOT AVAILABLE");
145         return;
146     }
147
148     // Watch the cartridge's own progress counter rather than
guessing a delay.
149     for (;;)
150     {
151         bootState = fuji_nes_boot_state(); ④
152         if (bootState == FUJI_NES_BOOT_READY)
153             break;
154         if (bootState == FUJI_NES_BOOT_FAILED)
155         {
156             quitStatus("LOBBY LOAD FAILED");
157             return;
158         }
159         if (fuji_nes_boot_percent() != pct)
160         {
161             pct = fuji_nes_boot_percent();
162             itoa(pct, pctText, 10);
163             strcat(pctText, "%");
164             drawText(18, HEIGHT - 1, pctText);
165         }
166     }
167
168     // Stop the firmware serving CONFIG at boot, then hand the
console to the
169     // cartridge's loader ROM. It does not return.
170     fuji_set_boot_config(0);
171     fuji_nes_boot(); ⑤

```

```

172 }
173
174 #endif /* BUILD_NES */
175

```

LISTING 16 SRC/NES/UTIL.C

```
1 #ifdef BUILD_NES
2
3 /**
4  * @brief   Utility Functions - NES
5  * @author   Thomas Cherryhomes
6  * @email    thom dot cherryhomes at gmail dot com
7  * @license  gpl v. 3, see LICENSE for details
8  */
9
10 #include <stdbool.h>
11 #include <stdint.h>
12 #include <string.h>
13 #include <stdlib.h>
14 #include <time.h>
15 #include "vars.h"
16 #include "../platform-specific/graphics.h"
17 #include "../platform-specific/util.h"
18
19 // cc65's NMI handler counts frames in clock(); the game timer
20 // is frames since
21 // the last resetTimer().
22 static clock_t timerBase;
23 static bool seeded;
24
25 /*
26  * cc65's own waitvsync() polls PPUSTATUS for the vblank flag.
27  * With NMI on that
28  * races the NMI: a read in the same cycle the flag rises
29  * suppresses the NMI,
30  * and with it that frame's ring-buffer flush. Waiting for the
31  * NMI's frame
32  * count to move instead is race-free, and needs NMI on, which it
33  * always is
34  * outside graphics.c's blitShadow().
35  */
36 void waitvsync(void)
37 {
38     uint8_t t = (uint8_t)clock();
39     while ((uint8_t)clock() == t) 6
40     ;
41 }
42
43 void resetTimer(void)
44 {
45     timerBase = clock();
46 }
47
48 uint16_t getTime(void)
49 {
50     return (uint16_t)(clock() - timerBase);
51 }
52
53 uint8_t getJiffiesPerSecond()
54 {
55     return 60;
56 }
57
58 uint8_t getRandomNumber(uint8_t maxExclusive)
59 {
60     if (!seeded)
61     {
62         // Seeded from the frame count at the first request,
63         // which arrives
64         // only after some human-paced input
65         seeded = true;
66         srand((unsigned int)clock());
67     }
68     return (uint8_t)(rand() % maxExclusive);
69 }
70
71 void housekeeping()
72 {
73     // quit() is in fujinet.c, with the rest of the cartridge calls.
74     #endif /* BUILD_NES */
75 }
```

LISTING 17 SRC/NES/INPUT.C

```

1  #ifndef BUILD_NES
2
3  /**
4   * @brief NES input routines
5   * @author Thomas Cherryhomes
6   * @license gpl v.3
7   */
8
9  #include <joystick.h>
10 #include "vars.h"
11 #include "../platform-specific/input.h"
12 #include "../platform-specific/graphics.h"
13
14 /*
15  * An NES controller is a d-pad and four buttons, read through
16  * cc65's joystick
17  * driver (installed by initGraphics()).
18  * Only the d-pad travels in readJoystick(). The buttons are
19  * synthesized as key
20  * codes behind kbhit()/cgetc() -- the surface the keyboard
21  * platforms present --
22  * the way the ColecoVision port reports its fire buttons and
23  * keypad, so a held
24  * button reports exactly once. A is KEY_RETURN, which the shared
25  * readCommonInput() maps to input.trigger and which also
26  * satisfies the screens
27  * that block on cgetc() ("press any key").
28  * SELECT is a shift key, because the shared screens want more
29  * letter
30  * shortcuts than there are buttons (vars.h's *_TEXT labels say
31  * which):
32  *
33  * A      RETURN   B      'r'      START
34  * ESCAPE  SELECT+A  'h'    SELECT+B  's'    SELECT+START
35  * 'q'    SELECT alone 'n', reported when it is let go without having
36  * shifted
37  * anything -- that is the only way to tell it from the start
38  * of a chord.
39  *
40  * A, B and START report on press.
41  */
42
43 #define DIRS (JOY_UP_MASK | JOY_DOWN_MASK | JOY_LEFT_MASK |
44             JOY_RIGHT_MASK)
45
46 static unsigned char lastButtons;
47 static unsigned char shifted;
48 static unsigned char pendingKey;
49
50 unsigned char readJoystick()
51 {
52     return joy_read(JOY_1) & DIRS;
53 }
54
55 static unsigned char poll(void)
56 {
57     unsigned char buttons = joy_read(JOY_1) & ~DIRS;
58     unsigned char pressed = buttons & ~lastButtons;
59
60     unsigned char released = lastButtons & ~buttons;
61     unsigned char sel = buttons & JOY_SELECT_MASK;
62     lastButtons = buttons;
63
64     if (pressed & JOY_SELECT_MASK)
65         shifted = 0;
66
67     if (pressed & JOY_BTN_A_MASK)
68     {
69         shifted = 1;
70         return sel ? 'h' : KEY_RETURN; 7
71     }
72
73     if (pressed & JOY_BTN_B_MASK)
74     {
75         shifted = 1;
76         return sel ? 's' : 'r';
77     }
78
79     if (pressed & JOY_START_MASK)
80     {
81         shifted = 1;
82         return sel ? 'q' : KEY_ESCAPE;
83     }
84
85     if ((released & JOY_SELECT_MASK) && !shifted)
86         return 'n';
87
88     return 0;
89 }
90
91 unsigned char kbhit(void)
92 {
93     if (!pendingKey)
94         pendingKey = poll();
95
96     return pendingKey != 0;
97 }
98
99 unsigned char cgetc(void)
100 {
101     unsigned char k;
102     while (!kbhit())
103         waitvsync();
104
105     k = pendingKey;
106     pendingKey = 0;
107     return k;
108 }
109
110 #endif /* BUILD_NES */

```

```

1  #ifndef BUILD_NES
2
3  /**
4   * @brief NES on-screen keyboard
5   * @author Thomas Cherryhomes
6   * @email thom dot cherrymhomes at gmail dot com
7   * @license gpl v. 3, see LICENSE for details
8   */
9
10 #include <joystick.h>
11 #include <time.h>
12 #include "vars.h"
13 #include "../platform-specific/graphics.h"
14 #include "../platform-specific/sound.h"
15
16 /*
17  An NES controller has no keys at all, so the player's name is
18  typed on screen. The grid is fujinet-battleship's src/nas/osk.c (itself
19  the ColecoVision/5 Card Stud/CONFIG grid), but here it is a KEY
20  SOURCE rather than an editor: while it is open, input.c's kbhit() asks
21  oskPoll() for a key, and the shared inputFieldCycle() does the editing, the
22  cursor chip and the minimum-length check exactly as it does on a keyboard
23  machine.
24  A types the cell under the cursor, 8 backspaces and START
25  accepts (as does the OK cell). The cursor position IS the character, so there
26  is no shift key and no paging. There is no cancel: the name screen only
27  opens when a name is required.
28  */
29
30 #define OSK_COLS 10
31 #define OSK_ROWS 4
32 #define OSK_PITCH 3 /* marker column + two label columns */
33 #define OSK_X 1
34 #define OSK_Y 17 /* below the "at least 2 letters" hint at
35 row 16 */
36 #define OSK_ROW_STEP 2 /* a blank line between rows, or the grid
37 reads as columns */
38
39 /* The last row is one short: its tenth cell does not exist. */
40 #define OSK_LAST_ROW_COLS 9
41
42 #define CELL_SPACE (OSK_COLS * 3 + 6) /* index 36 */
43 #define CELL_DEL (CELL_SPACE + 1)
44 #define CELL_OK (CELL_SPACE + 2)
45
46 static const char cells[] =
47 "ABCDEFHIJ"
48 "KLMNOPQRST"
49 "UVWXYZ0123"
50
51 "456789";
52
53 #define DIRS (JOY_UP_MASK | JOY_DOWN_MASK | JOY_LEFT_MASK |
54 JOY_RIGHT_MASK)
55
56 #define REPEAT_FIRST 18 /* frames before a held stick repeats */
57 #define REPEAT_NEXT 5
58
59 unsigned char oskActive;
60
61 static unsigned char curX, curY, hold, lastDir, lastButtons,
62 lastFrame;
63
64 static unsigned char cellCols(unsigned char row)
65 {
66     return row == OSK_ROWS - 1 ? OSK_LAST_ROW_COLS : OSK_COLS;
67 }
68
69 static void drawCell(unsigned char col, unsigned char row,
70 unsigned char marked)
71 {
72     unsigned char idx = row * OSK_COLS + col;
73     unsigned char x = OSK_X + col * OSK_PITCH;
74     char glyph[3];
75
76     if (marked)
77         drawIcon(x, OSK_Y + row * OSK_ROW_STEP, ICON_MARK);
78     else
79         drawBlank(x, OSK_Y + row * OSK_ROW_STEP);
80
81     if (idx == CELL_SPACE)
82         drawTextAlt(x + 1, OSK_Y + row * OSK_ROW_STEP, "sp");
83     else if (idx == CELL_DEL)
84         drawTextAlt(x + 1, OSK_Y + row * OSK_ROW_STEP, "dl");
85     else if (idx == CELL_OK)
86         drawTextAlt(x + 1, OSK_Y + row * OSK_ROW_STEP, "ok");
87     else
88     {
89         glyph[0] = cells[idx];
90         glyph[1] = ' ';
91         glyph[2] = 0;
92         drawText(x + 1, OSK_Y + row * OSK_ROW_STEP, glyph);
93     }
94 }
95
96 void oskOpen(void)
97 {
98     unsigned char row, col;
99
100     curX = curY = 0;
101     for (row = 0; row < OSK_ROWS; row++)
102         for (col = 0; col < cellCols(row); col++)
103             drawCell(col, row, row == curY && col == curX);
104
105     drawTextAlt(WIDTH / 2 - 12, HEIGHT - 1, "A:TYPE B:DELETE
106 START:OK");
107
108     // Whatever is already held (the A that opened this screen)
109     is not a press.

```

```

102     lastButtons = joy_read(JOY_1) & ~DIRS;
103     lastDir = joy_read(JOY_1) & DIRS;
104     hold = REPEAT_FIRST;
105     lastFrame = (unsigned char)clock();
106     oskActive = 1;
107 }
108
109 void oskClose(void)
110 {
111     unsigned char row;
112
113     oskActive = 0;
114     for (row = 0; row < OSK_ROWS; row++)
115         drawSpace(0, OSK_Y + row * OSK_ROW_STEP, WIDTH);
116     drawSpace(0, HEIGHT - 1, WIDTH);
117 }
118
119 /* One key for inputFieldCycle(), or 0. inputFieldCycle() spins
120 on kbhit()
121 without waiting for vblank, so the pad is read once a frame
122 here: that is
123 what auto-repeat and the button edges are counted in. */
124 unsigned char oskPoll(void) 1
125 {
126     unsigned char pad, dir, move, pressed, idx;
127
128     if ((unsigned char)clock() == lastFrame)
129         return 0;
130     lastFrame = (unsigned char)clock();
131
132     pad = joy_read(JOY_1);
133     dir = pad & DIRS;
134     pressed = pad & ~DIRS & ~lastButtons;
135     lastButtons = pad & ~DIRS;
136
137     move = 0;
138     if (dir != lastDir)
139     {
140         lastDir = dir;
141         hold = REPEAT_FIRST;
142         move = dir;
143     }
144     else if (dir && --hold == 0)
145     {
146         hold = REPEAT_NEXT;
147         move = dir;
148     }
149
150     if (move)
151     {
152         drawCell(curX, curY, 0);
153
154         if ((move & JOY_LEFT_MASK) && curX)
155             curX--;
156         else if ((move & JOY_RIGHT_MASK) && curX + 1 <
157             cellCols(curY))
158             curX++;
159         else if ((move & JOY_UP_MASK) && curY)
160             curY--;
161         else if ((move & JOY_DOWN_MASK) && curY + 1 < OSK_ROWS)
162             curY++;
163
164         if (curX >= cellCols(curY))
165             curX = cellCols(curY) - 1;
166
167         if (curY >= OSK_ROWS)
168             curY = OSK_ROWS - 1;
169
170         drawCell(curX, curY, 1);
171     }
172
173     return move;
174 }

```

```

164     drawCell(curX, curY, 1);
165     soundCursor();
166 }
167
168 /* A takes whatever the cursor is sitting on; B and START
169 are shortcuts
170 for the backspace and OK cells. */
171 if (pressed & JOY_BTN_B_MASK)
172     idx = CELL_DEL;
173 else if (pressed & JOY_START_MASK)
174     idx = CELL_OK;
175 else if (pressed & JOY_BTN_A_MASK)
176     idx = curY * OSK_COLS + curX;
177 else
178     return 0;
179
180 if (idx == CELL_OK)
181     return KEY_RETURN;
182 if (idx == CELL_DEL)
183     return KEY_BACKSPACE;
184
185 soundCursor();
186 return idx == CELL_SPACE ? KEY_SPACEBAR : cells[idx];
187 }
188 #endif /* BUILD_NES */
189

```

LISTING 19 SRC/NES/INPUT.C

```
1 #ifdef BUILD_NES
2
3 /**
4  * @brief NES input routines
5  * @author Thomas Cherryhomes
6  * @license gpl v.3
7  */
8
9 #include <joystick.h>
10 #include "vars.h"
11 #include "../platform-specific/input.h"
12 #include "../platform-specific/graphics.h"
13 #include "../misc.h"
14
15 /*
16  * An NES controller is a d-pad and four buttons, read through
17  * cc65's joystick driver (installed by initGraphics()).
18  *
19  * Only the d-pad travels in readJoystick(). The buttons are
20  * synthesized as key codes behind kbhit()/cgetc() -- the surface the keyboard
21  * platforms present -- so a held button reports exactly once. A is KEY_RETURN,
22  * which the shared readCommonInput() maps to input.trigger and which also
23  * satisfies the screens that block on cgetc() ("press any key").
24  *
25  * SELECT is a shift key, because the shared screens want more
26  * letter shortcuts than there are buttons (vars.h's *TEXT labels say
27  * which):
28  *
29  * A RETURN B 'h' help START
30  * ESCAPE SELECT+B 's' SELECT+START 'q' quit
31  * SELECT alone 'r' refresh, reported when it is let go without
32  * having shifted anything -- the only way to tell it
33  * from a chord.
34  *
35  * A, B and START report on press. While the on-screen keyboard
36  * is open (osk.c) it owns the pad, and kbhit() reports what it types
37  * instead.
38  */
39 #define DIRS (JOY_UP_MASK | JOY_DOWN_MASK | JOY_LEFT_MASK | JOY_RIGHT_MASK)
40
41 extern unsigned char oskActive; /* osk.c */
42 unsigned char oskPoll(void); /* osk.c */
43
44 static unsigned char lastButtons;
45 static unsigned char shifted;
46 static unsigned char pendingKey;
47
48 unsigned char readJoystick()
49 {
50     return oskActive ? 0 : joy_read(JOY_1) & DIRS;
51 }
52
53 static unsigned char poll(void)
54 {
55     unsigned char buttons = joy_read(JOY_1) & ~DIRS;
56     unsigned char pressed = buttons & ~lastButtons;
57     unsigned char released = lastButtons & ~buttons;
58     unsigned char sel = buttons & JOY_SELECT_MASK;
59
60     lastButtons = buttons;
61
62     if (pressed & JOY_SELECT_MASK)
63         shifted = 0;
64
65     if (pressed & JOY_BTN_A_MASK)
66     {
67         shifted = 1;
68         return KEY_RETURN;
69     }
70
71     if (pressed & JOY_BTN_B_MASK)
72     {
73         shifted = 1;
74         return sel ? 's' : 'h';
75     }
76
77     if (pressed & JOY_START_MASK)
78     {
79         shifted = 1;
80         return sel ? 'q' : KEY_ESCAPE;
81     }
82
83     if ((released & JOY_SELECT_MASK) && !shifted)
84         return 'r';
85
86     return 0;
87 }
88
89 unsigned char kbhit(void)
90 {
91     if (!pendingKey)
92     {
93         if (oskActive)
94         {
95             pendingKey = oskPoll();
96             // Whatever the grid used up is not a press once it
97             closes.
98             lastButtons = joy_read(JOY_1) & ~DIRS;
99             shifted = 1;
100         }
101         else
102             pendingKey = poll();
103     }
104     return pendingKey != 0;
105 }
106
107 char cgetc(void)
108 {
109     unsigned char k;
110     while (!kbhit())
111         waitvsync();
112     k = pendingKey;
113 }
```

```

110     pendingKey = 0;
111     return k;
112 }
113
114 #endif /* BUILD_NES */
115

```

LISTING 20 SRC/NES/UTIL.C

```

1 #ifdef BUILD_NES
2
3 /**
4  * @brief   Utility Functions - NES
5  * @author  Thomas Cherryhomes
6  * @email   thom dot cherryhomes at gmail dot com
7  * @license  gpl v. 3, see LICENSE for details
8  */
9
10 #include <stdbool.h>
11 #include <stdint.h>
12 #include <time.h>
13 #include "../misc.h"
14 #include "../platform-specific/graphics.h"
15 #include "../platform-specific/util.h"
16
17 /* The server sends the packed wire layout straight into clientState.game,
18  * so cc65 must not pad Game. Fails the build if it ever does. */
19 typedef char game_wire_size_check[(sizeof(Game) == 599) ? 1 : -1]; ❸
20
21 // cc65's NMI handler counts frames in clock(); the game timer is frames since
22 // the last resetTimer().
23 static clock_t timerBase;
24
25 /*
26  * cc65's own waitvsync() polls PPUSTATUS for the vblank flag. With NMI on that
27  * races the NMI: a read in the same cycle the flag rises suppresses the NMI,
28  * and with it that frame's ring-buffer flush. Waiting for the NMI's frame
29  * count to move instead is race-free, and needs NMI on, which it always is
30  * outside graphics.c's whole-screen copies.
31  */
32 void waitvsync(void)
33 {
34     uint8_t t = (uint8_t)clock();
35     while ((uint8_t)clock() == t)
36         ;
37 }
38
39 void resetTimer(void)
40 {
41     timerBase = clock();
42 }
43
44 uint16_t getTime(void)
45 {
46     return (uint16_t)(clock() - timerBase);
47 }
48
49 uint8_t getJiffiesPerSecond()
50 {
51     return 60;
52 }
53
54

```

```

55 void housekeeping()
56 {
57 }
58
59 // quit() is in fujinet.c, with the rest of the cartridge calls.
60
61 #endif /* BUILD_NES */
62

```

TEXAS HOLD'EM

LISTING 21 SRC/MAIN.C

```

1  /**
2   * @brief 5-card-stud
3   * @author Eric Carr, Thomas Cherryhomes, (insert names here)
4   * @license gpl v. 3
5   * @verbose main
6   */
7
8  #ifndef _CMOC_VERSION_
9  #include "coco/coco_bool.h"
10 #else
11 #include <stdlib.h>
12 #include <stdint.h>
13 #include <stdbool.h>
14 #include <string.h>
15 #endif /* _CMOC_VERSION_ */
16
17 #include "platform-specific/graphics.h"
18 #include "platform-specific/util.h"
19 #include "platform-specific/input.h"
20 #include "misc.h"
21 #include "platform-specific/network.h"
22 #include "platform-specific/appkey.h"
23 #include "platform-specific/sound.h"
24
25 #include "stateclient.h"
26 #include "gamelogic.h"
27 #include "screens.h"
28
29 // Store default server endpoint in case lobby did not set app
30 key
31 char serverEndpoint[50] = "https://th.carr-designs.com/";
32 //char serverEndpoint[50] = "http://127.0.0.1:8080/"; // local
33 // "N: for apple, but not C64"
34
35 // Empty at boot so the table selection screen is always shown;
36 // the room list
37 // comes from the server's /tables endpoint
38 char query[QUERY_LEN] = "";
39 char playerName[12] = "";
40
41 //GameState state;
42 // On the ColecoVision this is a macro for the cartridge's reply
43 // window, not an
44 // object -- see src/misc.h.
45 #ifndef BUILD_COLECO
46 ClientState clientState;
47 #endif
48
49 // State helper vars
50 #ifndef __WATCOMC__
51 int inputKey;
52 #else
53 char inputKey;
54 #endif
55 unsigned char prevPlayerCount, prevRound, currentCard,
56 cardIndex, cursorX, cursorY, waitCount, wasViewing;
57 signed char inputDirX, inputDirY;
58 uint16_t prevPot, maxJifs;
59 bool noAnim, doAnim, finalFlip, inputTrigger;
60
61 unsigned char playerX[8], playerY[8], moveLoc[5];
62 signed char playerBetX[8], playerBetY[8], playerDir[8];
63
64 // Common local scope temp variables
65 unsigned char h, i, j, k, x, y, xx;
66 char tempBuffer[TEMP_BUFFER_LEN];
67
68 char prefs[4];
69
70 char *hand, *requestedMove;
71
72 #ifndef _CMOC_VERSION_
73 int main(void)
74 #else
75 void main(void)
76 #endif /* _CMOC_VERSION_ */
77 {
78     // uint8_t x, result;
79     // x=0;
80     // x=x+2;
81
82     // unsigned char i; while (1) { if (kbhit()) { inputKey = cgetc();
83     printf("\nKEY: %u", inputKey); for(i=0; i<10; i++)
84     { pause(2); printf("."); } }
85     loadPrefs();
86     initGraphics();
87     initSound();
88
89     // Checking MSX C compiler bug
90     // x=10;
91     // result=(x>5)+10;
92     // itoa(result, tempBuffer, 10);
93
94     // cputs(tempBuffer);
95     // cgetc();
96

```

```

92 #ifdef USE_PLATFORM_SPECIFIC_INPUT
93     initPlatformKeyboardInput();
94 #endif
95
96     showWelcomeScreen();
97     showTableSelectionScreen();
98
99     // Main in-game loop
100     while (true) {
101
102         // Get latest state and draw on screen, then prompt player
103         for move if their turn
104         if (getStateFromServer()) {
105             #if defined(BUILD_COLECO) || defined(BUILD_NES)
106                 /*
107                  * The ColecoVision has no back buffer (1K of RAM), so
108                  * showGameScreen()
109                  * repaints the table straight to VRAM. Every server poll
110                  * would otherwise
111                  * redraw the whole table even when nothing changed --
112                  * invisible on a real
113                  * display (identical bytes rewritten in place) but a
114                  * waste, and it makes a
115                  * redraw always be in flight, which tears on capture. Skip
116                  * the repaint
117                  * when the fetched state is byte-for-byte what is already
118                  * on screen.
119
120                  state is the reply window here (see src/misc.h), valid
121                  until the next
122                  fujit.* call -- which is exactly now, right after the
123                  fetch. A one-word
124                  sum is enough: a rare collision just defers a repaint to
125                  the next
126                  differing poll. requestPlayerMove() still runs, so our
127                  own turn (where
128                  the countdown does change the state every second) always
129                  draws.
130
131                  The NES draws through cc65's vblank ring buffer, a few
132                  dozen tiles a
133                  frame, so an unchanged repaint there costs a backlog the
134                  next real
135                  change has to queue behind. Its state is an ordinary
136                  object in WRAM.
137                  */
138                  {
139                      static unsigned int lastDrawnSum = 1; // != any first
140                      real sum
141                      unsigned int sum = 0;
142                      unsigned int n; // Game is 429
143                      bytes: not a char
144                      unsigned char *sp = &clientState.firstByte;
145                      for (n = 0; n < sizeof(Game); n++)
146                          sum += sp[n] ^ (unsigned char)n; // position-
147                      weighted vs byte swaps 1
148
149                      if (sum != lastDrawnSum) {
150                          lastDrawnSum = sum;
151                          showGameScreen();
152                      }
153                  }
154
155                 #else
156                     showGameScreen();
157                 #endif
158
159                 requestPlayerMove();
160             }
161         }
162     }

```

```

139     } else {
140         // Wait a bit to avoid hammering the server if getting
141         bad responses
142         pause(30);
143     }
144     // Handle other key presses
145     readCommonInput();
146
147     switch(inputKey) {
148         case KEY_ESCAPE: // Esc
149         case KEY_ESCAPE_ALT: // Esc Alt
150             showInGameMenuScreen();
151             break;
152     }
153
154     #ifdef _CMOC_VERSION_
155     }
156     #endif
157     return 0;
158 #endif
159 }
160

```

APPENDIX D. NETCAT

A terminal for the NES: type a URL, connect, and talk to whatever is at the other end – a bulletin board, a chat server, a program of your own.



Fig. 21 – Entering a URL; connected to a test server that answers each line; and typing the next line over the bottom of the pane.

USING IT

1. **Connect:** The first screen asks for a URL, with the last one already filled in – N:TCP://BBS.FOZZTEXX.COM:23/ to begin with. Edit it on the on-screen keyboard and press Start.
2. **Read:** Whatever the other end sends scrolls up the screen. Telnet's option bytes and ANSI colour codes are thrown away.
3. **Write:** Press Start to type a line. The keyboard covers the bottom of the screen; Start again sends the line with CR LF, and the pane comes back. With a Family BASIC or Subor keyboard plugged in, just type: every key goes straight to the other end, and Return sends CR LF.
4. **Hang up:** Select. Then A takes you back to the URL.

On the on-screen keyboard the pad moves, A presses, B deletes, Select is SHIFT – lower case – and Start is DONE.

HOW IT WORKS

Four small files:

File	Lines	Does
netcat.c	224	The two screens and the session loop.
term.c	83	The pane: a shadow copy in RAM, scrolled with memmove, drawn a row at a time when it changes.
osk.c	111	The on-screen keyboard.
pad.c	49	The controller, with auto-repeat, and frame().

The session loop, once a frame:

- STATUS:** how many bytes are waiting? If some, READ up to 128 of them and feed them through the Telnet and ANSI filter into the pane, then send the rows that changed to the screen. If none, and the connection is gone, hang up.
- Read the pad. Select hangs up; Start opens the keyboard.
- If a keyboard is plugged in, send what was typed.

NETCAT.C: THE SESSION LOOP

```
/* What has the other end sent? STATUS says how much is waiting. */
if (network_status(url, &avail, &connected, &err) != FN_ERR_OK)
    break;
if (avail) {
    n = network_read_nb(url, buf, avail < sizeof buf ? avail : sizeof buf);
    for (i = 0; n > 0 && i < (uint8_t) n; i++)
        receive(buf[i]);
    term_flush();
} else if (!connected)
    break;

pad = pad_poll();
if (pad & PAD_SELECT)
    break;

c = kbd ? fuji_nes_kbd_getc() : 0;
if (c == FUJI_NES_KEY_ENTER)
    send("\r\n", 2);
else if (c >= ' ' && c <= '~')
    send(&c, 1);
```

cc65's NMI resets the scroll every frame, so the pane cannot be scrolled by the PPU. Every character goes into a 24×30 shadow copy instead; a new line at the bottom moves the copy up with memmove and marks every row as changed. term_flush() then sends only the marked rows, through cc65's queue. A burst of text that scrolls the pane ten times costs one redraw, not ten.

The keyboard, when it is open, draws over the bottom of the pane. The shadow still holds what was there, so term_repaint() puts it back afterwards.

BUILDING IT

BUILD AND RUN

```
make -C listings netcat          # listings/build/netcat.nes
emu/run.sh listings/build/netcat.nes # or open it in FujiNet Go NES Desktop
```

For testing, emu/echo.py is a server that answers every line; build netcat pointed at it with -DDEFAULT_URL='\"N:TCP://127.0.0.1:7777/\"', as was done for the pictures here.

LISTINGS

LISTING 22 NETCAT.C

```
1 /* netcat.c -- a network terminal for the NES.
2  *
3  * Asks for a URL (N:TCP://host:port/ or N:TELNET://...),
4  * connects, and then
5  * shows whatever the other end sends. Press Start to type a
6  * line on the
7  * on-screen keyboard -- Start again sends it -- or just type,
8  * if a Family
9  * BASIC or Subor keyboard is plugged in. Select hangs up. */
10
11 #include <conio.h>
12 #include <string.h>
13 #include <fujinet-network.h>
14 #include <fujinet-nes.h>
15 #include "pad.h"
16 #include "osk.h"
17 #include "term.h"
18
19 #ifndef DEFAULT_URL
20 #define DEFAULT_URL "N:TCP://BBS.FOZZTEXX.COM:23/"
21 #endif
22
23 #define URL_MAX 120
24 #define LINE_MAX 60
25 #define OSK_TOP 17
26 /* conio row of the
27 keyboard */
28 #define EDIT_ROW 15
29 /* conio row of the line
30 being typed */
31
32 static char url[URL_MAX + 1] = DEFAULT_URL;
33 static char line[LINE_MAX + 3];
34 static uint8_t buf[128];
35 static uint8_t kbd; /* FUJI_NES_KBD_* */
36
37 /* ---- small screen helpers ---- */
38
39 static void blank(uint8_t y, uint8_t n)
40 {
41     while (n--)
42         cclearxy(0, y++, 32);
43 }
44
45 static void title(const char *s)
46 {
47     revers(1);
48     cclearxy(0, 0, 32);
49     cputsxy(1, 0, s);
50     revers(0);
51 }
52
53 static void help(const char *s)
54 {
55     blank(27, 1);
56     cputsxy(1, 27, s);
57 }
58
59 /* Show the last 'rows' x 30 characters of s, so the end of a
60 long string --
```

```

53 where the typing happens -- is always in view. */
54 static void show_field(uint8_t y, uint8_t rows, const char *s)
55 {
56     uint8_t len = strlen(s), skip = 0, i;
57
58     if (len >= rows * 30)
59         skip = len - rows * 30 + 1;
60     blank(y, rows);
61     gotoxy(1, y);
62     for (i = skip; i < len; i++) {
63         if ((i - skip) % 30 == 0)
64             gotoxy(1, y + (i - skip) / 30);
65         cputc(s[i]);
66     }
67     revers(1);
68     cputc(' ');
69     revers(0);
70 }
71
72 /* Edit s in place with the joy pad keyboard (or a real one).
73 Returns when
74 DONE is pressed. */
75 static void edit(char *s, uint8_t max, uint8_t y, uint8_t rows)
76 {
77     char c;
78     uint8_t len;
79     show_field(y, rows, s);
80     for (;;) {
81         frame();
82         c = osk_input(pad_poll());
83         if (!c && kbd)
84             c = fuji_nes_kbd_getc();
85         if (!c)
86             continue;
87         len = strlen(s);
88         if (c == OSK_DONE || c == FUJI_NES_KEY_ENTER)
89             return;
90         if (c == OSK_DEL) {
91             if (len)
92                 s[len - 1] = '\0';
93         } else if (c >= ' ' && c <= '~' && len < max) {
94             s[len] = c;
95             s[len + 1] = '\0';
96         } else
97             continue;
98         show_field(y, rows, s);
99     }
100 }
101
102 /* ---- the two screens ---- */
103
104 static void ask_url(void)
105 {
106     clrscr();
107     title("NETCAT");
108     cputsxy(1, 3, "CONNECT TO:");
109     osk_draw(OSK_TOP);
110     help("A KEY B DEL SEL SHIFT ST GO");
111     edit(url, URL_MAX, 5, 4);
112 }
113
114 /* Telnet servers open with IAC option negotiation, and BBSes
115 send ANSI
116 colour codes. Neither means anything here: swallow them. */
117 static uint8_t skip_iac, in_esc;

```

```

117 static void receive(uint8_t c)
118 {
119     if (skip_iac) {
120         and opt */
121         --skip_iac;
122         return;
123     }
124     if (c == 0xFF) { ④
125         skip_iac = 2;
126         return;
127     }
128     if (in_esc) {
129         /* ESC [ params letter
130         if ((c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z'))
131             in_esc = 0;
132         return;
133     }
134     if (c == 0x1B) {
135         in_esc = 1;
136         return;
137     }
138     term_putc(c);
139 }
140
141 static void send(const char *s, uint16_t n)
142 {
143     network_write(url, (uint8_t *) s, n);
144 }
145
146 static void session(void)
147 {
148     uint16_t avail;
149     uint8_t connected, err, pad, i;
150     int16_t n;
151     char c;
152
153     clrscr();
154     title(url);
155     term_clear();
156     term_flush();
157     help("CONNECTING...");
158     if (network_open(url, OPEN_MODE_RW, OPEN_TRANS_NONE) !=
159         FN_ERR_OK) { ①
160         help("CANNOT CONNECT. PRESS A.");
161         while (!pad_poll() & PAD_A)
162             frame();
163         return;
164     }
165     help("ST TYPE A LINE SEL HANG UP");
166     skip_iac = in_esc = 0;
167
168     for (;;) {
169         frame();
170
171         /* What has the other end sent? STATUS says how much is
172         waiting. */
173         if (network_status(url, &avail, &connected, &err) !=
174             FN_ERR_OK) ②
175             break;
176         if (avail) {

```

```

174     n = network_read_nb(url, buf, avail < sizeof buf ? avail :
sizeof buf); ③
175     for (i = 0; n > 0 && i < (uint8_t) n; i++)
176         receive(buf[i]);
177     term_flush();
178 } else if (!connected)
179     break;
180
181 pad = pad_poll();
182 if (pad & PAD_SELECT)
183     break;
184
185 c = kbd ? fuji_nes_kbd_getc() : 0;
186 if (c == FUJI_NES_KEY_ENTER)
187     send("\r\n", 2);
188 else if (c >= ' ' && c <= '~')
189     send(&c, 1);
190
191 if (pad & PAD_START) {                /* compose a line */
192     line[0] = '\0';
193     blank(EDIT_ROW, 27 - EDIT_ROW); /* the keyboard covers
the pane */
194     osk_draw(OSK_TOP);
195     help("ST SEND");
196     edit(line, LINE_MAX, EDIT_ROW, 1);
197     strcat(line, "\r\n");
198     send(line, strlen(line));
199     term_repaint();                /* back from the shadow
copy */ ⑤
200     help("ST TYPE A LINE  SEL HANG UP");
201 }
202 }
203
204 network_close(url);
205 help("DISCONNECTED. PRESS A.");
206 while (!(pad_poll() & PAD_A))
207     frame();
208 }
209
210 void main(void)
211 {
212     if (!fuji_nes_present()) {
213         clrscr();
214         cputs("NO FUJINET CARTRIDGE");
215         for (;;) ;
216     }
217     network_init();
218     kbd = fuji_nes_kbd_detect();
219
220     for (;;) {
221         ask_url();
222         session();
223     }
224 }
225

```

LISTING 23 TERM.C

```

1 /* term.c -- the terminal pane.
2 *

```

```

3 * cc65's NMI handler puts the scroll back to 0,0 every frame,
  so the pane

```

```

4  * cannot be scrolled by the PPU. Instead every character goes
5  into a shadow
6  * copy of the pane in RAM, scrolling is a memmove of that copy,
7  and
8  * term_flush() sends the rows that changed to the screen
9  through conio,
10 * whose writes are queued for the vblank NMI. A burst of text
11 costs one
12 * redraw, not one per line. */
13
14 #include <conio.h>
15 #include <string.h>
16 #include "term.h"
17
18 static char shadow[TERM_ROWS][TERM_COLS];
19 static uint8_t dirty[TERM_ROWS];
20 static uint8_t col, row;
21
22 void term_clear(void)
23 {
24     memset(shadow, ' ', sizeof shadow);
25     memset(dirty, 1, sizeof dirty);
26     col = row = 0;
27 }
28
29 static void scroll(void)
30 {
31     memmove(shadow[0], shadow[1], (TERM_ROWS - 1) * TERM_COLS); 6
32     memset(shadow[TERM_ROWS - 1], ' ', TERM_COLS);
33     memset(dirty, 1, sizeof dirty);
34 }
35
36 static void newline(void)
37 {
38     col = 0;
39     if (row == TERM_ROWS - 1)
40         scroll();
41     else
42         ++row;
43 }
44
45 void term_putc(char c)
46 {
47     if (c == '\r') {
48         col = 0;
49         return;
50     }
51     if (c == '\n') {
52         newline();
53         return;
54     }
55     if (c == '\b') {
56         if (col)
57             --col;
58         return;
59     }
60     if (c < 0x20 || c > 0x7E)
61         /* nothing else has a
62         glyph */
63         return;
64     if (col == TERM_COLS)
65         newline();
66     shadow[row][col++] = c;
67     dirty[row] = 1;
68 }
69
70 void term_flush(void)
71 {
72     uint8_t r, c;
73     for (r = 0; r < TERM_ROWS; r++) {
74         if (!dirty[r])
75             continue;
76         gotoxy(TERM_LEFT, TERM_TOP + r);
77         for (c = 0; c < TERM_COLS; c++)
78             cputc(shadow[r][c]);
79         /* queued; written in
80         the next vblank */ 7
81     }
82     dirty[r] = 0;
83 }
84
85 void term_repaint(void)
86 {
87     memset(dirty, 1, sizeof dirty);
88     term_flush();
89 }

```

LISTING 24 OSK.C

```

1  /* osk.c -- an on-screen keyboard.
2  *
3  * Thirteen keys across, four rows of characters and a row of
4  * actions,
5  * double-spaced because the font has no gap between rows. The
6  * D-pad moves the highlight, A presses the key under it; B is a
7  * shortcut for
8  * DEL, Start for DONE and Select for SHIFT. */
9
10 #include <conio.h>
11 #include "osk.h"
12 #include "pad.h"
13
14 #define COLS 13
15 #define LEFT 3                /* screen column of the
16 first key */
17
18 static const char keys[4][COLS + 1] = {
19     "ABCDEFGHJKLMN",
20     "NOPQRSTUVWXYZ",
21     "0123456789.:/;",
22     "-_?=&@#%+*,!";
23 };
24
25 /* The action row: where each label starts and which key columns
26 pick it. */
27 static const char *const label[4] = { "SHIFT", "SPACE", "DEL",
28 "DONE" };
29
30 static const uint8_t lx[4] = { 3, 10, 17, 23 };
31 static const uint8_t first[4] = { 0, 3, 7, 10 }; /* key column
32 -> label */
33
34 static uint8_t top, kx, ky, shift;
35
36 static uint8_t action(uint8_t x)
37 {
38     uint8_t a = 3;
39     while (x < first[a])
40         --a;
41     return a;
42 }
43
44 static char keychar(uint8_t y, uint8_t x)
45 {
46     char c = keys[y][x];
47     if (shift && y < 2)
48         c += 'A' - 'A';
49     return c;
50 }
51
52 /* Draw one key, highlighted or not. */
53 static void key(uint8_t y, uint8_t x, uint8_t on)
54 {
55     revers(on);
56     if (y < 4)
57         cputcxy(LEFT + 2 * x, top + 2 * y, keychar(y, x));
58     else
59         cputsxy(lx[action(x)], top + 8, label[action(x)]);
60     revers(0);
61 }
62
63 static void letters(void)
64 {
65     uint8_t y, x;
66     for (y = 0; y < 2; y++)
67         for (x = 0; x < COLS; x++)
68             key(y, x, y == ky && x == kx);
69 }
70
71 void osk_draw(uint8_t t)
72 {
73     uint8_t y, x;
74     top = t;
75     for (y = 0; y < 4; y++)
76         for (x = 0; x < COLS; x++)
77             key(y, x, 0);
78     for (x = 0; x < 4; x++)
79         cputsxy(lx[x], top + 8, label[x]);
80     key(ky, kx, 1);
81 }
82
83 static char press(void)
84 {
85     if (ky < 4)
86         return keychar(ky, kx);
87     switch (action(kx)) {
88     case 0: shift ^= 1; letters(); return 0;
89     case 1: return ' ';
90     case 2: return OSK_DEL;
91     default: return OSK_DONE;
92     }
93 }
94
95 char osk_input(uint8_t pad)
96 {
97     uint8_t ox = kx, oy = ky;
98     if (pad & PAD_LEFT) kx = kx ? kx - 1 : COLS - 1;
99     if (pad & PAD_RIGHT) kx = kx < COLS - 1 ? kx + 1 : 0;
100     if (pad & PAD_UP) ky = ky ? ky - 1 : 4;
101     if (pad & PAD_DOWN) ky = ky < 4 ? ky + 1 : 0;
102     if (kx != ox || ky != oy) {
103         key(oy, ox, 0); /* also clears a whole
104         action label */
105         key(ky, kx, 1);
106     }
107     if (pad & PAD_A) return press();
108     if (pad & PAD_B) return OSK_DEL;
109     if (pad & PAD_START) return OSK_DONE;
110     if (pad & PAD_SELECT) { shift ^= 1; letters(); }
111     return 0;
112 }

```

LISTING 25 PAD.C

```
1 /* pad.c -- controller 1, with auto-repeat on the D-pad.
2 *
3 * $4016 is the controller port, not the mailbox, so the usual rules about
4 * read-modify-write instructions do not apply here. */
5
6 #include <time.h>
7 #include "pad.h"
8
9 #define JOY1 (*(volatile uint8_t *) 0x4016)
10
11 #define REPEAT_DELAY 18 /* frames before a held arrow repeats */
12 #define REPEAT_RATE 5 /* frames between repeats */
13
14 static uint8_t held, timer;
15
16 static uint8_t read_pad(void)
17 {
18     uint8_t i, b = 0;
19
20     JOY1 = 1; /* latch the buttons ... */ 8
21     JOY1 = 0; /* ... and shift them out */
22     for (i = 0; i < 8; i++)
23         b = (b << 1) | (JOY1 & 1); /* A, B, Select, Start, Up ... Right */
24     return b;
25 }
26
27 uint8_t pad_poll(void)
28 {
29     uint8_t now = read_pad();
30     uint8_t down = now & ~held;
31     uint8_t arrows = now & 0x0F;
32
33     if (down)
34         timer = REPEAT_DELAY;
35     else if (arrows && --timer == 0) {
36         timer = REPEAT_RATE;
37         down |= arrows;
38     }
39     held = now;
40     return down;
41 }
42
43 void frame(void)
44 {
45     clock_t t = clock(); /* cc65's NMI counts frames here */
46
47     while (clock() == t) 9
48         ;
49 }
50
```

FUJINET NES CARTRIDGE PARTS LIST

The parts of the Rev0 design, from its schematic. No board has yet been made from it; the list is here so that you know what your program is talking to.

Part No.	Part Name	What it does	Qty.
RP2354B	Microcontroller, 2 MB flash	The mailbox, the loader ROM, the work RAM, the bank tables, the USB link	1
AS6C4008-55	512K × 8 static RAM, 55 ns	PRG and CHR, in place of ROM	2
74HCT595	Shift register	SRAM enable, write gates, mirroring, LED	1
74HCT253	Dual 4-to-1 selector	CIRAM A10: the nametable mirroring	1
74HCT14	Schmitt inverter	PWR_OK: drive nothing into a console that is off	1
74HCT00 / 08 / 32	Gates	The \$5000 decode and the SRAM strobes	2-3
ESP32-S3	WiFi microcontroller	The FujiNet itself, at the other end of the USB link	1

The cartridge's firmware, its schematic, the FujiNet firmware, fujinet-lib, CONFIG, the four games and the programs in this manual are all open source. Their sources are at github.com/FujiNetWIFI.

LIMITED WARRANTY

FUJINET SOFTWARE AND THIS MANUAL

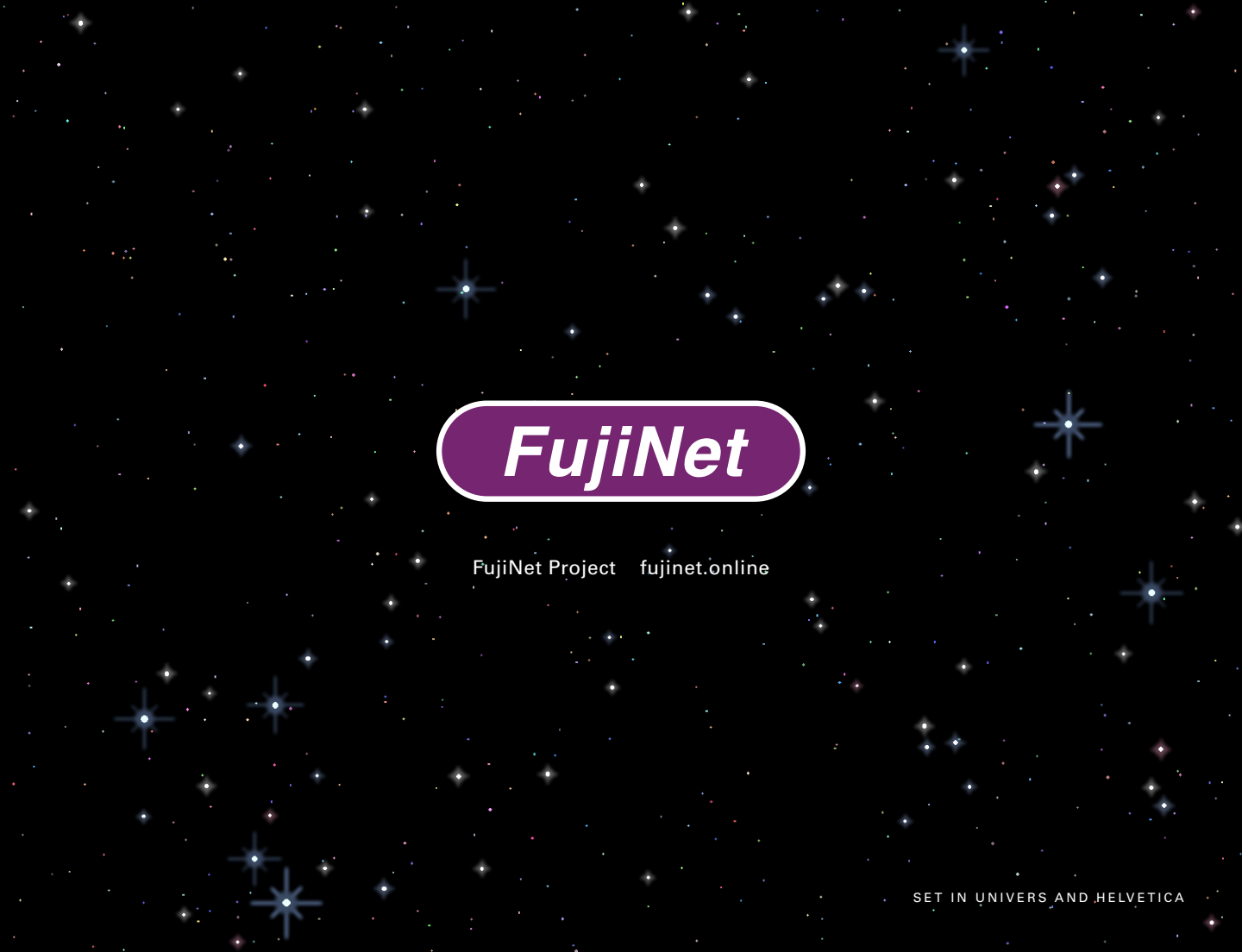
FREE SOFTWARE: The FujiNet firmware, its libraries and the programs printed in this manual are free software. You can redistribute them and/or modify them under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version. The MAME device that models the cartridge is distributed under the BSD 3-Clause license, as MAME requires.

WHAT IS COVERED: Everything in this manual was built and run before it was printed. That is the whole of the promise. The cartridge has been run only in emulation; the authors will be as interested as you are in what a real one does.

HOW TO GET SERVICE: Ask the FujiNet community. Reports of a fault, and better still a fix, are welcome at github.com/FujiNetWIFI, where the sources live.

LIMITATIONS: THIS PROGRAM IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL, BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE GNU GENERAL PUBLIC LICENSE FOR MORE DETAILS. IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MODIFIES AND/OR CONVEYS THE PROGRAM, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE PROGRAM, INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE PROGRAM TO OPERATE WITH ANY OTHER PROGRAMS, EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

TRADEMARKS: Nintendo and Nintendo Entertainment System are trademarks of Nintendo. FujiNet is not affiliated with Nintendo, and this manual is not a Nintendo publication.



FujiNet

FujiNet Project fujinet.online

SET IN UNIVERS AND HELVETICA